

13 | Visão Computacional

Muitas aplicações na área de visão computacional estão intimamente relacionadas às nossas vidas diárias, agora e no futuro, sejam diagnósticos médicos, veículos sem motorista, monitoramento de câmeras ou filtros inteligentes. Nos últimos anos, a tecnologia de aprendizado profundo melhorou muito o desempenho dos sistemas de visão computacional. Pode-se dizer que as aplicações de visão computacional mais avançadas são quase inseparáveis do aprendizado profundo.

Introduzimos modelos de aprendizagem profunda comumente usados na área de visão computacional no capítulo “Redes Neurais Convolucionais” e praticamos tarefas simples de classificação de imagens. Neste capítulo, apresentaremos os métodos de aumento e ajuste fino de imagens e os aplicaremos à classificação de imagens. Em seguida, exploraremos vários métodos de detecção de objetos. Depois disso, aprenderemos como usar redes totalmente convolucionais para realizar segmentação semântica em imagens. Em seguida, explicamos como usar a tecnologia de transferência de estilo para gerar imagens que se parecem com a capa deste livro. Finalmente, realizaremos exercícios práticos em dois importantes conjuntos de dados de visão computacional para revisar o conteúdo deste capítulo e dos capítulos anteriores.

13.1 Aumento de Imagem

Mencionamos que conjuntos de dados em grande escala são pré-requisitos para a aplicação bem-sucedida de redes neurais profundas em [Section 7.1](#). A tecnologia de aumento de imagem expande a escala dos conjuntos de dados de treinamento, fazendo uma série de alterações aleatórias nas imagens de treinamento para produzir exemplos de treinamento semelhantes, mas diferentes. Outra maneira de explicar o aumento de imagem é que exemplos de treinamento que mudam aleatoriamente podem reduzir a dependência de um modelo em certas propriedades, melhorando assim sua capacidade de generalização. Por exemplo, podemos recortar as imagens de diferentes maneiras, para que os objetos de interesse apareçam em diferentes posições, reduzindo a dependência do modelo da posição onde os objetos aparecem. Também podemos ajustar o brilho, a cor e outros fatores para reduzir a sensibilidade do modelo à cor. Pode-se dizer que a tecnologia de aumento de imagem contribuiu muito para o sucesso do AlexNet. Nesta seção, discutiremos essa tecnologia, que é amplamente usada na visão computacional.

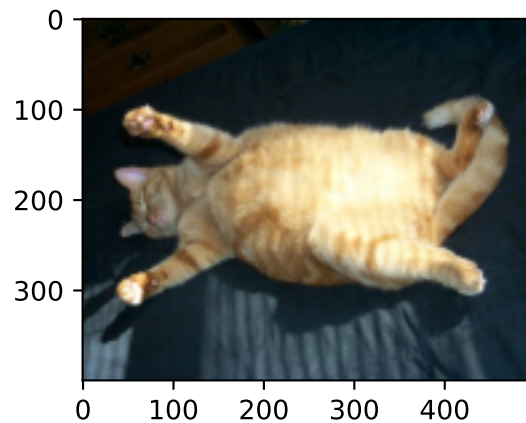
Primeiro, importe os pacotes ou módulos necessários para o experimento nesta seção.

```
%matplotlib inline
import torch
import torchvision
from torch import nn
from d2l import torch as d2l
```

13.1.1 Método Comum de Aumento de Imagem

Neste experimento, usaremos uma imagem com um formato de 400×500 como exemplo.

```
d2l.set_figsize()
img = d2l.Image.open('../img/cat1.jpg')
d2l.plt.imshow(img);
```



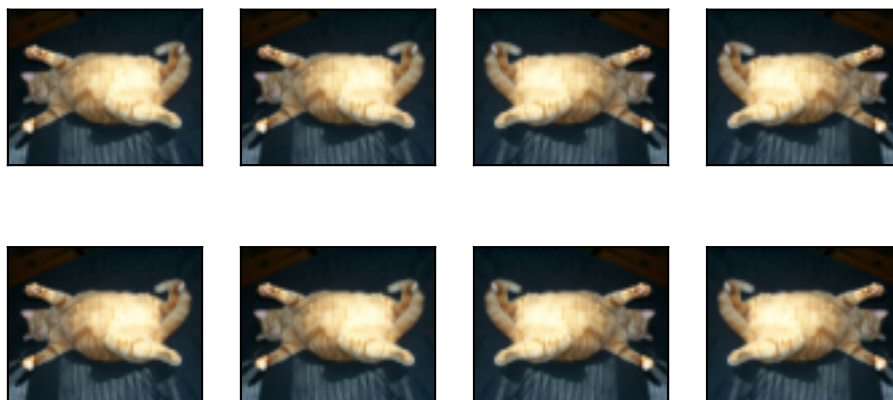
A maioria dos métodos de aumento de imagem tem um certo grau de aleatoriedade. Para facilitar a observação do efeito do aumento da imagem, definimos a seguir a função auxiliar `aplicar`. Esta função executa o método de aumento de imagem `aug` várias vezes na imagem de entrada `img` e mostra todos os resultados.

```
def aplicar(img, aug, num_rows=2, num_cols=4, scale=1.5):
    Y = [aug(img) for _ in range(num_rows * num_cols)]
    d2l.show_images(Y, num_rows, num_cols, scale=scale)
```

Invertendo e Recortando

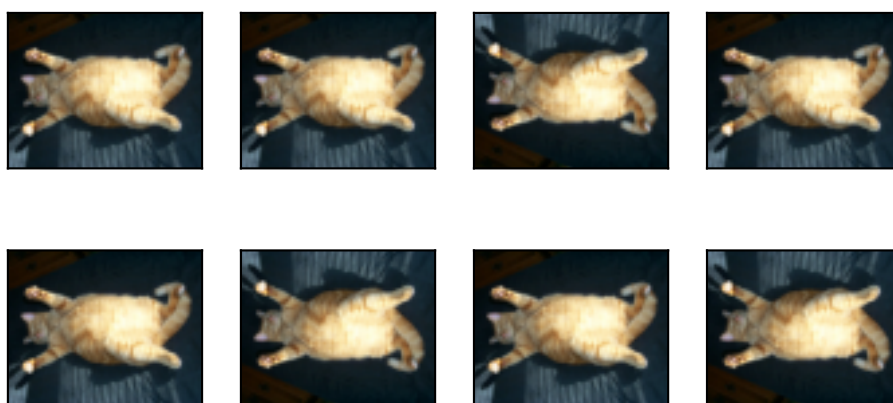
Virar a imagem para a esquerda e para a direita geralmente não altera a categoria do objeto. Este é um dos métodos mais antigos e mais amplamente usados de aumento de imagem. Em seguida, usamos o módulo `transforms` para criar a instância `RandomFlipLeftRight`, que apresenta uma chance de 50% de que a imagem seja virada para a esquerda e para a direita.

```
aplicar(img, torchvision.transforms.RandomHorizontalFlip())
```



Virar para cima e para baixo não é tão comumente usado como girar para a esquerda e para a direita. No entanto, pelo menos para esta imagem de exemplo, virar para cima e para baixo não impede o reconhecimento. Em seguida, criamos uma instância `RandomFlipTopBottom` para uma chance de 50% de virar a imagem para cima e para baixo.

```
apply(img, torchvision.transforms.RandomVerticalFlip())
```



Na imagem de exemplo que usamos, o gato está no meio da imagem, mas pode não ser o caso para todas as imagens. Em [Section 6.5](#), explicamos que a camada de pooling pode reduzir a sensibilidade da camada convolucional ao local de destino. Além disso, podemos fazer os objetos aparecerem em diferentes posições na imagem em diferentes proporções aleatoriamente recortando a imagem. Isso também pode reduzir a sensibilidade do modelo à posição de destino.

No código a seguir, recortamos aleatoriamente uma região com uma área de 10% a 100% da área original, e a proporção entre largura e altura da região é selecionada aleatoriamente entre 0,5 e 2. Em seguida, a largura e a altura de as regiões são dimensionadas para 200 pixels. Salvo indicação em contrário, o número aleatório entre a e b nesta seção refere-se a um valor contínuo obtido por amostragem uniforme no intervalo $[a, b]$.

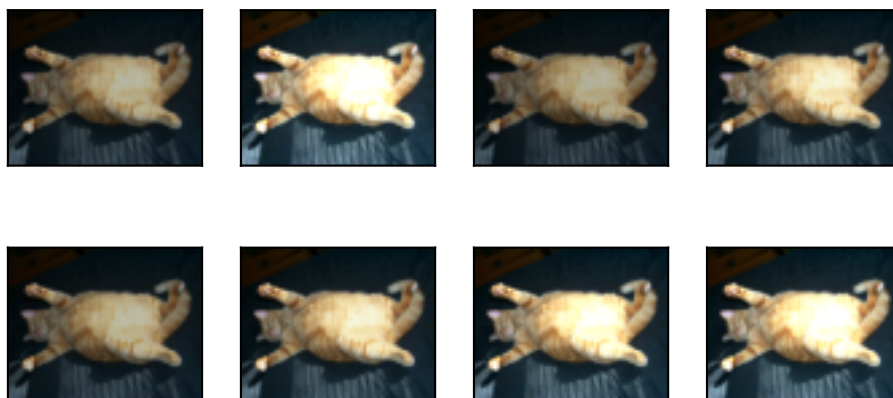
```
shape_aug = torchvision.transforms.RandomResizedCrop(
    (200, 200), scale=(0.1, 1), ratio=(0.5, 2))
apply(img, shape_aug)
```



Mudando a Cor

Outro método de aumento é mudar as cores. Podemos alterar quatro aspectos da cor da imagem: brilho, contraste, saturação e matiz. No exemplo abaixo, alteramos aleatoriamente o brilho da imagem para um valor entre 50% ($1 - 0.5$) e 150% ($1 + 0.5$) da imagem original.

```
apply(img, torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0, saturation=0, hue=0))
```



Da mesma forma, podemos alterar aleatoriamente o matiz da imagem.

```
apply(img, torchvision.transforms.ColorJitter(
    brightness=0, contrast=0, saturation=0, hue=0.5))
```



Também podemos criar uma instância `RandomColorJitter` e definir como alterar aleatoriamente o brightness, contrast, saturation, e hue da imagem ao mesmo tempo.

```
color_aug = torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0.5, saturation=0.5, hue=0.5)
apply(img, color_aug)
```



Métodos de Aumento de Imagem Múltipla Sobreposta

Na prática, iremos sobrepor vários métodos de aumento de imagem. Podemos sobrepor os diferentes métodos de aumento de imagem definidos acima e aplicá-los a cada imagem usando uma instância `Compose`.

```
aug = torchvision.transforms.Compose([
    torchvision.transforms.RandomHorizontalFlip(), color_aug, shape_aug])
apply(img, aug)
```



13.1.2 Usando um Modelo de Treinamento de Aumento de Imagem

A seguir, veremos como aplicar o aumento de imagem no treinamento real. Aqui, usamos o conjunto de dados CIFAR-10, em vez do conjunto de dados Fashion-MNIST que usamos. Isso ocorre porque a posição e o tamanho dos objetos no conjunto de dados Fashion-MNIST foram normalizados, e as diferenças de cor e tamanho dos objetos no conjunto de dados CIFAR-10 são mais significativas. As primeiras 32 imagens de treinamento no conjunto de dados CIFAR-10 são mostradas abaixo.

```
all_images = torchvision.datasets.CIFAR10(train=True, root="../data",
                                         download=True)
d2l.show_images([all_images[i][0] for i in range(32)], 4, 8, scale=0.8);
```

```
Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to ../data/cifar-10-
python.tar.gz
9.6%
```

Para obter resultados definitivos durante a previsão, geralmente aplicamos apenas o aumento da imagem ao exemplo de treinamento e não usamos o aumento da imagem com operações aleatórias durante a previsão. Aqui, usamos apenas o método de inversão aleatório da esquerda para a direita mais simples. Além disso, usamos uma instância ToTensor para converter imagens de minibatch no formato exigido pelo MXNet, ou seja, números de ponto flutuante de 32 bits com a forma de (tamanho do lote, número de canais, altura, largura) e intervalo de valores entre 0 e 1.

```
train_augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor()])

test_augs = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor()])
```

A seguir, definimos uma função auxiliar para facilitar a leitura da imagem e aplicar o aumento da imagem. A função `transform_first` fornecida pelo conjunto de dados do Gluon aplica o aumento da imagem ao primeiro elemento de cada exemplo de treinamento (imagem e rótulo), ou seja, o elemento na parte superior da imagem. Para descrições detalhadas de `DataLoader`, consulte [Section 3.5](#).

```
def load_cifar10(is_train, augs, batch_size):
    dataset = torchvision.datasets.CIFAR10(root="../data", train=is_train,
                                           transform=augs, download=True)
    dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                             shuffle=is_train, num_workers=d2l.get_dataloader_workers())
    return dataloader
```

Usando um Modelo de Treinamento Multi-GPU

Treinamos o modelo ResNet-18 descrito em: numref: sec_resnet no conjunto de dados CIFAR-10. Também aplicaremos os métodos descritos em [Section 12.6](#) e usaremos um modelo de treinamento multi-GPU.

Em seguida, definimos a função de treinamento para treinar e avaliar o modelo usando várias GPUs.

```
#@save
def train_batch_ch13(net, X, y, loss, trainer, devices):
    if isinstance(X, list):
        # Required for BERT Fine-tuning (to be covered later)
        X = [x.to(devices[0]) for x in X]
    else:
        X = X.to(devices[0])
    y = y.to(devices[0])
    net.train()
    trainer.zero_grad()
    pred = net(X)
    l = loss(pred, y)
    l.sum().backward()
    trainer.step()
    train_loss_sum = l.sum()
    train_acc_sum = d2l.accuracy(pred, y)
    return train_loss_sum, train_acc_sum
```

```
#@save
def train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
               devices=d2l.try_all_gpus()):
    timer, num_batches = d2l.Timer(), len(train_iter)
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0, 1],
                           legend=['train loss', 'train acc', 'test acc'])
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        # Store training_loss, training_accuracy, num_examples, num_features
        metric = d2l.Accumulator(4)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = train_batch_ch13(
                net, features, labels, loss, trainer, devices)
            metric.add(l, acc, labels.shape[0], labels.numel())
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[2], metric[1] / metric[3],
```

(continues on next page)


```

        None))
    test_acc = d2l.evaluate_accuracy_gpu(net, test_iter)
    animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {metric[0] / metric[2]:.3f}, train acc '
          f'{metric[1] / metric[3]:.3f}, test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec on '
          f'{str(devices)}')

```

Agora, podemos definir a função `train_with_data_aug` para usar o aumento da imagem para treinar o modelo. Esta função obtém todas as GPUs disponíveis e usa Adam como o algoritmo de otimização para o treinamento. Em seguida, ele aplica o aumento da imagem ao conjunto de dados de treinamento e, finalmente, chama a função `train_ch13` definida para treinar e avaliar o modelo.

```

batch_size, devices, net = 256, d2l.try_all_gpus(), d2l.resnet18(10, 3)

def init_weights(m):
    if type(m) in [nn.Linear, nn.Conv2d]:
        nn.init.xavier_uniform_(m.weight)

net.apply(init_weights)

def train_with_data_aug(train_augs, test_augs, net, lr=0.001):
    train_iter = load_cifar10(True, train_augs, batch_size)
    test_iter = load_cifar10(False, test_augs, batch_size)
    loss = nn.CrossEntropyLoss(reduction="none")
    trainer = torch.optim.Adam(net.parameters(), lr=lr)
    train_ch13(net, train_iter, test_iter, loss, trainer, 10, devices)

```

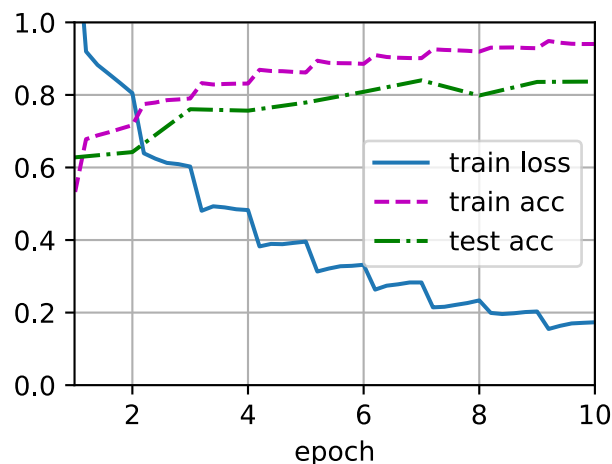
Now we train the model using image augmentation of random flipping left and right.

```
train_with_data_aug(train_augs, test_augs, net)
```

```

loss 0.173, train acc 0.940, test acc 0.837
5044.9 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]

```



13.1.3 Resumo

- O aumento de imagem gera imagens aleatórias com base nos dados de treinamento existentes para lidar com o sobreajuste.
- Para obter resultados definitivos durante a previsão, geralmente aplicamos apenas o aumento da imagem ao exemplo de treinamento e não usamos o aumento da imagem com operações aleatórias durante a previsão.
- Podemos obter classes relacionadas ao aumento de imagem do módulo `transforms` do Gluon.

13.1.4 Exercícios

1. Treine o modelo sem usar aumento de imagem: `train_with_data_aug (no_aug, no_aug)`. Compare a precisão do treinamento e do teste ao usar e não usar o aumento de imagem. Este experimento comparativo pode apoiar o argumento de que o aumento da imagem pode mitigar o sobreajuste? Por quê?
2. Adicione diferentes métodos de aumento de imagem no treinamento do modelo com base no *dataset* CIFAR-10. Observe os resultados da implementação.
3. Com referência à documentação do MXNet, que outros métodos de aumento de imagem são fornecidos no módulo `transforms` do Gluon?

Discussões¹⁵⁵

13.2 Ajustes

Nos capítulos anteriores, discutimos como treinar modelos no conjunto de dados de treinamento Fashion-MNIST, que tem apenas 60.000 imagens. Também descrevemos o ImageNet, o conjunto de dados de imagens em grande escala mais usado no mundo acadêmico, com mais de 10 milhões de imagens e objetos de mais de 1000 categorias. No entanto, o tamanho dos conjuntos de dados com os quais frequentemente lidamos é geralmente maior do que o primeiro, mas menor do que o segundo.

Suponha que queremos identificar diferentes tipos de cadeiras nas imagens e, em seguida, enviar o link de compra para o usuário. Um método possível é primeiro encontrar cem cadeiras comuns, obter mil imagens diferentes com diferentes ângulos para cada cadeira e, em seguida, treinar um modelo de classificação no conjunto de dados de imagens coletado. Embora esse conjunto de dados possa ser maior do que o Fashion-MNIST, o número de exemplos ainda é menor que um décimo do ImageNet. Isso pode resultar em sobreajuste do modelo complicado aplicável ao ImageNet neste conjunto de dados. Ao mesmo tempo, devido à quantidade limitada de dados, a precisão do modelo final treinado pode não atender aos requisitos práticos.

Para lidar com os problemas acima, uma solução óbvia é coletar mais dados. No entanto, coletar e rotular dados pode consumir muito tempo e dinheiro. Por exemplo, para coletar os conjuntos de dados ImageNet, os pesquisadores gastaram milhões de dólares em financiamento de pesquisa. Embora, recentemente, os custos de coleta de dados tenham caído significativamente, os custos ainda não podem ser ignorados.

¹⁵⁵ <https://discuss.d2l.ai/t/1404>

Outra solução é aplicar o aprendizado de transferência para migrar o conhecimento aprendido do conjunto de dados de origem para o conjunto de dados de destino. Por exemplo, embora as imagens no ImageNet não tenham relação com cadeiras, os modelos treinados neste conjunto de dados podem extrair recursos de imagem mais gerais que podem ajudar a identificar bordas, texturas, formas e composição de objetos. Esses recursos semelhantes podem ser igualmente eficazes para o reconhecimento de uma cadeira.

Nesta seção, apresentamos uma técnica comum no aprendizado por transferência: o ajuste fino. Conforme mostrado em :numref:fig_finetune, o ajuste fino consiste nas quatro etapas a seguir:

1. Pré-treine um modelo de rede neural, ou seja, o modelo de origem, em um conjunto de dados de origem (por exemplo, o conjunto de dados ImageNet).
2. Crie um novo modelo de rede neural, ou seja, o modelo de destino. Isso replica todos os designs de modelo e seus parâmetros no modelo de origem, exceto a camada de saída. Assumimos que esses parâmetros do modelo contêm o conhecimento aprendido com o conjunto de dados de origem e esse conhecimento será igualmente aplicável ao conjunto de dados de destino. Também assumimos que a camada de saída do modelo de origem está intimamente relacionada aos rótulos do conjunto de dados de origem e, portanto, não é usada no modelo de destino.
3. Adicione uma camada de saída cujo tamanho de saída é o número de categorias de conjunto de dados de destino ao modelo de destino e inicialize aleatoriamente os parâmetros do modelo desta camada.
4. Treine o modelo de destino em um conjunto de dados de destino, como um conjunto de dados de cadeira. Vamos treinar a camada de saída do zero, enquanto os parâmetros de todas as camadas restantes são ajustados com base nos parâmetros do modelo de origem.

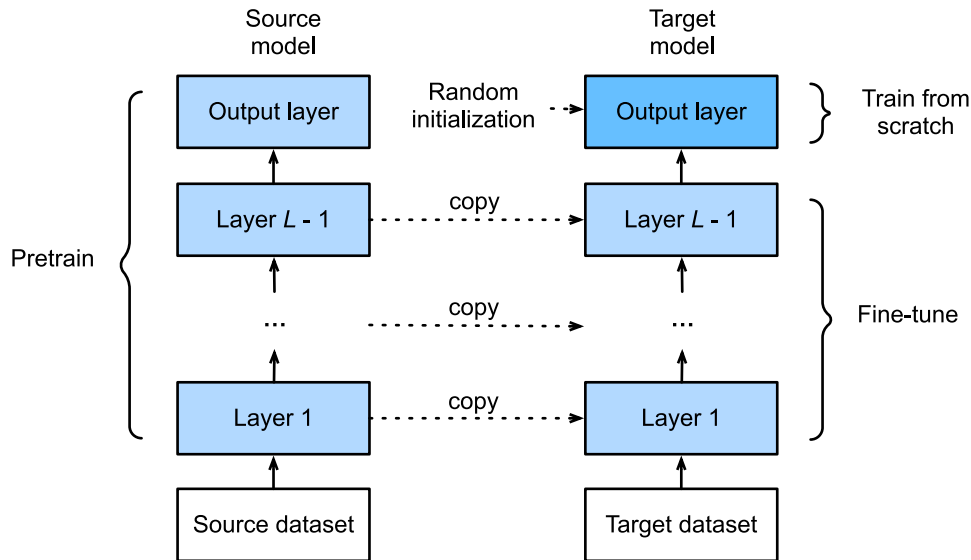


Fig. 13.2.1: Fine tuning.

13.2.1 Reconhecimento de Cachorro-quente

A seguir, usaremos um exemplo específico para prática: reconhecimento de cachorro-quente. Faremos o ajuste fino do modelo ResNet treinado no conjunto de dados ImageNet com base em um pequeno conjunto de dados. Este pequeno conjunto de dados contém milhares de imagens, algumas das quais contêm cachorros-quentes. Usaremos o modelo obtido pelo ajuste fino para identificar se uma imagem contém cachorro-quente.

Primeiro, importe os pacotes e módulos necessários para o experimento. O pacote `model_zoo` do Gluon fornece um modelo comum pré-treinado. Se você deseja obter mais modelos pré-treinados para visão computacional, você pode usar o [GluonCV Toolkit](https://gluon-cv.mxnet.io)¹⁵⁶.

```
%matplotlib inline
import os
import torch
import torchvision
from torch import nn
from d2l import torch as d2l
```

Obtendo o Dataset

O conjunto de dados de cachorro-quente que usamos foi obtido de imagens online e contém 1.400 imagens positivas contendo cachorros-quentes e o mesmo número de imagens negativas contendo outros alimentos. 1,000 imagens de várias classes são usadas para treinamento e o resto é usado para teste.

Primeiro, baixamos o conjunto de dados compactado e obtemos duas pastas `hotdog/train` e `hotdog/test`. Ambas as pastas têm subpastas das categorias `hotdog` e `not-hotdog`, cada uma com arquivos de imagem correspondentes.

```
#@save
d2l.DATA_HUB['hotdog'] = (d2l.DATA_URL+'hotdog.zip',
                        'fba480ffa8aa7e0febbb511d181409f899b9baa5')

data_dir = d2l.download_extract('hotdog')
```

Downloading ../data/hotdog.zip from <http://d2l-data.s3-accelerate.amazonaws.com/hotdog.zip>...

Criamos duas instâncias `ImageFolderDataset` para ler todos os arquivos de imagem no conjunto de dados de treinamento e teste, respectivamente.

```
train_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'train'))
test_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'test'))
```

Os primeiros 8 exemplos positivos e as últimas 8 imagens negativas são mostrados abaixo. Como você pode ver, as imagens variam em tamanho e proporção.

```
hotdogs = [train_imgs[i][0] for i in range(8)]
not_hotdogs = [train_imgs[-i - 1][0] for i in range(8)]
d2l.show_images(hotdogs + not_hotdogs, 2, 8, scale=1.4);
```

¹⁵⁶ <https://gluon-cv.mxnet.io>



Durante o treinamento, primeiro recortamos uma área aleatória com tamanho e proporção aleatória da imagem e, em seguida, dimensionamos a área para uma entrada com altura e largura de 224 pixels. Durante o teste, dimensionamos a altura e a largura das imagens para 256 pixels e, em seguida, recortamos a área central com altura e largura de 224 pixels para usar como entrada. Além disso, normalizamos os valores dos três canais de cores RGB (vermelho, verde e azul). A média de todos os valores do canal é subtraída de cada valor e o resultado é dividido pelo desvio padrão de todos os valores do canal para produzir a saída.

```
# We specify the mean and variance of the three RGB channels to normalize the
# image channel
normalize = torchvision.transforms.Normalize(
    [0.485, 0.456, 0.406], [0.229, 0.224, 0.225])

train_augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomResizedCrop(224),
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor(),
    normalize])

test_augs = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
    normalize])
```

Definindo e Inicializando o Modelo

Usamos o ResNet-18, que foi pré-treinado no conjunto de dados ImageNet, como modelo de origem. Aqui, especificamos `pretrained = True` para baixar e carregar automaticamente os parâmetros do modelo pré-treinado. Na primeira vez em que são usados, os parâmetros do modelo precisam ser baixados da Internet.

```
pretrained_net = torchvision.models.resnet18(pretrained=True)
```

A instância do modelo de origem pré-treinada contém duas variáveis de membro: `features` e `output`. O primeiro contém todas as camadas do modelo, exceto a camada de saída, e o último é a camada de saída do modelo. O principal objetivo desta divisão é facilitar o ajuste fino dos parâmetros do modelo de todas as camadas, exceto a camada de saída. A variável membro `output` do modelo de origem é fornecida abaixo. Como uma camada totalmente conectada, ele transforma a saída final da camada de agrupamento média global do ResNet em uma saída de 1000 classes no conjunto de dados ImageNet.

```
pretrained_net.fc
```

```
Linear(in_features=512, out_features=1000, bias=True)
```

Em seguida, construímos uma nova rede neural para usar como modelo-alvo. Ele é definido da mesma forma que o modelo de origem pré-treinado, mas o número final de saídas é igual ao número de categorias no conjunto de dados de destino. No código abaixo, os parâmetros do modelo na variável membro `features` da instância do modelo de destino `finetune_net` são inicializados para modelar os parâmetros da camada correspondente do modelo de origem. Como os parâmetros do modelo em `features` são obtidos por pré-treinamento no conjunto de dados ImageNet, é bom o suficiente. Portanto, geralmente só precisamos usar pequenas taxas de aprendizado para “ajustar” esses parâmetros. Em contraste, os parâmetros do modelo na variável membro `output` são inicializados aleatoriamente e geralmente requerem uma taxa de aprendizado maior para aprender do zero. Suponha que a taxa de aprendizado na instância `Trainer` seja η e use uma taxa de aprendizado de 10η para atualizar os parâmetros do modelo na variável membro `output`.

```
finetune_net = torchvision.models.resnet18(pretrained=True)
finetune_net.fc = nn.Linear(finetune_net.fc.in_features, 2)
nn.init.xavier_uniform_(finetune_net.fc.weight);
# If 'param_group=True', the model parameters in fc layer will be updated
# using a learning rate ten times greater, defined in the trainer.
```

Ajustando o Modelo

Primeiro definimos uma função de treinamento `train_fine_tuning` que usa ajuste fino para que possa ser chamada várias vezes.

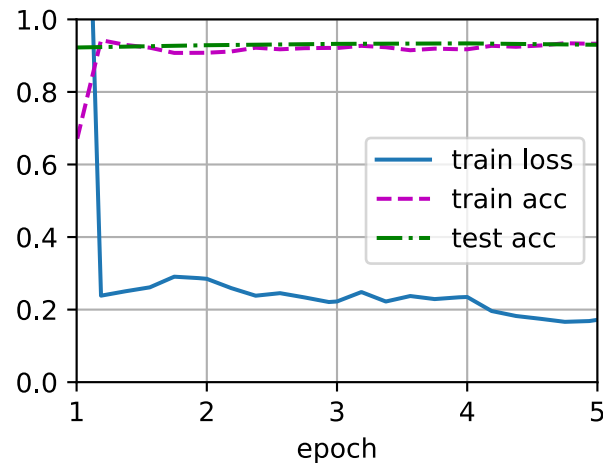
```
def train_fine_tuning(net, learning_rate, batch_size=128, num_epochs=5,
                      param_group=True):
    train_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
        os.path.join(data_dir, 'train'), transform=train_augs),
        batch_size=batch_size, shuffle=True)
    test_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
        os.path.join(data_dir, 'test'), transform=test_augs),
        batch_size=batch_size)
    devices = d2l.try_all_gpus()
    loss = nn.CrossEntropyLoss(reduction="none")
    if param_group:
        params_1x = [param for name, param in net.named_parameters()
                     if name not in ["fc.weight", "fc.bias"]]
        trainer = torch.optim.SGD([{'params': params_1x},
                                    {'params': net.fc.parameters(),
                                     'lr': learning_rate * 10}],
                                   lr=learning_rate, weight_decay=0.001)
    else:
        trainer = torch.optim.SGD(net.parameters(), lr=learning_rate,
                                   weight_decay=0.001)
    d2l.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
                  devices)
```

Definimos a taxa de aprendizado na instância `Trainer` para um valor menor, como 0,01, a fim de ajustar os parâmetros do modelo obtidos no pré-treinamento. Com base nas configurações

anteriores, treinaremos os parâmetros da camada de saída do modelo de destino do zero, usando uma taxa de aprendizado dez vezes maior.

```
train_fine_tuning(finetune_net, 5e-5)
```

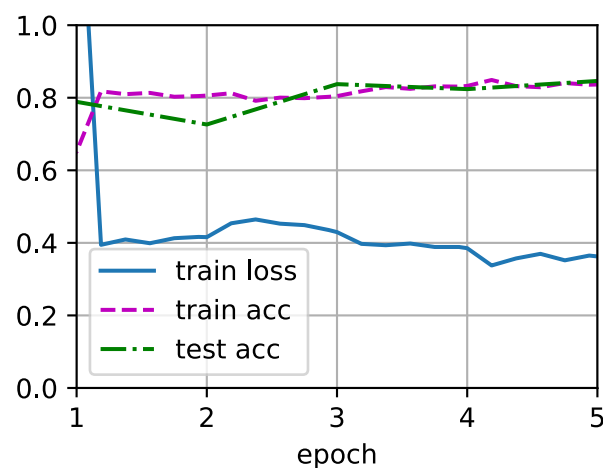
```
loss 0.172, train acc 0.933, test acc 0.930  
827.3 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



Para comparação, definimos um modelo idêntico, mas inicializamos todos os seus parâmetros de modelo para valores aleatórios. Como todo o modelo precisa ser treinado do zero, podemos usar uma taxa de aprendizado maior.

```
scratch_net = torchvision.models.resnet18()  
scratch_net.fc = nn.Linear(scratch_net.fc.in_features, 2)  
train_fine_tuning(scratch_net, 5e-4, param_group=False)
```

```
loss 0.363, train acc 0.837, test acc 0.846  
1613.8 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



Como você pode ver, o modelo ajustado tende a obter maior precisão na mesma época porque os

valores iniciais dos parâmetros são melhores.

13.2.2 Resumo

- A aprendizagem de transferência migra o conhecimento aprendido do conjunto de dados de origem para o conjunto de dados de destino. O ajuste fino é uma técnica comum para a aprendizagem por transferência.
- O modelo de destino replica todos os designs de modelo e seus parâmetros no modelo de origem, exceto a camada de saída, e ajusta esses parâmetros com base no conjunto de dados de destino. Em contraste, a camada de saída do modelo de destino precisa ser treinada do zero.
- Geralmente, os parâmetros de ajuste fino usam uma taxa de aprendizado menor, enquanto o treinamento da camada de saída do zero pode usar uma taxa de aprendizado maior.

13.2.3 Exercícios

1. Continue aumentando a taxa de aprendizado de `finetune_net`. Como a precisão do modelo muda?
2. Ajuste ainda mais os hiperparâmetros de `finetune_net` e `scratch_net` no experimento comparativo. Eles ainda têm precisões diferentes?
3. Defina os parâmetros em `finetune_net.features` para os parâmetros do modelo de origem e não os atualize durante o treinamento. O que vai acontecer? Você pode usar o seguinte código.

```
for param in finetune_net.parameters():  
    param.requires_grad = False
```

4. Na verdade, também existe uma classe “hotdog” no conjunto de dados ImageNet. Seu parâmetro de peso correspondente na camada de saída pode ser obtido usando o código a seguir. Como podemos usar este parâmetro?

```
weight = pretrained_net.fc.weight  
hotdog_w = torch.split(weight.data, 1, dim=0)[713]  
hotdog_w.shape
```

```
torch.Size([1, 512])
```

Discussões¹⁵⁷

¹⁵⁷ <https://discuss.d2l.ai/t/1439>

13.3 Detecção de Objetos e Caixas Delimitadoras

Na seção anterior, apresentamos muitos modelos para classificação de imagens. Nas tarefas de classificação de imagens, presumimos que haja apenas uma característica principal na imagem e nos concentramos apenas em como identificar a categoria de destino. No entanto, em muitas situações, existem várias características na imagem que nos interessam. Não queremos apenas classificá-las, mas também queremos obter suas posições específicas na imagem. Na visão computacional, nos referimos a tarefas como detecção de objetos (ou reconhecimento de objetos).

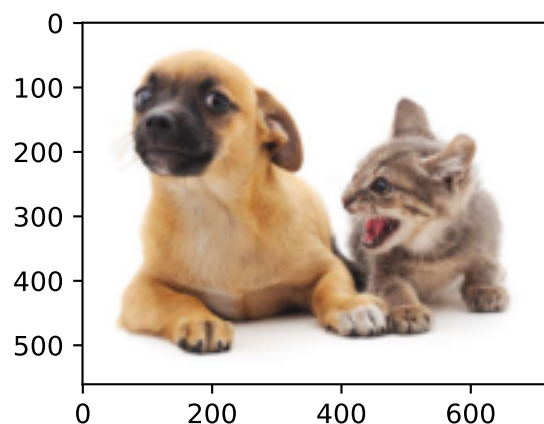
A detecção de objetos é amplamente usada em muitos campos. Por exemplo, na tecnologia de direção autônoma, precisamos planejar rotas identificando a localização de veículos, pedestres, estradas e obstáculos na imagem de vídeo capturada. Os robôs geralmente realizam esse tipo de tarefa para detectar alvos de interesse. Os sistemas no campo da segurança precisam detectar alvos anormais, como intrusos ou bombas.

Nas próximas seções, apresentaremos vários modelos de aprendizado profundo usados para detecção de objetos. Antes disso, devemos discutir o conceito de localização de destino. Primeiro, importe os pacotes e módulos necessários para o experimento.

```
%matplotlib inline
import torch
from d2l import torch as d2l
```

A seguir, carregaremos as imagens de amostra que serão usadas nesta seção. Podemos ver que há um cachorro no lado esquerdo da imagem e um gato no lado direito. Eles são os dois alvos principais desta imagem.

```
d2l.set_figsize()
img = d2l.plt.imread('../img/catdog.jpg')
d2l.plt.imshow(img);
```



13.3.1 Caixa Delimitadora

Na detecção de objetos, geralmente usamos uma caixa delimitadora para descrever o local de destino. A caixa delimitadora é uma caixa retangular que pode ser determinada pelas coordenadas dos eixos x e y no canto superior esquerdo e pelas coordenadas dos eixos x e y no canto inferior direito. Outra representação de caixa delimitadora comumente usada são as coordenadas dos eixos x e y do centro da caixa delimitadora e sua largura e altura. Aqui definimos funções para converter entre essas duas representações, `box_corner_to_center` converte da representação de dois cantos para a apresentação centro-largura-altura, e vice-versa `box_center_to_corner`. O argumento de entrada `boxes` pode ter um tensor de comprimento 4, ou um tensor $(N, 4)$ 2-dimensional.

```
#@save
def box_corner_to_center(boxes):
    """Convert from (upper_left, bottom_right) to (center, width, height)"""
    x1, y1, x2, y2 = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2
    w = x2 - x1
    h = y2 - y1
    boxes = torch.stack((cx, cy, w, h), axis=-1)
    return boxes

#@save
def box_center_to_corner(boxes):
    """Convert from (center, width, height) to (upper_left, bottom_right)"""
    cx, cy, w, h = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    x1 = cx - 0.5 * w
    y1 = cy - 0.5 * h
    x2 = cx + 0.5 * w
    y2 = cy + 0.5 * h
    boxes = torch.stack((x1, y1, x2, y2), axis=-1)
    return boxes
```

Vamos definir as caixas delimitadoras do cão e do gato na imagem baseada nas informações de coordenadas. A origem das coordenadas na imagem é o canto superior esquerdo da imagem, e para a direita e para baixo estão as direções positivas do eixo x e do eixo y , respectivamente.

```
# bbox is the abbreviation for bounding box
dog_bbox, cat_bbox = [60.0, 45.0, 378.0, 516.0], [400.0, 112.0, 655.0, 493.0]
```

Podemos verificar a exatidão das funções de conversão da caixa convertendo duas vezes.

```
boxes = torch.tensor((dog_bbox, cat_bbox))
box_center_to_corner(box_corner_to_center(boxes)) - boxes
```

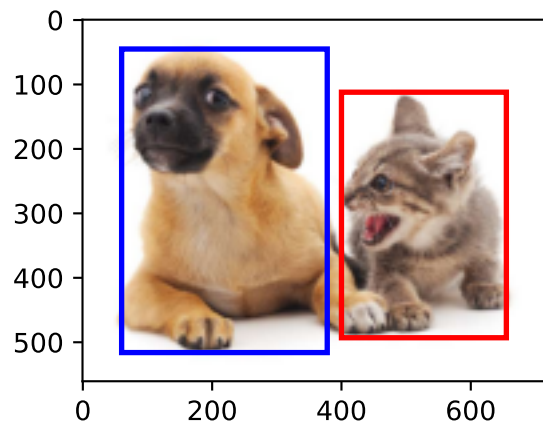
```
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.]])
```

Podemos desenhar a caixa delimitadora na imagem para verificar se ela é precisa. Antes de desenhar a caixa, definiremos uma função auxiliar `bbox_to_rect`. Ele representa a caixa delimitadora no formato de caixa delimitadora de `matplotlib`.

```
#@save
def bbox_to_rect(bbox, color):
    """Convert bounding box to matplotlib format."""
    # Convert the bounding box (top-left x, top-left y, bottom-right x,
    # bottom-right y) format to matplotlib format: ((upper-left x,
    # upper-left y), width, height)
    return d2l.plt.Rectangle(
        xy=(bbox[0], bbox[1]), width=bbox[2]-bbox[0], height=bbox[3]-bbox[1],
        fill=False, edgecolor=color, linewidth=2)
```

Depois de carregar a caixa delimitadora na imagem, podemos ver que o contorno principal do alvo está basicamente dentro da caixa.

```
fig = d2l.plt.imshow(img)
fig.axes.add_patch(bbox_to_rect(dog_bbox, 'blue'))
fig.axes.add_patch(bbox_to_rect(cat_bbox, 'red'));
```



13.3.2 Resumo

- Na detecção de objetos, não precisamos apenas identificar todos os objetos de interesse na imagem, mas também suas posições. As posições são geralmente representadas por uma caixa delimitadora retangular.

13.3.3 Exercícios

1. Encontre algumas imagens e tente rotular uma caixa delimitadora que contém o alvo. Compare a diferença entre o tempo que leva para rotular a caixa delimitadora e rotular a categoria.

Discussões¹⁵⁸

¹⁵⁸ <https://discuss.d2l.ai/t/1527>

13.4 Caixas de Âncora

Os algoritmos de detecção de objetos geralmente amostram um grande número de regiões na imagem de entrada, determinam se essas regiões contêm objetos de interesse e ajustam as bordas das regiões de modo a prever a caixa delimitadora da verdade terrestre do alvo com mais precisão. Diferentes modelos podem usar diferentes métodos de amostragem de região. Aqui, apresentamos um desses métodos: ele gera várias caixas delimitadoras com diferentes tamanhos e proporções de aspecto, enquanto é centralizado em cada pixel. Essas caixas delimitadoras são chamadas de caixas de âncora. Praticaremos a detecção de objetos com base em caixas de âncora nas seções a seguir.

Primeiro, importe os pacotes ou módulos necessários para esta seção. Aqui, modificamos a precisão de impressão do PyTorch. Como os tensores de impressão, na verdade, chamam a função de impressão de PyTorch, os números de ponto flutuante nos tensores impressos nesta seção são mais concisos.

```
%matplotlib inline
import torch
from d2l import torch as d2l

torch.set_printoptions(2)
```

13.4.1 Gerando Várias Caixas de Âncora

Suponha que a imagem de entrada tenha uma altura de h e uma largura de w . Geramos caixas de âncora com diferentes formas centralizadas em cada pixel da imagem. Suponha que o tamanho seja $s \in (0, 1]$, a proporção da imagem é $r > 0$ e a largura e a altura da caixa de âncora são $ws\sqrt{r}$ e $hs/\sqrt{r}$, respectivamente. Quando a posição central é fornecida, uma caixa de âncora com largura e altura conhecidas é determinada.

Abaixo, definimos um conjunto de tamanhos $s_1, \dots, s_n$ e um conjunto de relações de aspecto $r_1, \dots, r_m$. Se usarmos uma combinação de todos os tamanhos e proporções com cada pixel como o centro, a imagem de entrada terá um total de $whnm$ caixas de âncora. Embora essas caixas de âncora possam abranger todas as caixas delimitadoras da verdade, a complexidade computacional costuma ser excessiva. Portanto, normalmente estamos interessados apenas em uma combinação contendo s_1 ou r_1 tamanhos e proporções, isto é:

$$(s_1, r_1), (s_1, r_2), \dots, (s_1, r_m), (s_2, r_1), (s_3, r_1), \dots, (s_n, r_1). \quad (13.4.1)$$

Ou seja, o número de caixas de âncora centradas no mesmo pixel é $n + m - 1$. Para toda a imagem de entrada, geraremos um total de $wh(n + m - 1)$ caixas de âncora.

O método acima para gerar caixas de âncora foi implementado na função `multibox_prior`. Especificamos a entrada, um conjunto de tamanhos e um conjunto de proporções, e esta função retornará todas as caixas de âncora inseridas.

```
#@save
def multibox_prior(data, sizes, ratios):
    in_height, in_width = data.shape[-2:]
    device, num_sizes, num_ratios = data.device, len(sizes), len(ratios)
    boxes_per_pixel = (num_sizes + num_ratios - 1)
```

(continues on next page)

```

size_tensor = torch.tensor(sizes, device=device)
ratio_tensor = torch.tensor(ratios, device=device)
# Offsets are required to move the anchor to center of a pixel
# Since pixel (height=1, width=1), we choose to offset our centers by 0.5
offset_h, offset_w = 0.5, 0.5
steps_h = 1.0 / in_height # Scaled steps in y axis
steps_w = 1.0 / in_width  # Scaled steps in x axis

# Generate all center points for the anchor boxes
center_h = (torch.arange(in_height, device=device) + offset_h) * steps_h
center_w = (torch.arange(in_width, device=device) + offset_w) * steps_w
shift_y, shift_x = torch.meshgrid(center_h, center_w)
shift_y, shift_x = shift_y.reshape(-1), shift_x.reshape(-1)

# Generate boxes_per_pixel number of heights and widths which are later
# used to create anchor box corner coordinates (xmin, xmax, ymin, ymax)
# cat (various sizes, first ratio) and (first size, various ratios)
w = torch.cat((size_tensor * torch.sqrt(ratio_tensor[0]),
               sizes[0] * torch.sqrt(ratio_tensor[1:])) \
               * in_height / in_width # handle rectangular inputs
h = torch.cat((size_tensor / torch.sqrt(ratio_tensor[0]),
               sizes[0] / torch.sqrt(ratio_tensor[1:]))
# Divide by 2 to get half height and half width
anchor_manipulations = torch.stack((-w, -h, w, h)).T.repeat(
                                in_height * in_width, 1) / 2

# Each center point will have boxes_per_pixel number of anchor boxes, so
# generate grid of all anchor box centers with boxes_per_pixel repeats
out_grid = torch.stack([shift_x, shift_y, shift_x, shift_y],
                        dim=1).repeat_interleave(boxes_per_pixel, dim=0)

output = out_grid + anchor_manipulations
return output.unsqueeze(0)

```

Podemos ver que a forma da variável de caixa de âncora retornada y é (tamanho do lote, número de caixas de âncora, 4).

```

img = d2l.plt.imread('../img/catdog.jpg')
h, w = img.shape[0:2]

print(h, w)
X = torch.rand(size=(1, 3, h, w)) # Construct input data
Y = multibox_prior(X, sizes=[0.75, 0.5, 0.25], ratios=[1, 2, 0.5])
Y.shape

```

```
561 728
```

```
torch.Size([1, 2042040, 4])
```

Depois de alterar a forma da variável da caixa de âncora y para (altura da imagem, largura da imagem, número de caixas de âncora centradas no mesmo pixel, 4), podemos obter todas as caixas de âncora centradas em uma posição de pixel especificada. No exemplo a seguir, acessamos a primeira caixa de âncora centrada em (250, 250). Ele tem quatro elementos: as coordenadas do

eixo x, y no canto superior esquerdo e as coordenadas do eixo x, y no canto inferior direito da caixa de âncora. Os valores das coordenadas dos eixos x e y são divididos pela largura e altura da imagem, respectivamente, portanto, o intervalo de valores está entre 0 e 1.

```
boxes = Y.reshape(h, w, 5, 4)
boxes[250, 250, 0, :]
```

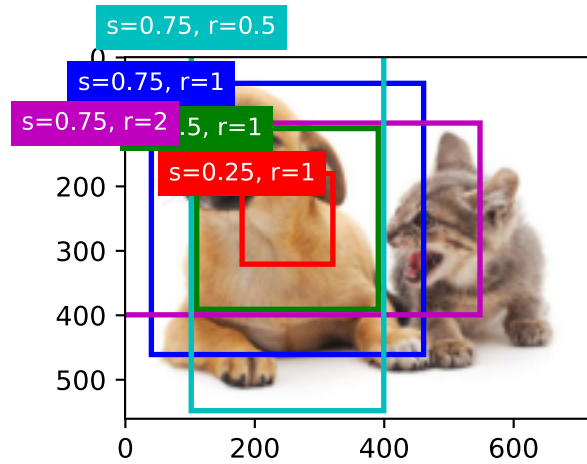
```
tensor([0.06, 0.07, 0.63, 0.82])
```

Para descrever todas as caixas de âncora centralizadas em um pixel na imagem, primeiro definimos a função `show_bboxes` para desenhar várias caixas delimitadoras na imagem.

```
@save
def show_bboxes(axes, bboxes, labels=None, colors=None):
    """Show bounding boxes."""
    def _make_list(obj, default_values=None):
        if obj is None:
            obj = default_values
        elif not isinstance(obj, (list, tuple)):
            obj = [obj]
        return obj
    labels = _make_list(labels)
    colors = _make_list(colors, ['b', 'g', 'r', 'm', 'c'])
    for i, bbox in enumerate(bboxes):
        color = colors[i % len(colors)]
        rect = d2l.bbox_to_rect(bbox.detach().numpy(), color)
        axes.add_patch(rect)
        if labels and len(labels) > i:
            text_color = 'k' if color == 'w' else 'w'
            axes.text(rect.xy[0], rect.xy[1], labels[i],
                      va='center', ha='center', fontsize=9, color=text_color,
                      bbox=dict(facecolor=color, lw=0))
```

Como acabamos de ver, os valores das coordenadas dos eixos x e y na variável caixas foram divididos pela largura e altura da imagem, respectivamente. Ao desenhar imagens, precisamos restaurar os valores das coordenadas originais das caixas de âncora e, portanto, definir a variável `bbox_scale`. Agora, podemos desenhar todas as caixas de âncora centralizadas em (250, 250) na imagem. Como você pode ver, a caixa de âncora azul com um tamanho de 0,75 e uma proporção de 1 cobre bem o cão na imagem.

```
d2l.set_figsize()
bbox_scale = torch.tensor((w, h, w, h))
fig = d2l.plt.imshow(img)
show_bboxes(fig.axes, boxes[250, 250, :, :] * bbox_scale,
            ['s=0.75, r=1', 's=0.5, r=1', 's=0.25, r=1', 's=0.75, r=2',
             's=0.75, r=0.5'])
```



13.4.2 Interseção sobre União

Acabamos de mencionar que a caixa de âncora cobre bem o cachorro na imagem. Se a caixa delimitadora da verdade básica do alvo é conhecida, como o “bem” pode ser quantificado aqui? Um método intuitivo é medir a semelhança entre as caixas de âncora e a caixa delimitadora da verdade absoluta. Sabemos que o índice de Jaccard pode medir a semelhança entre dois conjuntos. Dados os conjuntos $\mathcal{A}$ e $\mathcal{B}$, seu índice de Jaccard é o tamanho de sua interseção dividido pelo tamanho de sua união:

$$J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|}. \quad (13.4.2)$$

Na verdade, podemos considerar a área de pixels de uma caixa delimitadora como uma coleção de pixels. Dessa forma, podemos medir a similaridade das duas caixas delimitadoras pelo índice de Jaccard de seus conjuntos de pixels. Quando medimos a similaridade de duas caixas delimitadoras, geralmente nos referimos ao índice de Jaccard como interseção sobre união (IoU), que é a razão entre a área de interseção e a área de união das duas caixas delimitadoras, conforme mostrado em Fig. 13.4.1. O intervalo de valores de IoU está entre 0 e 1: 0 significa que não há pixels sobrepostos entre as duas caixas delimitadoras, enquanto 1 indica que as duas caixas delimitadoras são iguais.

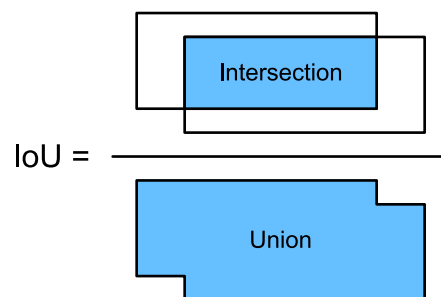


Fig. 13.4.1: IoU é a razão entre a área de interseção e a área de união de duas caixas delimitadoras.

Para o restante desta seção, usaremos IoU para medir a semelhança entre as caixas de âncora e as caixas delimitadoras de verdade terrestre e entre as diferentes caixas de âncora.


```
#@save
def box_iou(boxes1, boxes2):
    """Compute IOU between two sets of boxes of shape (N,4) and (M,4)."""
    # Compute box areas
    box_area = lambda boxes: ((boxes[:, 2] - boxes[:, 0]) *
                               (boxes[:, 3] - boxes[:, 1]))
    area1 = box_area(boxes1)
    area2 = box_area(boxes2)
    lt = torch.max(boxes1[:, None, :2], boxes2[:, :2]) # [N,M,2]
    rb = torch.min(boxes1[:, None, 2:], boxes2[:, 2:]) # [N,M,2]
    wh = (rb - lt).clamp(min=0) # [N,M,2]
    inter = wh[:, :, 0] * wh[:, :, 1] # [N,M]
    unioun = area1[:, None] + area2 - inter
    return inter / unioun
```

13.4.3 Rotulagem de Treinamento para Definir Caixas de Âncora

No conjunto de treinamento, consideramos cada caixa de âncora como um exemplo de treinamento. Para treinar o modelo de detecção de objetos, precisamos marcar dois tipos de rótulos para cada caixa de âncora: primeiro, a categoria do alvo contido na caixa de âncora (categoria) e, em segundo lugar, o deslocamento da caixa delimitadora da verdade básica em relação à caixa de âncora (deslocamento). Na detecção de objetos, primeiro geramos várias caixas de âncora, predizemos as categorias e deslocamentos para cada caixa de âncora, ajustamos a posição da caixa de âncora de acordo com o deslocamento previsto para obter as caixas delimitadoras a serem usadas para previsão e, finalmente, filtramos as caixas delimitadoras de predição que precisam ser produzidos.

Sabemos que, no conjunto de treinamento de detecção de objetos, cada imagem é rotulada com a localização da caixa delimitadora da verdade terrestre e a categoria do alvo contido. Depois que as caixas de âncora são geradas, rotulamos principalmente as caixas de âncora com base na localização e nas informações de categoria das caixas delimitadoras de verdade terrestre semelhantes às caixas de âncora. Então, como atribuímos caixas delimitadoras de verdade terrestre a caixas de ancoragem semelhantes a elas?

Suponha que as caixas de ancoragem na imagem sejam $A_1, A_2, \dots, A_{n_a}$ e as caixas delimitadoras de verdade são $B_1, B_2, \dots, B_{n_b}$ e $n_a \geq n_b$. Defina a matriz $\mathbf{X} \in \mathbb{R}^{n_a \times n_b}$, onde o elemento x_{ij} na linha i^{th} e coluna j^{th} é a IoU da caixa de âncora A_i para a caixa delimitadora da verdade básica B_j . Primeiro, encontramos o maior elemento na matriz $\mathbf{X}$ e registramos o índice da linha e o índice da coluna do elemento como i_1, j_1 . Atribuímos a caixa delimitadora da verdade básica B_{j_1} à caixa âncora A_{i_1} . Obviamente, a caixa de âncora A_{i_1} e a caixa delimitadora da verdade básica B_{j_1} têm a maior similaridade entre todos os pares “caixa de âncora - caixa delimitadora da verdade”. A seguir, descarte todos os elementos da i_1^{a} linha e da j_1^{a} coluna da matriz $\mathbf{X}$. Encontre o maior elemento restante na matriz $\mathbf{X}$ e registre o índice da linha e o índice da coluna do elemento como i_2, j_2 . Atribuímos a caixa delimitadora de verdade básica B_{j_2} à caixa de ancoragem A_{i_2} e, em seguida, descartamos todos os elementos na i_2^{a} linha e na j_2^{a} coluna na matriz $\mathbf{X}$. Neste ponto, os elementos em duas linhas e duas colunas na matriz $\mathbf{X}$ foram descartados.

Prosseguimos até que todos os elementos da coluna n_b da matriz $\mathbf{X}$ sejam descartados. Neste momento, atribuímos uma caixa delimitadora de verdade terrestre a cada uma das caixas de âncora n_b . Em seguida, percorremos apenas as caixas de âncora $n_a - n_b$ restantes. Dada a caixa de âncora A_i , encontre a caixa delimitadora B_j com o maior IoU com A_i de acordo com a i^{th} linha da matriz $\mathbf{X}$, e apenas atribua o terreno -caixa delimitadora da verdade B_j para ancorar a caixa A_i quando o

IoU é maior do que o limite predeterminado.

Conforme mostrado em Fig. 13.4.2 (esquerda), assumindo que o valor máximo na matriz $\mathbf{X}$ é x_{23} , iremos atribuir a caixa delimitadora da verdade básica B_3 à caixa de âncora A_2 . Em seguida, descartamos todos os elementos na linha 2 e coluna 3 da matriz, encontramos o maior elemento x_{71} da área sombreada restante e atribuímos a caixa delimitadora de verdade básica B_1 à caixa de ancoragem A_7 . Então, como mostrado em Fig. 13.4.2 (meio), descarte todos os elementos na linha 7 e coluna 1 da matriz, encontre o maior elemento x_{54} da área sombreada restante e atribua a caixa delimitadora de verdade fundamental B_4 para a caixa âncora A_5 . Finalmente, como mostrado em Fig. 13.4.2 (direita), descarte todos os elementos na linha 5 e coluna 4 da matriz, encontre o maior elemento x_{92} da área sombreada restante e atribua a caixa delimitadora da verdade fundamental B_2 para a caixa âncora A_9 . Depois disso, só precisamos atravessar as caixas de âncora restantes de A_1, A_3, A_4, A_6, A_8 e determinar se devemos atribuir caixas delimitadoras de verdade fundamental às caixas de âncora restantes de acordo com o limite.

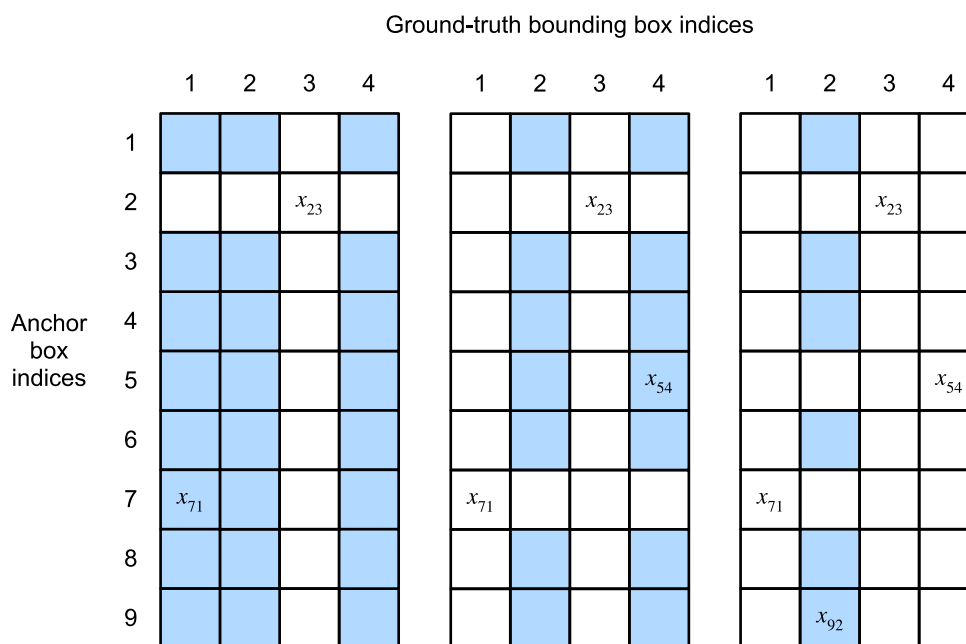


Fig. 13.4.2: Atribua caixas delimitadoras de base de verdade às caixas de ancoragem.

```
#@save
def match_anchor_to_bbox(ground_truth, anchors, device, iou_threshold=0.5):
    """Assign ground-truth bounding boxes to anchor boxes similar to them."""
    num_anchors, num_gt_boxes = anchors.shape[0], ground_truth.shape[0]
    # Element 'x_ij' in the 'i^th' row and 'j^th' column is the IoU
    # of the anchor box 'anc_i' to the ground-truth bounding box 'box_j'
    jaccard = box_iou(anchors, ground_truth)
    # Initialize the tensor to hold assigned ground truth bbox for each anchor
    anchors_bbox_map = torch.full((num_anchors,), -1, dtype=torch.long,
                                  device=device)
    # Assign ground truth bounding box according to the threshold
    max_iou, indices = torch.max(jaccard, dim=1)
    anc_i = torch.nonzero(max_iou >= 0.5).reshape(-1)
    box_j = indices[max_iou >= 0.5]
    anchors_bbox_map[anc_i] = box_j
    # Find the largest iou for each bbox
```

(continues on next page)

```

col_discard = torch.full((num_anchors,), -1)
row_discard = torch.full((num_gt_boxes,), -1)
for _ in range(num_gt_boxes):
    max_idx = torch.argmax(jaccard)
    box_idx = (max_idx % num_gt_boxes).long()
    anc_idx = (max_idx / num_gt_boxes).long()
    anchors_bbox_map[anc_idx] = box_idx
    jaccard[:, box_idx] = col_discard
    jaccard[anc_idx, :] = row_discard
return anchors_bbox_map

```

Agora podemos rotular as categorias e deslocamentos das caixas de âncora. Se uma caixa de âncora A for atribuída a uma caixa delimitadora de verdade fundamental B , a categoria da caixa de âncora A será definida como a categoria de B . E o deslocamento da caixa âncora A é definido de acordo com a posição relativa das coordenadas centrais de B e A e os tamanhos relativos das duas caixas. Como as posições e tamanhos de várias caixas no conjunto de dados podem variar, essas posições e tamanhos relativos geralmente requerem algumas transformações especiais para tornar a distribuição de deslocamento mais uniforme e fácil de ajustar. Suponha que as coordenadas centrais da caixa de âncora A e sua caixa delimitadora de verdade fundamental B sejam (x_a, y_a) , (x_b, y_b) , as larguras de A e B são w_a, w_b , e suas alturas são h_a, h_b , respectivamente. Neste caso, uma técnica comum é rotular o deslocamento de A como

$$\left(\frac{\frac{x_b - x_a}{w_a} - \mu_x}{\sigma_x}, \frac{\frac{y_b - y_a}{h_a} - \mu_y}{\sigma_y}, \frac{\log \frac{w_b}{w_a} - \mu_w}{\sigma_w}, \frac{\log \frac{h_b}{h_a} - \mu_h}{\sigma_h} \right), \quad (13.4.3)$$

Os valores padrão da constante são $\mu_x = \mu_y = \mu_w = \mu_h = 0$, $\sigma_x = \sigma_y = 0.1$, e $\sigma_w = \sigma_h = 0.2$. Esta transformação é implementada abaixo na função `offset_boxes`. Se uma caixa de âncora não for atribuída a uma caixa delimitadora de verdade, só precisamos definir a categoria da caixa de âncora como segundo plano. As caixas de âncora cuja categoria é o plano de fundo costumam ser chamadas de caixas de âncora negativas e o restante é chamado de caixas de âncora positivas.

```

#@save
def offset_boxes(anchors, assigned_bb, eps=1e-6):
    c_anc = d2l.box_corner_to_center(anchors)
    c_assigned_bb = d2l.box_corner_to_center(assigned_bb)
    offset_xy = 10 * (c_assigned_bb[:, :2] - c_anc[:, :2]) / c_anc[:, :2]
    offset_wh = 5 * torch.log(eps + c_assigned_bb[:, :2] / c_anc[:, :2])
    offset = torch.cat([offset_xy, offset_wh], axis=1)
    return offset

```

```

#@save
def multibox_target(anchors, labels):
    batch_size, anchors = labels.shape[0], anchors.squeeze(0)
    batch_offset, batch_mask, batch_class_labels = [], [], []
    device, num_anchors = anchors.device, anchors.shape[0]
    for i in range(batch_size):
        label = labels[i, :, :]
        anchors_bbox_map = match_anchor_to_bbox(label[:, 1:], anchors, device)
        bbox_mask = ((anchors_bbox_map >= 0).float().unsqueeze(-1)).repeat(1, 4)
        # Initialize class_labels and assigned bbox coordinates with zeros
        class_labels = torch.zeros(num_anchors, dtype=torch.long,
                                   device=device)

```

(continues on next page)

```

assigned_bb = torch.zeros((num_anchors, 4), dtype=torch.float32,
                           device=device)
# Assign class labels to the anchor boxes using matched gt bbox labels
# If no gt bbox is assigned to an anchor box, then let the
# class_labels and assigned_bb remain zero, i.e the background class
indices_true = torch.nonzero(anchors_bbox_map >= 0)
bb_idx = anchors_bbox_map[indices_true]
class_labels[indices_true] = label[bb_idx, 0].long() + 1
assigned_bb[indices_true] = label[bb_idx, 1:]
# offset transformations
offset = offset_boxes(anchors, assigned_bb) * bbox_mask
batch_offset.append(offset.reshape(-1))
batch_mask.append(bbox_mask.reshape(-1))
batch_class_labels.append(class_labels)
bbox_offset = torch.stack(batch_offset)
bbox_mask = torch.stack(batch_mask)
class_labels = torch.stack(batch_class_labels)
return (bbox_offset, bbox_mask, class_labels)

```

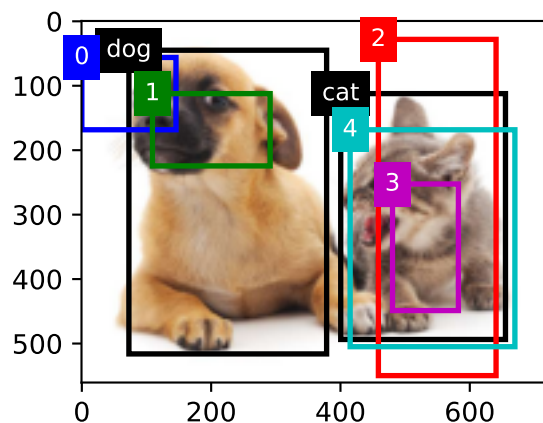
Abaixo, demonstramos um exemplo detalhado. Definimos caixas delimitadoras de verdade para o gato e o cachorro na imagem lida, onde o primeiro elemento é a categoria (0 para cachorro, 1 para gato) e os quatro elementos restantes são as coordenadas do eixo x, y no canto superior esquerdo e coordenadas do eixo x, y no canto inferior direito (o intervalo de valores está entre 0 e 1). Aqui, construímos cinco caixas de âncora para serem rotuladas pelas coordenadas do canto superior esquerdo e do canto inferior direito, que são registradas como $A_0, \dots, A_4$, respectivamente (o índice no programa começa em 0). Primeiro, desenhe as posições dessas caixas de âncora e das caixas delimitadoras da verdade fundamental na imagem.

```

ground_truth = torch.tensor([[0, 0.1, 0.08, 0.52, 0.92],
                             [1, 0.55, 0.2, 0.9, 0.88]])
anchors = torch.tensor([[0, 0.1, 0.2, 0.3], [0.15, 0.2, 0.4, 0.4],
                        [0.63, 0.05, 0.88, 0.98], [0.66, 0.45, 0.8, 0.8],
                        [0.57, 0.3, 0.92, 0.9]])

fig = d2l.plt.imshow(img)
show_bboxes(fig.axes, ground_truth[:, 1:] * bbox_scale, ['dog', 'cat'], 'k')
show_bboxes(fig.axes, anchors * bbox_scale, ['0', '1', '2', '3', '4']);

```



Podemos rotular categorias e deslocamentos para caixas de âncora usando a função `multibox_target`. Esta função define a categoria de fundo para 0 e incrementa o índice inteiro da categoria de destino de zero por 1 (1 para cachorro e 2 para gato).

Adicionamos dimensões de exemplo às caixas de âncora e caixas delimitadoras de verdade e construímos resultados preditos aleatórios com uma forma de (tamanho do lote, número de categorias incluindo fundo, número de caixas de âncora) usando a função `unsqueeze`.

```
labels = multibox_target(anchors.unsqueeze(dim=0),
                        ground_truth.unsqueeze(dim=0))
```

Existem três itens no resultado retornado, todos no formato tensor. O terceiro item é representado pela categoria rotulada para a caixa de âncora.

```
labels[2]
```

```
tensor([[0, 1, 2, 0, 2]])
```

Analizamos essas categorias rotuladas com base nas posições das caixas de âncora e das caixas delimitadoras de informações básicas na imagem. Em primeiro lugar, em todos os pares de “caixa de âncora - caixa delimitadora de verdade básica”, a IoU da caixa de ancoragem A_4 para a caixa delimitadora de verdade básica do gato é a maior, então a categoria de caixa de ancoragem A_4 é rotulada como gato. Sem considerar a caixa de âncora A_4 ou a caixa delimitadora de verdade do solo do gato, nos pares restantes “caixa de âncora - caixa de ligação de verdade”, o par com a maior IoU é a caixa de âncora A_1 e a caixa delimitadora da verdade do cachorro, portanto, a categoria da caixa de âncora A_1 é rotulada como cachorro. Em seguida, atravesse as três caixas de âncora restantes sem etiqueta. A categoria da caixa delimitadora de verdade básica com o maior IoU com caixa de âncora A_0 é cachorro, mas o IoU é menor que o limite (o padrão é 0,5), portanto, a categoria é rotulada como plano de fundo; a categoria da caixa delimitadora de verdade básica com a maior IoU com caixa de âncora A_2 é gato e a IoU é maior que o limite, portanto, a categoria é rotulada como gato; a categoria da caixa delimitadora de verdade básica com a maior IoU com caixa de âncora A_3 é cat, mas a IoU é menor que o limite, portanto, a categoria é rotulada como plano de fundo.

O segundo item do valor de retorno é uma variável de máscara, com a forma de (tamanho do lote, quatro vezes o número de caixas de âncora). Os elementos na variável de máscara correspondem um a um com os quatro valores de deslocamento de cada caixa de âncora. Como não nos importamos com a detecção de fundo, os deslocamentos da classe negativa não devem afetar a função de destino. Multiplicando por elemento, o 0 na variável de máscara pode filtrar os deslocamentos de classe negativos antes de calcular a função de destino.

```
labels[1]
```

```
tensor([[0., 0., 0., 0., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 1., 1.,
        1., 1.]])
```

O primeiro item retornado são os quatro valores de deslocamento rotulados para cada caixa de âncora, com os deslocamentos das caixas de âncora de classe negativa rotulados como 0.

```
labels[0]
```

```
tensor([[ -0.00e+00, -0.00e+00, -0.00e+00, -0.00e+00,  1.40e+00,  1.00e+01,
          2.59e+00,  7.18e+00, -1.20e+00,  2.69e-01,  1.68e+00, -1.57e+00,
          -0.00e+00, -0.00e+00, -0.00e+00, -0.00e+00, -5.71e-01, -1.00e+00,
          4.17e-06,  6.26e-01]])
```

13.4.4 Caixas Delimitadoras para Previsão

Durante a fase de previsão do modelo, primeiro geramos várias caixas de âncora para a imagem e, em seguida, predizemos categorias e deslocamentos para essas caixas de âncora, uma por uma. Em seguida, obtemos caixas delimitadoras de previsão com base nas caixas de âncora e seus deslocamentos previstos.

Abaixo, implementamos a função `offset_inverse` que leva âncoras e previsões de deslocamento como entradas e aplica transformações de deslocamento inversas para retornar as coordenadas da caixa delimitadora prevista.

```
#@save
def offset_inverse(anchors, offset_preds):
    c_anc = d2l.box_corner_to_center(anchors)
    c_pred_bb_xy = (offset_preds[:, :2] * c_anc[:, 2:] / 10) + c_anc[:, :2]
    c_pred_bb_wh = torch.exp(offset_preds[:, 2:] / 5) * c_anc[:, 2:]
    c_pred_bb = torch.cat((c_pred_bb_xy, c_pred_bb_wh), axis=1)
    predicted_bb = d2l.box_center_to_corner(c_pred_bb)
    return predicted_bb
```

Quando há muitas caixas de âncora, muitas caixas delimitadoras de predição semelhantes podem ser geradas para o mesmo alvo. Para simplificar os resultados, podemos remover caixas delimitadoras de predição semelhantes. Um método comumente usado é chamado de supressão não máxima (NMS).

Vamos dar uma olhada em como o NMS funciona. Para uma caixa delimitadora de previsão B , o modelo calcula a probabilidade prevista para cada categoria. Suponha que a maior probabilidade prevista seja p , a categoria correspondente a essa probabilidade é a categoria prevista de B . Também nos referimos a p como o nível de confiança da caixa delimitadora de predição B . Na mesma imagem, classificamos as caixas delimitadoras de previsão com categorias previstas diferentes do plano de fundo por nível de confiança de alto a baixo e obtemos a lista L . Seleccionamos a caixa delimitadora de predição B_1 com o nível de confiança mais alto de L como linha de base e remove todas as caixas delimitadoras de predição não comparativas com um IoU com B_1 maior que um determinado limite de L . O limite aqui é um hiperparâmetro predefinido. Nesse ponto, L retém a caixa delimitadora de predição com o nível de confiança mais alto e remove outras caixas delimitadoras de predição semelhantes a ela. Em seguida, seleccionamos a caixa delimitadora de predição B_2 com o segundo nível de confiança mais alto de L como linha de base e removemos todas as caixas delimitadoras de predição não comparativas com um IoU com B_2 maior que um determinado limite de L . Repetimos esse processo até que todas as caixas delimitadoras de previsão em L tenham sido usadas como linha de base. Neste momento, a IoU de qualquer par de caixas delimitadoras de predição em L é menor que o limite. Finalmente, produzimos todas as caixas delimitadoras de predição na lista L .

```
#@save
def nms(boxes, scores, iou_threshold):
```

(continues on next page)

```

# sorting scores by the descending order and return their indices
B = torch.argsort(scores, dim=-1, descending=True)
keep = [] # boxes indices that will be kept
while B.numel() > 0:
    i = B[0]
    keep.append(i)
    if B.numel() == 1: break
    iou = box_iou(boxes[i, :].reshape(-1, 4),
                  boxes[B[1:], :].reshape(-1, 4)).reshape(-1)
    inds = torch.nonzero(iou <= iou_threshold).reshape(-1)
    B = B[inds + 1]
return torch.tensor(keep, device=boxes.device)

#@save
def multibox_detection(cls_probs, offset_preds, anchors, nms_threshold=0.5,
                      pos_threshold=0.00999999978):
    device, batch_size = cls_probs.device, cls_probs.shape[0]
    anchors = anchors.squeeze(0)
    num_classes, num_anchors = cls_probs.shape[1], cls_probs.shape[2]
    out = []
    for i in range(batch_size):
        cls_prob, offset_pred = cls_probs[i], offset_preds[i].reshape(-1, 4)
        conf, class_id = torch.max(cls_prob[1:], 0)
        predicted_bb = offset_inverse(anchors, offset_pred)
        keep = nms(predicted_bb, conf, 0.5)
        # Find all non_keep indices and set the class_id to background
        all_idx = torch.arange(num_anchors, dtype=torch.long, device=device)
        combined = torch.cat((keep, all_idx))
        uniques, counts = combined.unique(return_counts=True)
        non_keep = uniques[counts == 1]
        all_id_sorted = torch.cat((keep, non_keep))
        class_id[non_keep] = -1
        class_id = class_id[all_id_sorted]
        conf, predicted_bb = conf[all_id_sorted], predicted_bb[all_id_sorted]
        # threshold to be a positive prediction
        below_min_idx = (conf < pos_threshold)
        class_id[below_min_idx] = -1
        conf[below_min_idx] = 1 - conf[below_min_idx]
        pred_info = torch.cat((class_id.unsqueeze(1),
                              conf.unsqueeze(1),
                              predicted_bb), dim=1)
        out.append(pred_info)
    return torch.stack(out)

```

A seguir, veremos um exemplo detalhado. Primeiro, construa quatro caixas de âncora. Para simplificar, assumimos que os deslocamentos previstos são todos 0. Isso significa que as caixas delimitadoras de previsão são caixas de âncora. Finalmente, construímos uma probabilidade prevista para cada categoria.

```

anchors = torch.tensor([[0.1, 0.08, 0.52, 0.92], [0.08, 0.2, 0.56, 0.95],
                        [0.15, 0.3, 0.62, 0.91], [0.55, 0.2, 0.9, 0.88]])
offset_preds = torch.tensor([0] * anchors.numel())
cls_probs = torch.tensor([0] * 4, # Predicted probability for background
                          [0.9, 0.8, 0.7, 0.1], # Predicted probability for dog

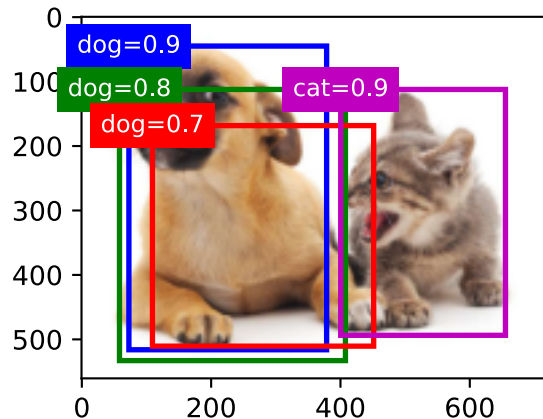
```

(continues on next page)


```
[0.1, 0.2, 0.3, 0.9]]) # Predicted probability for cat
```

Imprima caixas delimitadoras de previsão e seus níveis de confiança na imagem.

```
fig = d2l.plt.imshow(img)
show_bboxes(fig.axes, anchors * bbox_scale,
            ['dog=0.9', 'dog=0.8', 'dog=0.7', 'cat=0.9'])
```



Usamos a função `multibox_detection` para executar NMS e definir o limite para 0,5. Isso adiciona uma dimensão de exemplo à entrada do tensor. Podemos ver que a forma do resultado retornado é (tamanho do lote, número de caixas de âncora, 6). Os 6 elementos de cada linha representam as informações de saída para a mesma caixa delimitadora de previsão. O primeiro elemento é o índice de categoria previsto, que começa em 0 (0 é cachorro, 1 é gato). O valor -1 indica fundo ou remoção no NMS. O segundo elemento é o nível de confiança da caixa delimitadora de previsão. Os quatro elementos restantes são as coordenadas do eixo x, y do canto superior esquerdo e as coordenadas do eixo x, y do canto inferior direito da caixa delimitadora de previsão (o intervalo de valores está entre 0 e 1).

```
output = multibox_detection(cls_probs.unsqueeze(dim=0),
                           offset_preds.unsqueeze(dim=0),
                           anchors.unsqueeze(dim=0),
                           nms_threshold=0.5)
```

output

```
tensor([[[[ 0.00,  0.90,  0.10,  0.08,  0.52,  0.92],
           [ 1.00,  0.90,  0.55,  0.20,  0.90,  0.88],
           [-1.00,  0.80,  0.08,  0.20,  0.56,  0.95],
           [-1.00,  0.70,  0.15,  0.30,  0.62,  0.91]]]])
```

Removemos as caixas delimitadoras de previsão da categoria -1 e visualizamos os resultados retidos pelo NMS.

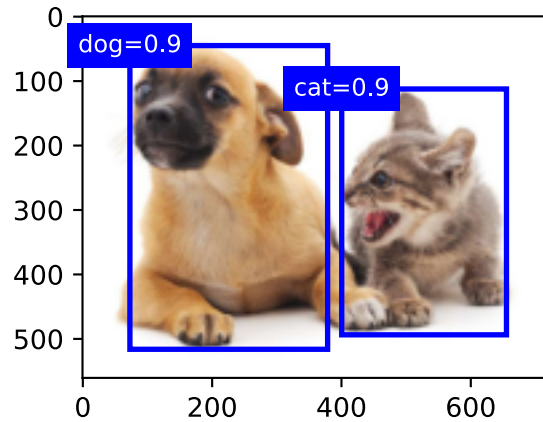
```
fig = d2l.plt.imshow(img)
for i in output[0].detach().numpy():
    if i[0] == -1:
```

(continues on next page)


```

continue
label = ('dog=', 'cat=')[int(i[0])] + str(i[1])
show_bboxes(fig.axes, [torch.tensor(i[2:]) * bbox_scale], label)

```



Na prática, podemos remover caixas delimitadoras de predição com níveis de confiança mais baixos antes de executar NMS, reduzindo assim a quantidade de computação para NMS. Também podemos filtrar a saída de NMS, por exemplo, retraindo apenas os resultados com níveis de confiança mais altos como saída final.

13.4.5 Resumo

- Geramos várias caixas de âncora com diferentes tamanhos e proporções de aspecto, centralizadas em cada pixel.
- IoU, também chamado de índice de Jaccard, mede a similaridade de duas caixas delimitadoras. É a proporção entre a área de intersecção e a área de união de duas caixas delimitadoras.
- No conjunto de treinamento, marcamos dois tipos de rótulos para cada caixa de âncora: um é a categoria do alvo contido na caixa de âncora e o outro é o deslocamento da caixa delimitadora de verdade em relação à caixa de âncora.
- Ao prever, podemos usar supressão não máxima (NMS) para remover caixas delimitadoras de previsão semelhantes, simplificando assim os resultados.

13.4.6 Exercícios

1. Altere os valores de `size_ratios` na função `multibox_prior` e observe as alterações nas caixas de âncora geradas.
2. Construa duas caixas delimitadoras com uma IoU de 0,5 e observe sua coincidência.
3. Verifique a saída de `offset_labels[0]` marcando os offsets da caixa de âncora conforme definido nesta seção (a constante é o valor padrão).
4. Modifique a variável `anchors` nas seções “Rotulando Caixas de Âncora de Conjunto de Treinamento” e “Caixas Limitadoras de Saída para Previsão”. Como os resultados mudam?

13.5 Detecção de Objetos Multiescala

Em [Section 13.4](#), geramos várias caixas de âncora centralizadas em cada pixel da imagem de entrada. Essas caixas de âncora são usadas para amostrar diferentes regiões da imagem de entrada. No entanto, se as caixas de âncora forem geradas centralizadas em cada pixel da imagem, logo haverá muitas caixas de âncora para calcularmos. Por exemplo, assumimos que a imagem de entrada tem uma altura e uma largura de 561 e 728 pixels, respectivamente. Se cinco formas diferentes de caixas de âncora são geradas centralizadas em cada pixel, mais de dois milhões de caixas de âncora ($561 \times 728 \times 5$) precisam ser previstas e rotuladas na imagem.

Não é difícil reduzir o número de caixas de âncora. Uma maneira fácil é aplicar amostragem uniforme em uma pequena parte dos pixels da imagem de entrada e gerar caixas de âncora centralizadas nos pixels amostrados. Além disso, podemos gerar caixas de âncora de números e tamanhos variados em várias escalas. Observe que é mais provável que objetos menores sejam posicionados na imagem do que objetos maiores. Aqui, usaremos um exemplo simples: Objetos com formas de 1×1 , 1×2 , e 2×2 podem ter 4, 2 e 1 posição(ões) possível(is) em uma imagem com a forma 2×2 . Portanto, ao usar caixas de âncora menores para detectar objetos menores, podemos amostrar mais regiões; ao usar caixas de âncora maiores para detectar objetos maiores, podemos amostrar menos regiões.

Para demonstrar como gerar caixas de âncora em várias escalas, vamos ler uma imagem primeiro. Tem uma altura e largura de 561×728 pixels.

```
%matplotlib inline
import torch
from d2l import torch as d2l

img = d2l.plt.imread('../img/catdog.jpg')
h, w = img.shape[0:2]
h, w
```

```
(561, 728)
```

Em [Section 6.2](#), a saída da matriz 2D da rede neural convolucional (CNN) é chamada um mapa de recursos. Podemos determinar os pontos médios de caixas de âncora uniformemente amostradas em qualquer imagem, definindo a forma do mapa de feições.

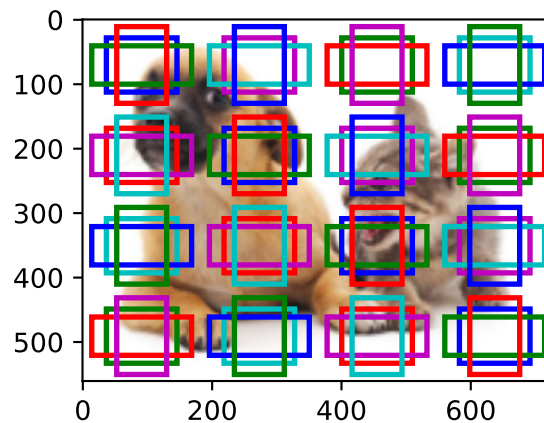
A função `display_anchors` é definida abaixo. Vamos gerar caixas de âncora `anchors` centradas em cada unidade (pixel) no mapa de feições `fmap`. Uma vez que as coordenadas dos eixos x e y nas caixas de âncora `anchors` foram divididas pela largura e altura do mapa de feições `fmap`, valores entre 0 e 1 podem ser usados para representar as posições relativas das caixas de âncora em o mapa de recursos. Uma vez que os pontos médios das “âncoras” das caixas de âncora se sobrepõem a todas as unidades no mapa de características “`fmap`”, as posições espaciais relativas dos pontos médios das “âncoras” em qualquer imagem devem ter uma distribuição uniforme. Especificamente, quando a largura e a altura do mapa de feições são definidas para `fmap_w` e `fmap_h` respectivamente, a função irá conduzir uma amostragem uniforme para linhas `fmap_h` e colunas de pixels `fmap_w` e usá-los como pontos médios para gerar caixas de âncora com tamanho `s` (assumimos que o comprimento da lista `s` é 1) e diferentes proporções (ratios).

¹⁵⁹ <https://discuss.d2l.ai/t/1603>

```
def display_anchors(fmap_w, fmap_h, s):
    d2l.set_figsize()
    # The values from the first two dimensions will not affect the output
    fmap = torch.zeros((1, 10, fmap_h, fmap_w))
    anchors = d2l.multibox_prior(fmap, sizes=s, ratios=[1, 2, 0.5])
    bbox_scale = torch.tensor((w, h, w, h))
    d2l.show_bboxes(d2l.plt.imshow(img).axes,
                    anchors[0] * bbox_scale)
```

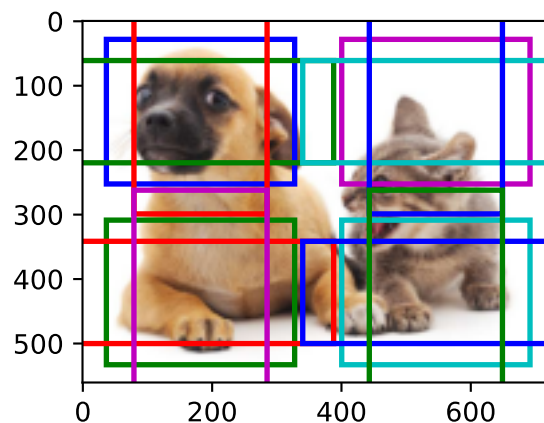
Primeiro, vamos nos concentrar na detecção de pequenos objetos. A fim de tornar mais fácil distinguir na exibição, as caixas de âncora com diferentes pontos médios aqui não se sobrepõem. Assumimos que o tamanho das caixas de âncora é 0,15 e a altura e largura do mapa de feições são 4. Podemos ver que os pontos médios das caixas de âncora das 4 linhas e 4 colunas da imagem estão uniformemente distribuídos.

```
display_anchors(fmap_w=4, fmap_h=4, s=[0.15])
```



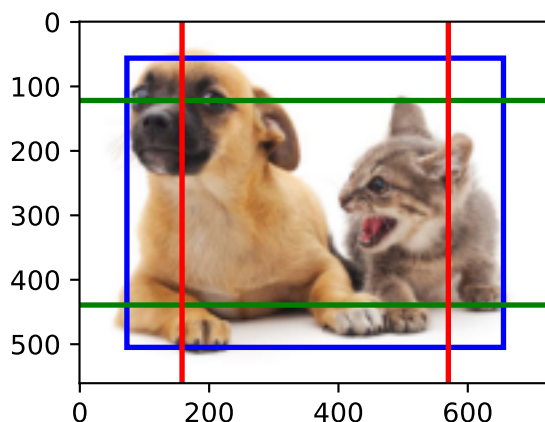
Vamos reduzir a altura e a largura do mapa de feições pela metade e usar uma caixa de âncora maior para detectar objetos maiores. Quando o tamanho é definido como 0,4, ocorrerão sobreposições entre as regiões de algumas caixas de âncora.

```
display_anchors(fmap_w=2, fmap_h=2, s=[0.4])
```



Finalmente, vamos reduzir a altura e a largura do mapa de feições pela metade e aumentar o tamanho da caixa de âncora para 0,8. Agora, o ponto médio da caixa de âncora é o centro da imagem.

```
display_anchors(fmap_w=1, fmap_h=1, s=[0.8])
```



Como geramos caixas de âncora de tamanhos diferentes em escalas múltiplas, vamos usá-las para detectar objetos de vários tamanhos em escalas diferentes. Agora vamos apresentar um método baseado em redes neurais convolucionais (CNNs).

Em uma determinada escala, suponha que geramos $h \times w$ conjuntos de caixas de âncora com diferentes pontos médios baseados em c_i mapas de feições com a forma $h \times w$ e o número de caixas de âncora em cada conjunto é a . Por exemplo, para a primeira escala do experimento, geramos 16 conjuntos de caixas de âncora com diferentes pontos médios com base em 10 (número de canais) mapas de recursos com uma forma de 4×4 , e cada conjunto contém 3 caixas de âncora. A seguir, cada caixa de âncora é rotulada com uma categoria e deslocamento com base na classificação e posição da caixa delimitadora de verdade. Na escala atual, o modelo de detecção de objeto precisa prever a categoria e o offset de $h \times w$ conjuntos de caixas de âncora com diferentes pontos médios com base na imagem de entrada.

Assumimos que os mapas de características c_i são a saída intermediária da CNN com base na imagem de entrada. Uma vez que cada mapa de características tem $h \times w$ posições espaciais diferentes, a mesma posição terá c_i unidades. De acordo com a definição de campo receptivo em [Section 6.2](#), as unidades c_i do mapa de feições na mesma posição espacial têm o mesmo campo receptivo na imagem de entrada. Assim, elas representam a informação da imagem de entrada neste mesmo campo receptivo. Portanto, podemos transformar as unidades c_i do mapa de feições na mesma posição espacial nas categorias e deslocamentos das a caixas de âncora geradas usando essa posição como um ponto médio. Não é difícil perceber que, em essência, usamos as informações da imagem de entrada em um determinado campo receptivo para prever a categoria e deslocamento das caixas de âncora perto do campo na imagem de entrada.

Quando os mapas de recursos de camadas diferentes têm campos receptivos de tamanhos diferentes na imagem de entrada, eles são usados para detectar objetos de tamanhos diferentes. Por exemplo, podemos projetar uma rede para ter um campo receptivo mais amplo para cada unidade no mapa de recursos que está mais perto da camada de saída, para detectar objetos com tamanhos maiores na imagem de entrada.

Implementaremos um modelo de detecção de objetos multiescala na seção seguinte.

13.5.1 Resumo

- Podemos gerar caixas de âncora com diferentes números e tamanhos em várias escalas para detectar objetos de diferentes tamanhos em várias escalas.
- A forma do mapa de feições pode ser usada para determinar o ponto médio das caixas de âncora que amostram uniformemente qualquer imagem.
- Usamos as informações da imagem de entrada de um determinado campo receptivo para prever a categoria e o deslocamento das caixas de âncora próximas a esse campo na imagem.

13.5.2 Exercícios

1. Dada uma imagem de entrada, suponha que $1 \times c_i \times h \times w$ seja a forma do mapa de características, enquanto c_i, h, w são o número, altura e largura do mapa de características. Em quais métodos você pode pensar para converter essa variável na categoria e deslocamento da caixa de âncora? Qual é o formato da saída?

Discussões¹⁶⁰

13.6 O Dataset de Detecção de Objetos

Não existem pequenos conjuntos de dados, como MNIST ou Fashion-MNIST, no campo de detecção de objetos. Para testar os modelos rapidamente, vamos montar um pequeno conjunto de dados. Primeiro, geramos 1000 imagens de banana de diferentes ângulos e tamanhos usando bananas grátis de nosso escritório. Em seguida, coletamos uma série de imagens de fundo e colocamos uma imagem de banana em uma posição aleatória em cada imagem.

13.6.1 Baixando Dataset

O conjunto de dados de detecção de banana com todas as imagens e arquivos de rótulo csv pode ser baixado diretamente da Internet.

```
%matplotlib inline
import os
import pandas as pd
import torch
import torchvision
from d2l import torch as d2l

#@save
d2l.DATA_HUB['banana-detection'] = (d2l.DATA_URL + 'banana-detection.zip',
                                    '5de26c8fce5ccdea9f91267273464dc968d20d72')
```

¹⁶⁰ <https://discuss.d2l.ai/t/1607>

13.6.2 Lendo o Dataset

Vamos ler o conjunto de dados de detecção de objetos na função `read_data_bananas`. O conjunto de dados inclui um arquivo csv para rótulos de classe de destino e coordenadas de caixa delimitadora de verdade fundamental no formato corner. Definimos `BananasDataset` para criar a instância do Dataset e finalmente definimos a função `load_data_bananas` para retornar os carregadores de dados. Não há necessidade de ler o conjunto de dados de teste em ordem aleatória.

```

#@save
def read_data_bananas(is_train=True):
    """Read the bananas dataset images and labels."""
    data_dir = d2l.download_extract('banana-detection')
    csv_fname = os.path.join(data_dir, 'bananas_train' if is_train
                              else 'bananas_val', 'label.csv')
    csv_data = pd.read_csv(csv_fname)
    csv_data = csv_data.set_index('img_name')
    images, targets = [], []
    for img_name, target in csv_data.iterrows():
        images.append(torchvision.io.read_image(
            os.path.join(data_dir, 'bananas_train' if is_train else
                          'bananas_val', 'images', f'{img_name}')))
        # Since all images have same object class i.e. category '0',
        # the 'label' column corresponds to the only object i.e. banana
        # The target is as follows : ('label', 'xmin', 'ymin', 'xmax', 'ymax')
        targets.append(list(target))
    return images, torch.tensor(targets).unsqueeze(1) / 256

#@save
class BananasDataset(torch.utils.data.Dataset):
    def __init__(self, is_train):
        self.features, self.labels = read_data_bananas(is_train)
        print('read ' + str(len(self.features)) + (f' training examples' if
                                                    is_train else f' validation examples'))

    def __getitem__(self, idx):
        return (self.features[idx].float(), self.labels[idx])

    def __len__(self):
        return len(self.features)

#@save
def load_data_bananas(batch_size):
    """Load the bananas dataset."""
    train_iter = torch.utils.data.DataLoader(BananasDataset(is_train=True),
                                              batch_size, shuffle=True)
    val_iter = torch.utils.data.DataLoader(BananasDataset(is_train=False),
                                           batch_size)

    return (train_iter, val_iter)
```

Abaixo, lemos um minibatch e imprimimos o formato da imagem e do *label*. A forma da imagem é a mesma da experiência anterior (tamanho do lote, número de canais, altura, largura). O formato do rótulo é (tamanho do lote, m , 5), onde m é igual ao número máximo de caixas delimitadoras contidas em uma única imagem no conjunto de dados. Embora o cálculo do minibatch seja muito eficiente, ele exige que cada imagem contenha o mesmo número de caixas delimitadoras para

que possam ser colocadas no mesmo lote. Como cada imagem pode ter um número diferente de caixas delimitadoras, podemos adicionar caixas delimitadoras ilegais às imagens que possuem menos de m caixas delimitadoras até que cada imagem contenha m caixas delimitadoras. Assim, podemos ler um minibatch de imagens a cada vez. O rótulo de cada caixa delimitadora na imagem é representado por um tensor de comprimento 5. O primeiro elemento no tensor é a categoria do objeto contido na caixa delimitadora. Quando o valor é -1, a caixa delimitadora é uma caixa delimitadora ilegal para fins de preenchimento. Os quatro elementos restantes da matriz representam as coordenadas do eixo x, y do canto superior esquerdo da caixa delimitadora e as coordenadas do eixo x, y do canto inferior direito da caixa delimitadora (o intervalo de valores é entre 0 e 1). O conjunto de dados da banana aqui tem apenas uma caixa delimitadora por imagem, então $m = 1$.

```
batch_size, edge_size = 32, 256
train_iter, _ = load_data_bananas(batch_size)
batch = next(iter(train_iter))
batch[0].shape, batch[1].shape
```

```
Downloading ../data/banana-detection.zip from http://d2l-data.s3-accelerate.amazonaws.com/
↳ banana-detection.zip...
read 1000 training examples
read 100 validation examples
```

```
(torch.Size([32, 3, 256, 256]), torch.Size([32, 1, 5]))
```

13.6.3 Demonstração

Temos dez imagens com caixas delimitadoras. Podemos ver que o ângulo, o tamanho e a posição da banana são diferentes em cada imagem. Claro, este é um conjunto de dados artificial simples. Na prática, os dados geralmente são muito mais complicados.

```
imgs = (batch[0][0:10].permute(0, 2, 3, 1)) / 255
axes = d2l.show_images(imgs, 2, 5, scale=2)
for ax, label in zip(axes, batch[1][0:10]):
    d2l.show_bboxes(ax, [label[0][1:5] * edge_size], colors=['w'])
```



13.6.4 Resumo

- O conjunto de dados de detecção de banana que sintetizamos pode ser usado para testar modelos de detecção de objetos.
- A leitura de dados para detecção de objetos é semelhante àquela para classificação de imagens. No entanto, depois de introduzirmos as caixas delimitadoras, a forma do rótulo e o aumento da imagem (por exemplo, corte aleatório) são alterados.

13.6.5 Exercícios

1. Referindo-se à documentação do MXNet, quais são os parâmetros para os construtores das classes `image.ImageDetIter` e `image.CreateDetAugmenter`? Qual é o seu significado?

Discussões¹⁶¹

13.7 Detecção *Single Shot Multibox* (SSD)

Nas poucas seções anteriores, apresentamos caixas delimitadoras, caixas de âncora, detecção de objetos multiescala e conjuntos de dados. Agora, usaremos esse conhecimento prévio para construir um modelo de detecção de objetos: detecção multibox de disparo único [*Single Shot Multibox Detection*] (SSD) (Liu et al., 2016). Este modelo rápido e fácil já é amplamente utilizado. Alguns dos conceitos de design e detalhes de implementação deste modelo também são aplicáveis a outros modelos de detecção de objetos.

13.7.1 Modelo

Fig. 13.7.1 mostra o design de um modelo SSD. Os principais componentes do modelo são um bloco de rede básico e vários blocos de recursos multiescala conectados em série. Aqui, o bloco de rede de base é usado para as características extras de imagens originais e geralmente assumem a forma de uma rede neural convolucional profunda. O artigo sobre SSDs opta por colocar um VGG truncado antes da camada de classificação (Liu et al., 2016), mas agora é comumente substituído pelo ResNet. Podemos projetar uma rede de base para que ela produza alturas e larguras maiores. Desta forma, mais caixas de âncora são geradas com base neste mapa de características, permitindo-nos detectar objetos menores. Em seguida, cada bloco de feições multiescala reduz a altura e largura do mapa de feições fornecidas pela camada anterior (por exemplo, pode reduzir os tamanhos pela metade). Os blocos então usam cada elemento no mapa de recursos para expandir o campo receptivo na imagem de entrada. Desta forma, quanto mais próximo um bloco de feições multiescala estiver do topo de Fig. 13.7.1 menor será o mapa de feições de saída e menos caixas de âncora são geradas com base no mapa de feições. Além disso, quanto mais próximo um bloco de recursos estiver do topo, maior será o campo receptivo de cada elemento no mapa de recursos e mais adequado será para detectar objetos maiores. Como o SSD gera diferentes números de caixas de âncora de tamanhos diferentes com base no bloco de rede de base e cada bloco de recursos multiescala e, em seguida, prevê como categorias e deslocamentos (ou seja, caixas delimitadoras previsão) das caixas de âncora para detectar objetos de tamanhos diferentes, SSD é um modelo de detecção de objetos multiescala.

¹⁶¹ <https://discuss.d2l.ai/t/1608>

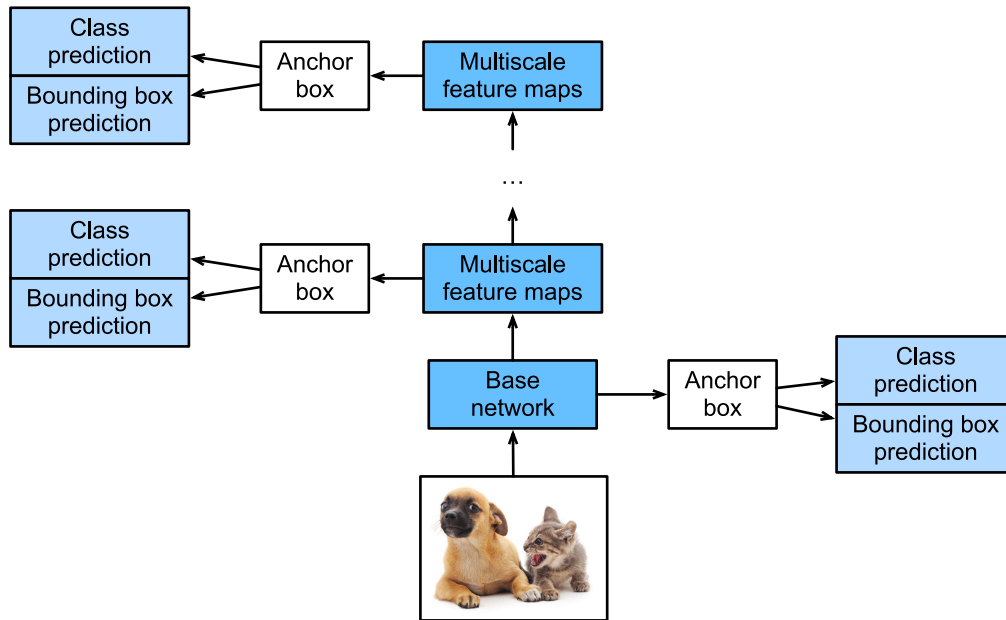


Fig. 13.7.1: O SSD é composto de um bloco de rede base e vários blocos de recursos multiescala conectados em série.

A seguir, descreveremos a implementação dos módulos em Fig. 13.7.1. Primeiro, precisamos discutir a implementação da previsão da categoria e da previsão da caixa delimitadora.

Camada de Previsão da Categoria

Defina o número de categorias de objeto como q . Nesse caso, o número de categorias de caixa de âncora é $q + 1$, com 0 indicando uma caixa de âncora que contém apenas o fundo. Para uma determinada escala, defina a altura e a largura do mapa de feições para h e w , respectivamente. Se usarmos cada elemento como o centro para gerar a caixas de âncora, precisamos classificar um total de hwa caixas de âncora. Se usarmos uma camada totalmente conectada (FCN) para a saída, isso provavelmente resultará em um número excessivo de parâmetros do modelo. Lembre-se de como usamos canais de camada convolucional para gerar previsões de categoria em Section 7.3. O SSD usa o mesmo método para reduzir a complexidade do modelo.

Especificamente, a camada de previsão de categoria usa uma camada convolucional que mantém a altura e largura de entrada. Assim, a saída e a entrada têm uma correspondência de um para um com as coordenadas espaciais ao longo da largura e altura do mapa de características. Supondo que a saída e a entrada tenham as mesmas coordenadas (x, y) , o canal para as coordenadas (x, y) no mapa de feição de saída contém as previsões de categoria para todas as caixas âncora geradas usando as coordenadas do mapa de feição de entrada (x, y) como o Centro. Portanto, existem $a(q + 1)$ canais de saída, com os canais de saída indexados como $i(q + 1) + j$ ($0 \leq j \leq q$) representando as previsões do índice de categoria j para o índice de caixa de âncora i .

Agora, vamos definir uma camada de previsão de categoria deste tipo. Depois de especificar os parâmetros a e q , ele usa uma camada convolucional 3×3 com um preenchimento de 1. As alturas e larguras de entrada e saída dessa camada convolucional permanecem inalteradas.

```
%matplotlib inline
import torch
```

(continues on next page)

```
import torchvision
from torch import nn
from torch.nn import functional as F
from d2l import torch as d2l

def cls_predictor(num_inputs, num_anchors, num_classes):
    return nn.Conv2d(num_inputs, num_anchors * (num_classes + 1),
                     kernel_size=3, padding=1)
```

Camada de Previsão de Caixa Delimitadora

O design da camada de previsão da caixa delimitadora é semelhante ao da camada de previsão da categoria. A única diferença é que, aqui, precisamos prever 4 deslocamentos para cada caixa de âncora, em vez de categorias $q + 1$.

```
def bbox_predictor(num_inputs, num_anchors):
    return nn.Conv2d(num_inputs, num_anchors * 4, kernel_size=3, padding=1)
```

Concatenando Previsões para Múltiplas Escalas

Como mencionamos, o SSD usa mapas de recursos com base em várias escalas para gerar caixas de âncora e prever suas categorias e deslocamentos. Como as formas e o número de caixas de âncora centradas no mesmo elemento diferem para os mapas de recursos de escalas diferentes, as saídas de predição em escalas diferentes podem ter formas diferentes.

No exemplo a seguir, usamos o mesmo lote de dados para construir mapas de características de duas escalas diferentes, Y1 e Y2. Aqui, Y2 tem metade da altura e metade da largura de Y1. Usando a previsão de categoria como exemplo, assumimos que cada elemento nos mapas de características Y1 e Y2 gera cinco (Y1) ou três (Y2) caixas de âncora. Quando há 10 categorias de objeto, o número de canais de saída de predição de categoria é $5 \times (10 + 1) = 55$ ou $3 \times (10 + 1) = 33$. O formato da saída de previsão é (tamanho do lote, número de canais, altura, largura). Como você pode ver, exceto pelo tamanho do lote, os tamanhos das outras dimensões são diferentes. Portanto, devemos transformá-los em um formato consistente e concatenar as previsões das várias escalas para facilitar o cálculo subsequente.

```
def forward(x, block):
    return block(x)

Y1 = forward(torch.zeros((2, 8, 20, 20)), cls_predictor(8, 5, 10))
Y2 = forward(torch.zeros((2, 16, 10, 10)), cls_predictor(16, 3, 10))
(Y1.shape, Y2.shape)
```

```
(torch.Size([2, 55, 20, 20]), torch.Size([2, 33, 10, 10]))
```

A dimensão do canal contém as previsões para todas as caixas de âncora com o mesmo centro. Primeiro movemos a dimensão do canal para a dimensão final. Como o tamanho do lote é o mesmo para todas as escalas, podemos converter os resultados da previsão para o formato binário

(tamanho do lote, altura $\times$ largura $\times$ número de canais) para facilitar a concatenação subsequente no 1st dimensão.

```
def flatten_pred(pred):
    return torch.flatten(pred.permute(0, 2, 3, 1), start_dim=1)

def concat_preds(preds):
    return torch.cat([flatten_pred(p) for p in preds], dim=1)
```

Assim, independentemente das diferentes formas de Y1 e Y2, ainda podemos concatenar os resultados da previsão para as duas escalas diferentes do mesmo lote.

```
concat_preds([Y1, Y2]).shape
```

```
torch.Size([2, 25300])
```

Bloco de Redução de Amostragem de Altura e Largura

Para detecção de objetos multiescala, definimos o seguinte bloco `down_sample_blk`, que reduz a altura e largura em 50%. Este bloco consiste em duas camadas convolucionais 3×3 com um preenchimento de 1 e uma camada de *pooling* máximo 2×2 com uma distância de 2 conectadas em uma série. Como sabemos, 3×3 camadas convolucionais com um preenchimento de 1 não alteram a forma dos mapas de características. No entanto, a camada de agrupamento subsequente reduz diretamente o tamanho do mapa de feições pela metade. Como $1 \times 2 + (3 - 1) + (3 - 1) = 6$, cada elemento no mapa de recursos de saída tem um campo receptivo no mapa de recursos de entrada da forma 6×6 . Como você pode ver, o bloco de redução de altura e largura aumenta o campo receptivo de cada elemento no mapa de recursos de saída.

```
def down_sample_blk(in_channels, out_channels):
    blk = []
    for _ in range(2):
        blk.append(nn.Conv2d(in_channels, out_channels,
                              kernel_size=3, padding=1))
        blk.append(nn.BatchNorm2d(out_channels))
        blk.append(nn.ReLU())
        in_channels = out_channels
    blk.append(nn.MaxPool2d(2))
    return nn.Sequential(*blk)
```

Ao testar a computação direta no bloco de redução de altura e largura, podemos ver que ele altera o número de canais de entrada e divide a altura e a largura pela metade.

```
forward(torch.zeros((2, 3, 20, 20)), down_sample_blk(3, 10)).shape
```

```
torch.Size([2, 10, 10, 10])
```

Bloco de Rede Base

O bloco de rede básico é usado para extrair recursos das imagens originais. Para simplificar o cálculo, construiremos uma pequena rede de base. Essa rede consiste em três blocos de *downsample* de altura e largura conectados em série, portanto, dobra o número de canais em cada etapa. Quando inserimos uma imagem original com a forma 256×256 , o bloco de rede base produz um mapa de características com a forma 32×32 .

```
def base_net():
    blk = []
    num_filters = [3, 16, 32, 64]
    for i in range(len(num_filters) - 1):
        blk.append(down_sample_blk(num_filters[i], num_filters[i+1]))
    return nn.Sequential(*blk)
```

```
forward(torch.zeros((2, 3, 256, 256)), base_net()).shape
```

```
torch.Size([2, 64, 32, 32])
```

O Modelo Completo

O modelo SSD contém um total de cinco módulos. Cada módulo produz um mapa de recursos usado para gerar caixas de âncora e prever as categorias e deslocamentos dessas caixas de âncora. O primeiro módulo é o bloco de rede base, os módulos de dois a quatro são blocos de redução de amostragem de altura e largura e o quinto módulo é um bloco global camada de pooling máxima que reduz a altura e largura para 1. Portanto, os módulos dois a cinco são todos blocos de recursos multiescala mostrados em Fig. 13.7.1.

```
def get_blk(i):
    if i == 0:
        blk = base_net()
    elif i == 1:
        blk = down_sample_blk(64, 128)
    elif i == 4:
        blk = nn.AdaptiveMaxPool2d((1,1))
    else:
        blk = down_sample_blk(128, 128)
    return blk
```

Agora, vamos definir o processo de computação progressiva para cada módulo. Em contraste com as redes neurais convolucionais descritas anteriormente, este módulo não só retorna a saída do mapa de características Y por computação convolucional, mas também as caixas de âncora da escala atual gerada a partir de Y e suas categorias e deslocamentos previstos.

```
def blk_forward(X, blk, size, ratio, cls_predictor, bbox_predictor):
    Y = blk(X)
    anchors = d2l.multibox_prior(Y, sizes=size, ratios=ratio)
    cls_preds = cls_predictor(Y)
    bbox_preds = bbox_predictor(Y)
    return (Y, anchors, cls_preds, bbox_preds)
```

Como mencionamos, quanto mais próximo um bloco de recursos multiescala está do topo em Fig. 13.7.1, maiores são os objetos que ele detecta e maiores são as caixas de âncora que deve gerar. Aqui, primeiro dividimos o intervalo de 0,2 a 1,05 em cinco partes iguais para determinar os tamanhos das caixas de âncora menores em escalas diferentes: 0,2, 0,37, 0,54, etc. Então, de acordo com $\sqrt{0.2 \times 0.37} = 0.272$, $\sqrt{0.37 \times 0.54} = 0.447$, e fórmulas semelhantes, determinamos os tamanhos de caixas de âncora maiores em escalas diferentes.

```
sizes = [[0.2, 0.272], [0.37, 0.447], [0.54, 0.619], [0.71, 0.79],
         [0.88, 0.961]]
ratios = [[1, 2, 0.5]] * 5
num_anchors = len(sizes[0]) + len(ratios[0]) - 1
```

Agora, podemos definir o modelo completo, TinySSD.

```
class TinySSD(nn.Module):
    def __init__(self, num_classes, **kwargs):
        super(TinySSD, self).__init__(**kwargs)
        self.num_classes = num_classes
        idx_to_in_channels = [64, 128, 128, 128, 128]
        for i in range(5):
            # The assignment statement is self.blk_i = get_blk(i)
            setattr(self, f'blk_{i}', get_blk(i))
            setattr(self, f'cls_{i}', cls_predictor(idx_to_in_channels[i],
                                                    num_anchors, num_classes))
            setattr(self, f'bbox_{i}', bbox_predictor(idx_to_in_channels[i],
                                                    num_anchors))

    def forward(self, X):
        anchors, cls_preds, bbox_preds = [None] * 5, [None] * 5, [None] * 5
        for i in range(5):
            # getattr(self, 'blk_%d' % i) accesses self.blk_i
            X, anchors[i], cls_preds[i], bbox_preds[i] = blk_forward(
                X, getattr(self, f'blk_{i}'), sizes[i], ratios[i],
                getattr(self, f'cls_{i}'), getattr(self, f'bbox_{i}'))
        # In the reshape function, 0 indicates that the batch size remains
        # unchanged
        anchors = torch.cat(anchors, dim=1)
        cls_preds = concat_preds(cls_preds)
        cls_preds = cls_preds.reshape(
            cls_preds.shape[0], -1, self.num_classes + 1)
        bbox_preds = concat_preds(bbox_preds)
        return anchors, cls_preds, bbox_preds
```

Agora criamos uma instância de modelo SSD e a usamos para realizar cálculos avançados no mini-batch de imagem X , que tem uma altura e largura de 256 pixels. Como verificamos anteriormente, o primeiro módulo gera um mapa de recursos com a forma 32×32 . Como os módulos dois a quatro são blocos de redução de altura e largura, o módulo cinco é uma camada de agrupamento global e cada elemento no mapa de recursos é usado como o centro para 4 caixas de âncora, um total de $(32^2 + 16^2 + 8^2 + 4^2 + 1) \times 4 = 5444$ caixas de âncora são geradas para cada imagem nas cinco escalas.

```
net = TinySSD(num_classes=1)
X = torch.zeros((32, 3, 256, 256))
anchors, cls_preds, bbox_preds = net(X)
```

(continues on next page)

```
print('output anchors:', anchors.shape)
print('output class preds:', cls_preds.shape)
print('output bbox preds:', bbox_preds.shape)
```

```
output anchors: torch.Size([1, 5444, 4])
output class preds: torch.Size([32, 5444, 2])
output bbox preds: torch.Size([32, 21776])
```

13.7.2 Treinamento

Agora, vamos explicar, passo a passo, como treinar o modelo SSD para detecção de objetos.

Leitura e Inicialização de Dados

Lemos o conjunto de dados de detecção de banana que criamos na seção anterior.

```
batch_size = 32
train_iter, _ = d2l.load_data_bananas(batch_size)
```

```
read 1000 training examples
read 100 validation examples
```

Existe 1 categoria no conjunto de dados de detecção de banana. Depois de definir o módulo, precisamos inicializar os parâmetros do modelo e definir o algoritmo de otimização.

```
device, net = d2l.try_gpu(), TinySSD(num_classes=1)
trainer = torch.optim.SGD(net.parameters(), lr=0.2, weight_decay=5e-4)
```

Definindo Funções de Perda e Avaliação

A detecção de objetos está sujeita a dois tipos de perdas. a primeira é a perda da categoria da caixa de âncora. Para isso, podemos simplesmente reutilizar a função de perda de entropia cruzada que usamos na classificação de imagens. A segunda perda é a perda de deslocamento da caixa de âncora positiva. A previsão de deslocamento é um problema de normalização. No entanto, aqui, não usamos a perda quadrática introduzida anteriormente. Em vez disso, usamos a perda de norma L_1 , que é o valor absoluto da diferença entre o valor previsto e o valor verdadeiro. A variável de máscara `bbox_masks` remove caixas de âncora negativas e caixas de âncora de preenchimento do cálculo de perda. Finalmente, adicionamos a categoria de caixa de âncora e compensamos as perdas para encontrar a função de perda final para o modelo.

```
cls_loss = nn.CrossEntropyLoss(reduction='none')
bbox_loss = nn.L1Loss(reduction='none')

def calc_loss(cls_preds, cls_labels, bbox_preds, bbox_labels, bbox_masks):
    batch_size, num_classes = cls_preds.shape[0], cls_preds.shape[2]
```

(continues on next page)

```

cls = cls_loss(cls_preds.reshape(-1, num_classes),
               cls_labels.reshape(-1)).reshape(batch_size, -1).mean(dim=1)
bbox = bbox_loss(bbox_preds * bbox_masks,
                 bbox_labels * bbox_masks).mean(dim=1)
return cls + bbox

```

Podemos usar a taxa de precisão para avaliar os resultados da classificação. Como usamos a perda de norma L_1 , usaremos o erro absoluto médio para avaliar os resultados da previsão da caixa delimitadora.

```

def cls_eval(cls_preds, cls_labels):
    # Because the category prediction results are placed in the final
    # dimension, argmax must specify this dimension
    return float((cls_preds.argmax(dim=-1).type(
        cls_labels.dtype) == cls_labels).sum())

def bbox_eval(bbox_preds, bbox_labels, bbox_masks):
    return float((torch.abs((bbox_labels - bbox_preds) * bbox_masks)).sum())

```

Treinando o Modelo

Durante o treinamento do modelo, devemos gerar caixas de âncora multiescala (âncoras) no processo de computação direta do modelo e prever a categoria (cls_preds) e o deslocamento (bbox_preds) para cada caixa de âncora. Depois, rotulamos a categoria (cls_labels) e o deslocamento (bbox_labels) de cada caixa de âncora gerada com base nas informações do rótulo Y. Finalmente, calculamos a função de perda usando a categoria predita e rotulada e os valores de compensação. Para simplificar o código, não avaliamos o conjunto de dados de treinamento aqui.

```

num_epochs, timer = 20, d2l.Timer()
animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                        legend=['class error', 'bbox mae'])

net = net.to(device)
for epoch in range(num_epochs):
    # accuracy_sum, mae_sum, num_examples, num_labels
    metric = d2l.Accumulator(4)
    net.train()
    for features, target in train_iter:
        timer.start()
        trainer.zero_grad()
        X, Y = features.to(device), target.to(device)
        # Generate multiscale anchor boxes and predict the category and
        # offset of each
        anchors, cls_preds, bbox_preds = net(X)
        # Label the category and offset of each anchor box
        bbox_labels, bbox_masks, cls_labels = d2l.multibox_target(anchors, Y)
        # Calculate the loss function using the predicted and labeled
        # category and offset values
        l = calc_loss(cls_preds, cls_labels, bbox_preds, bbox_labels,
                      bbox_masks)
        l.mean().backward()
        trainer.step()

```

(continues on next page)


```

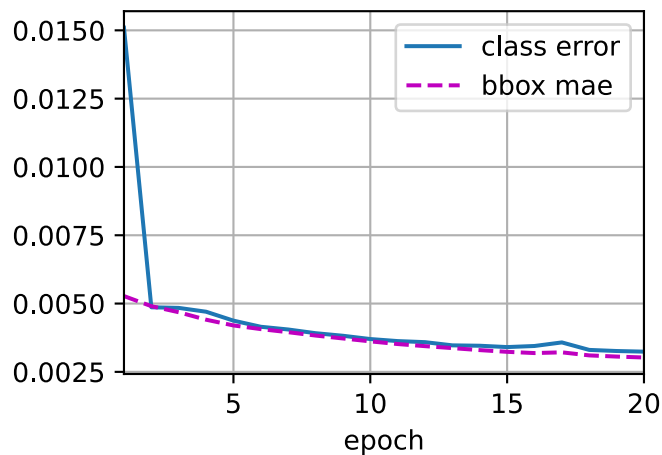
        metric.add(cls_eval(cls_preds, cls_labels), cls_labels.numel(),
                    bbox_eval(bbox_preds, bbox_labels, bbox_masks),
                    bbox_labels.numel())
    cls_err, bbox_mae = 1 - metric[0] / metric[1], metric[2] / metric[3]
    animator.add(epoch + 1, (cls_err, bbox_mae))
    print(f'class err {cls_err:.2e}, bbox mae {bbox_mae:.2e}')
    print(f'{len(train_iter.dataset) / timer.stop():.1f} examples/sec on '
          f'{str(device)}')

```

```

class err 3.24e-03, bbox mae 3.03e-03
5130.7 examples/sec on cuda:0

```



13.7.3 Predição

Na fase de previsão, queremos detectar todos os objetos de interesse na imagem. Abaixo, lemos a imagem de teste e transformamos seu tamanho. Então, nós o convertemos para o formato quadridimensional exigido pela camada convolucional.

```

X = torchvision.io.read_image('../img/banana.jpg').unsqueeze(0).float()
img = X.squeeze(0).permute(1,2,0).long()

```

Usando a função `multibox_detection`, prevemos as caixas delimitadoras com base nas caixas de âncora e seus deslocamentos previstos. Em seguida, usamos a supressão não máxima para remover caixas delimitadoras semelhantes.

```

def predict(X):
    net.eval()
    anchors, cls_preds, bbox_preds = net(X.to(device))
    cls_probs = F.softmax(cls_preds, dim=2).permute(0, 2, 1)
    output = d2l.multibox_detection(cls_probs, bbox_preds, anchors)
    idx = [i for i, row in enumerate(output[0]) if row[0] != -1]
    return output[0, idx]

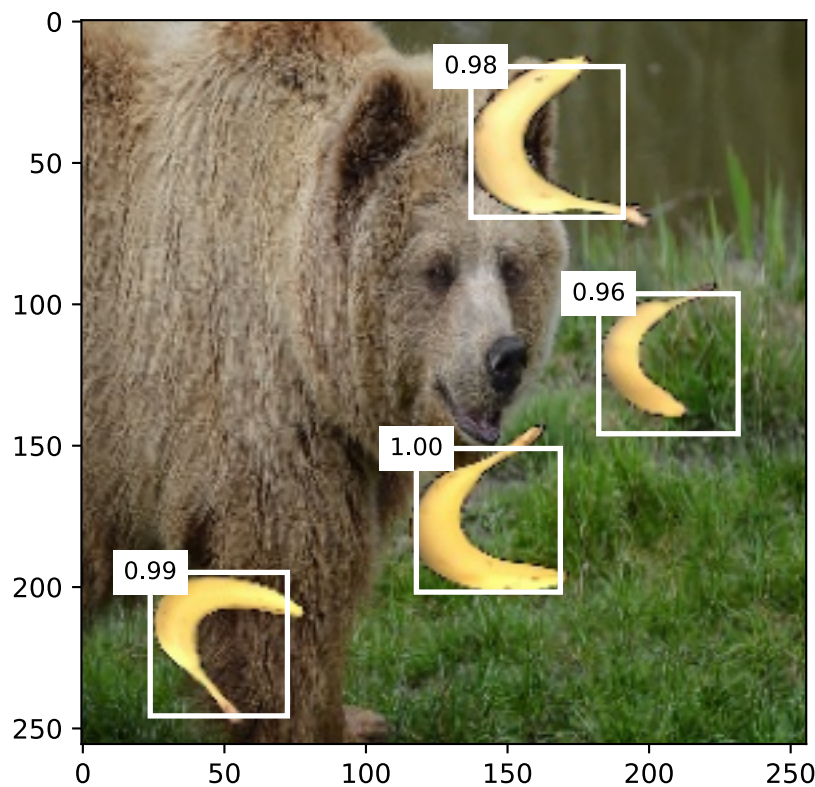
output = predict(X)

```

Por fim, pegamos todas as caixas delimitadoras com um nível de confiança de pelo menos 0,9 e as exibimos como a saída final.

```
def display(img, output, threshold):
    d2l.set_figsize((5, 5))
    fig = d2l.plt.imshow(img)
    for row in output:
        score = float(row[1])
        if score < threshold:
            continue
        h, w = img.shape[0:2]
        bbox = [row[2:6] * torch.tensor((w, h, w, h), device=row.device)]
        d2l.show_bboxes(fig.axes, bbox, '%.2f' % score, 'w')

display(img, output.cpu(), threshold=0.9)
```



13.7.4 Resumo

- SSD é um modelo de detecção de objetos multiescala. Este modelo gera diferentes números de caixas de âncora de tamanhos diferentes com base no bloco de rede de base e cada bloco de recursos multiescala e prevê as categorias e deslocamentos das caixas de âncora para detectar objetos de tamanhos diferentes.
- Durante o treinamento do modelo SSD, a função de perda é calculada usando a categoria prevista e rotulada e os valores de deslocamento.

13.7.5 Exercícios

1. Devido a limitações de espaço, ignoramos alguns dos detalhes de implementação do modelo SSD neste experimento. Você pode melhorar ainda mais o modelo nas seguintes áreas?

Função de Perda

A. Para as compensações previstas, substitua L_1 perda de norma por L_1 de perda de regularização. Esta função de perda usa uma função quadrada em torno de zero para maior suavidade. Esta é a área regularizada controlada pelo hiperparâmetro σ :

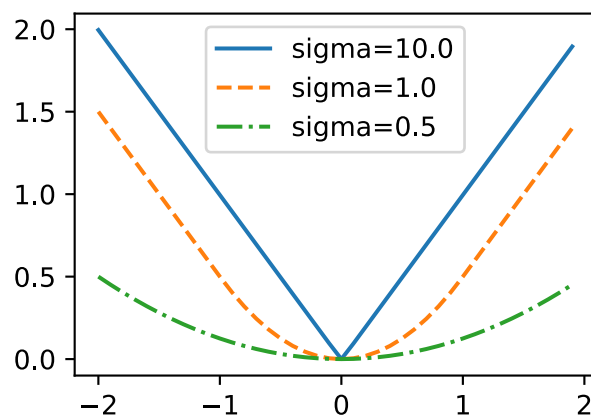
$$f(x) = \begin{cases} (\sigma x)^2/2, & \text{if } |x| < 1/\sigma^2 \\ |x| - 0.5/\sigma^2, & \text{otherwise} \end{cases} \quad (13.7.1)$$

Quando σ é grande, essa perda é semelhante à perda normal de L_1 . Quando o valor é pequeno, a função de perda é mais suave.

```
def smooth_l1(data, scalar):
    out = []
    for i in data:
        if abs(i) < 1 / (scalar ** 2):
            out.append(((scalar * i) ** 2) / 2)
        else:
            out.append(abs(i) - 0.5 / (scalar ** 2))
    return torch.tensor(out)

sigmas = [10, 1, 0.5]
lines = ['-', '--', '-.']
x = torch.arange(-2, 2, 0.1)
d2l.set_figsize()

for l, s in zip(lines, sigmas):
    y = smooth_l1(x, scalar=s)
    d2l.plt.plot(x, y, l, label='sigma=%.1f' % s)
d2l.plt.legend();
```



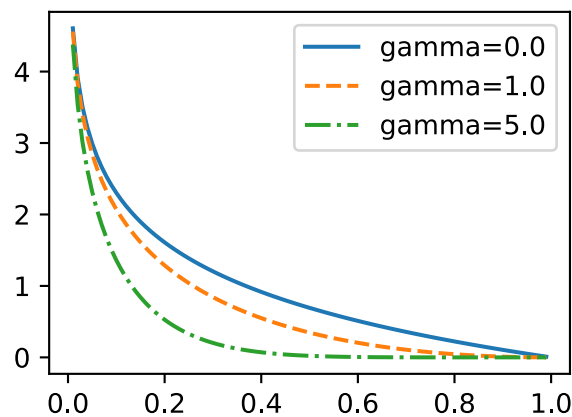
No experimento, usamos a perda de entropia cruzada para a previsão da categoria. Agora, assuma que a probabilidade de predição da categoria real j é p_j e a perda de entropia cruzada é $-\log p_j$.

Também podemos usar a perda focal (Lin et al., 2017a). Dados os hiperparâmetros positivos γ e α , essa perda é definida como:

$$-\alpha(1 - p_j)^\gamma \log p_j. \quad (13.7.2)$$

Como você pode ver, ao aumentar γ , podemos efetivamente reduzir a perda quando a probabilidade de prever a categoria correta for alta.

```
def focal_loss(gamma, x):  
    return -(1 - x) ** gamma * torch.log(x)  
  
x = torch.arange(0.01, 1, 0.01)  
for l, gamma in zip(lines, [0, 1, 5]):  
    y = d2l.plt.plot(x, focal_loss(gamma, x), l, label='gamma=%.1f' % gamma)  
d2l.plt.legend();
```



Treinamento e Previsão

B. Quando um objeto é relativamente grande em comparação com a imagem, o modelo normalmente adota um tamanho de imagem de entrada maior.

C. Isso geralmente produz um grande número de caixas de âncora negativas ao rotular as categorias da caixa de âncora. Podemos amostrar as caixas de âncora negativas para equilibrar melhor as categorias de dados. Para fazer isso, podemos definir um parâmetro `negative_mining_ratio` na função `multibox_target`.

D. Atribuir hiperparâmetros com pesos diferentes para a perda de categoria da caixa de âncora e a perda de deslocamento da caixa de âncora positiva na função de perda.

E. Consulte o documento SSD. Quais métodos podem ser usados para avaliar a precisão dos modelos de detecção de objetos (Liu et al., 2016)?

Discussões¹⁶²

¹⁶² <https://discuss.d2l.ai/t/1604>

13.8 Region-based CNNs (R-CNNs)

Redes neurais convolucionais baseadas em regiões ou regiões com recursos CNN (R-CNNs) são uma abordagem pioneira que aplica modelos profundos para detecção de objetos (Girshick et al., 2014). Nesta seção, discutiremos R-CNNs e uma série de melhorias feitas a eles: Fast R-CNN (Girshick, 2015), Faster R-CNN (Ren et al., 2015) e Mask R-CNN (He et al., 2017). Devido às limitações de espaço, limitaremos nossa discussão aos designs desses modelos.

13.8.1 R-CNNs

Os modelos R-CNN primeiro selecionam várias regiões propostas de uma imagem (por exemplo, as caixas de âncora são um tipo de método de seleção) e, em seguida, rotulam suas categorias e caixas delimitadoras (por exemplo, deslocamentos). Em seguida, eles usam uma CNN para realizar cálculos avançados para extrair recursos de cada área proposta. Depois, usamos os recursos de cada região proposta para prever suas categorias e caixas delimitadoras. Fig. 13.8.1 mostra um modelo R-CNN.

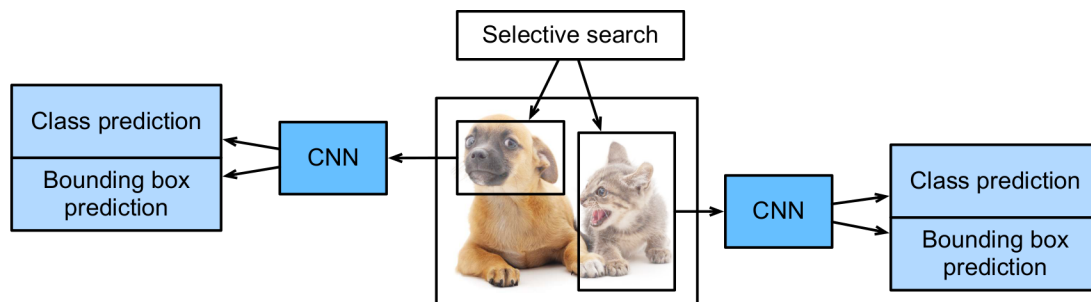


Fig. 13.8.1: Modelo R-CNN.

Especificamente, os R-CNNs são compostos por quatro partes principais:

1. A pesquisa seletiva é realizada na imagem de entrada para selecionar várias regiões propostas de alta qualidade (Uijlings et al., 2013). Essas regiões propostas são geralmente selecionadas em várias escalas e têm diferentes formas e tamanhos. A categoria e a caixa delimitadora da verdade fundamental de cada região proposta é etiquetada.
2. Um CNN pré-treinado é selecionado e colocado, de forma truncada, antes da camada de saída. Ele transforma cada região proposta nas dimensões de entrada exigido pela rede e usa computação direta para gerar os recursos extraídos das regiões propostas.
3. Os recursos e a categoria rotulada de cada região proposta são combinados como um exemplo para treinar várias máquinas de vetores de suporte para classificação de objeto. Aqui, cada máquina de vetor de suporte é usada para determinar se um exemplo pertence a uma determinada categoria.
4. Os recursos e a caixa delimitadora rotulada de cada região proposta são combinados como um exemplo para treinar um modelo de regressão linear para a predição da caixa delimitadora de verdade básica.

Embora os modelos R-CNN usem CNNs pré-treinados para extrair recursos de imagem com eficácia, a principal desvantagem é a velocidade lenta. Como você pode imaginar, podemos selecionar

milhares de regiões propostas a partir de uma única imagem, exigindo milhares de cálculos diretos da CNN para realizar a detecção de objetos. Essa enorme carga de computação significa que os R-CNNs não são amplamente usados em aplicativos reais.

13.8.2 Fast R-CNN

O principal gargalo de desempenho de um modelo R-CNN é a necessidade de extrair recursos de forma independente para cada região proposta. Como essas regiões têm um alto grau de sobreposição, a extração de recursos independentes resulta em um alto volume de cálculos repetitivos. Fast R-CNN melhora o R-CNN por apenas executar CNN computação progressiva na imagem como um todo.

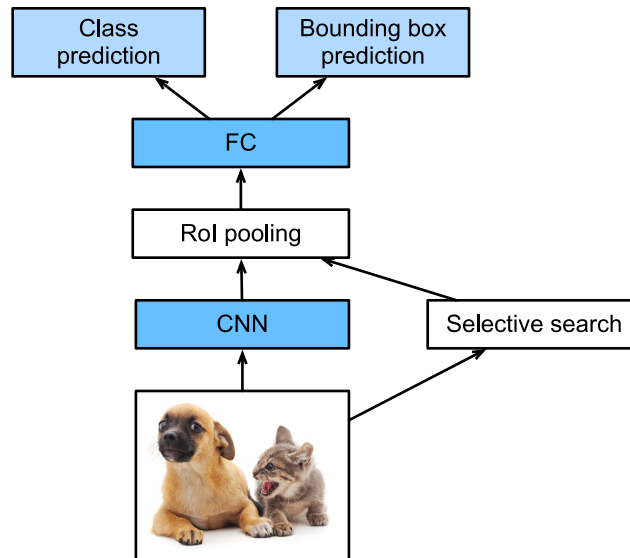


Fig. 13.8.2: Modelo Fast R-CNN.

Fig. 13.8.2 mostra um modelo Fast R-CNN. Suas principais etapas de computação são descritas abaixo:

1. Comparado a um modelo R-CNN, um modelo Fast R-CNN usa a imagem inteira como o Entrada da CNN para extração de recursos, em vez de cada região proposta. Além disso, esta rede é geralmente treinada para atualizar os parâmetros do modelo. Enquanto a entrada é uma imagem inteira, a forma de saída do CNN é $1 \times c \times h_1 \times w_1$.
2. Supondo que a pesquisa seletiva gere n regiões propostas, suas diferentes formas indicam regiões de interesses (RoIs) de diferentes formas na CNN resultado. Características das mesmas formas devem ser extraídas dessas RoIs (aqui assumimos que a altura é h_2 e a largura é w_2). R-CNN rápido apresenta o pool de RoI, que usa a saída CNN e RoIs como entrada para saída uma concatenação dos recursos extraídos de cada região proposta com o forma $n \times c \times h_2 \times w_2$.
3. Uma camada totalmente conectada é usada para transformar a forma de saída em $n \times d$, onde d é determinado pelo design do modelo.
4. Durante a previsão da categoria, a forma da saída da camada totalmente conectada é novamente transformada em $n \times q$ e usamos a regressão softmax (q é o número de categorias). Durante a previsão da caixa delimitadora, a forma da saída da camada conectada é nova-

mente transformada em $n \times 4$. Isso significa que prevemos a categoria e a caixa delimitadora para cada região proposta.

A camada de pooling de RoI no Fast R-CNN é um pouco diferente das camadas de pool que discutimos antes. Em uma camada de pooling normal, definimos a janela de pool, preenchimento e passo para controlar a forma de saída. Em uma camada de pooling de RoI, podemos especificar diretamente a forma de saída de cada região, como especificar a altura e a largura de cada região como h_2, w_2 . Supondo que o altura e largura da janela RoI são h e w , esta janela é dividida em uma grade de subjanelas com a forma $h_2 \times w_2$. **O tamanho de cada subjanela é de cerca de $(h/h_2) \times (w/w_2)$** . A altura e largura da subjanela devem ser sempre inteiros e o maior elemento é usado como saída para um determinada subjanela. Isso permite que a camada de pooling de RoI extraia recursos do mesmo formato de RoIs de formatos diferentes.

Em Fig. 13.8.3, selecionamos uma região 3×3 como um RoI da entrada 4×4 . Para este RoI, usamos uma camada de pool de 2×2 RoI para obter uma única saída 2×2 . Quando dividimos a região em quatro subjanelas, elas contêm respectivamente os elementos 0, 1, 4 e 5 (5 é o maior); 2 e 6 (6 é o maior); 8 e 9 (9 é o maior); e 10.

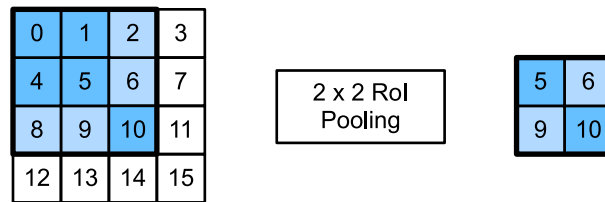


Fig. 13.8.3: Camada de *pooling* RoI 2×2 .

Usamos a função `roi_pool` de `torchvision` para demonstrar a computação da camada de pooling RoI. Suponha que a CNN extraia o elemento X com altura e largura 4 e apenas um único canal.

```
import torch
import torchvision
```

```
X = torch.arange(16.).reshape(1, 1, 4, 4)
X
```

```
tensor([[[[ 0.,  1.,  2.,  3.],
           [ 4.,  5.,  6.,  7.],
           [ 8.,  9., 10., 11.],
           [12., 13., 14., 15.]])])
```

Suponha que a altura e a largura da imagem sejam de 40 pixels e que a busca seletiva gere duas regiões propostas na imagem. Cada região é expressa como cinco elementos: a categoria de objeto da região e as coordenadas x, y de seus cantos superior esquerdo e inferior direito.

```
rois = torch.Tensor([[0, 0, 0, 20, 20], [0, 0, 10, 30, 30]])
```

Como a altura e largura de X são 1/10 da altura e largura da imagem, as coordenadas das duas regiões propostas são multiplicadas por 0,1 de acordo com `escala_espacial`, e então os RoIs são rotulados em X como $X[:, :, 0: 3, 0: 3]$ e $X[:, :, 1: 4, 0: 4]$, respectivamente. Por fim, dividimos os dois RoIs em uma grade de subjanela e extraímos recursos com altura e largura 2.


```
torchvision.ops.roi_pool(X, rois, output_size=(2, 2), spatial_scale=0.1)
```

```
tensor([[[[ 5.,  6.],  
          [ 9., 10.]]],  
  
        [[[ 9., 11.],  
          [13., 15.]]]])
```

13.8.3 R-CNN Mais Rápido

A fim de obter resultados precisos de detecção de objeto, Fast R-CNN geralmente requer que muitas regiões propostas sejam geradas em busca seletiva. O Faster R-CNN substitui a pesquisa seletiva por uma rede de proposta regional. Isso reduz o número de regiões propostas geradas, garantindo a detecção precisa do objeto.

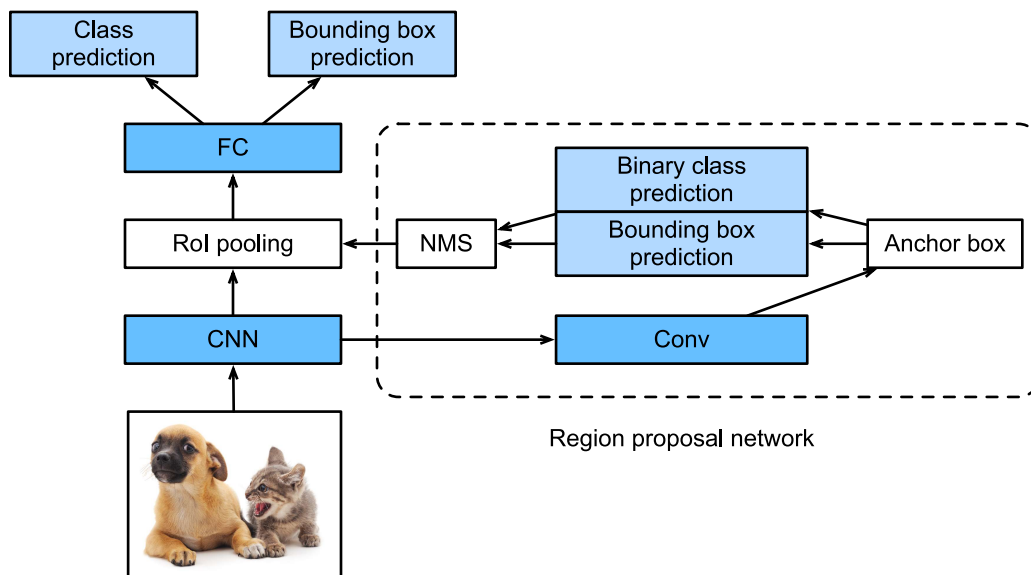


Fig. 13.8.4: Modelo R-CNN mais rápido.

Fig. 13.8.4 mostra um modelo Faster R-CNN. Comparado ao Fast R-CNN, o Faster R-CNN apenas muda o método para gerar regiões propostas de pesquisa seletiva para rede de proposta de região. As outras partes do modelo permanecem inalteradas. O processo de computação de rede de proposta de região detalhada é descrito abaixo:

1. Usamos uma camada convolucional 3×3 com um preenchimento de 1 para transformar a saída CNN e definir o número de canais de saída para c . Assim, cada elemento no mapa de recursos que a CNN extrai da imagem é um novo recurso com um comprimento de c .
2. Usamos cada elemento no mapa de recursos como um centro para gerar várias caixas de âncora de diferentes tamanhos e proporções de aspecto e, em seguida, etiquetá-las.
3. Usamos os recursos dos elementos de comprimento c no centro da âncora caixas para prever a categoria binária (objeto ou fundo) e caixa delimitadora para suas respectivas caixas de âncora.

4. Em seguida, usamos a supressão não máxima para remover resultados semelhantes da caixa delimitadora que correspondem às previsões da categoria de “objeto”. Finalmente, nós produzimos as caixas delimitadoras previstas como as regiões propostas exigidas pelo *pooling* de RoI camada.

É importante notar que, como parte do modelo R-CNN mais rápido, a rede proposta da região é treinada em conjunto com o resto do modelo. Além disso, as funções de objeto do Faster R-CNN incluem as previsões de categoria e caixa delimitadora na detecção de objetos, bem como a categoria binária e previsões de caixa delimitadora para as caixas de âncora na rede de proposta da região. Finalmente, a rede proposta de região pode aprender como gerar regiões propostas de alta qualidade, o que reduz o número de regiões propostas, enquanto mantém a precisão da detecção de objetos.

13.8.4 Máscara R-CNN

Se os dados de treinamento forem rotulados com as posições de nível de pixel de cada objeto em uma imagem, um modelo Mask R-CNN pode usar efetivamente esses rótulos detalhados para melhorar ainda mais a precisão da detecção de objeto.

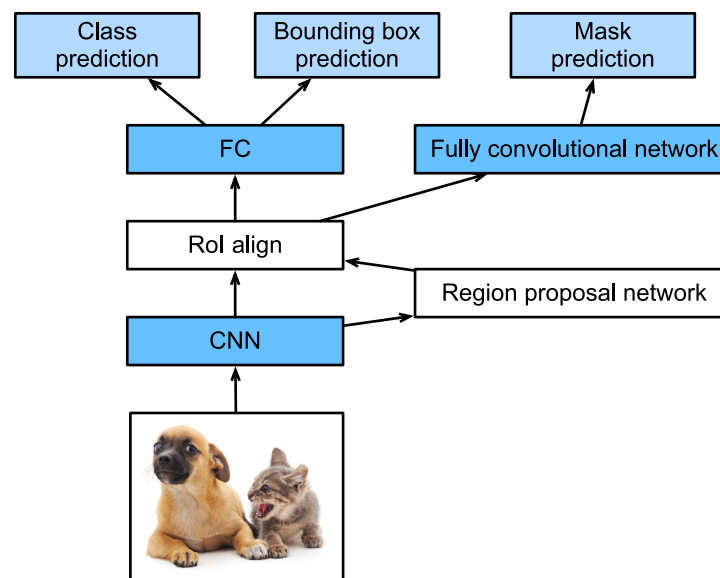


Fig. 13.8.5: Modelo de máscara R-CNN.

Conforme mostrado em Fig. 13.8.5, Mask R-CNN é uma modificação do modelo Faster R-CNN. Os modelos de máscara R-CNN substituem a camada de *pooling* RoI por uma camada de alinhamento RoI. Isso permite o uso de interpolação bilinear para reter informações espaciais em mapas de características, tornando o Mask R-CNN mais adequado para previsões em nível de pixel. A camada de alinhamento de RoI produz mapas de recursos do mesmo formato para todos os RoIs. Isso não apenas prevê as categorias e caixas delimitadoras de RoIs, mas nos permite usar uma rede totalmente convolucional adicional para prever as posições de objetos em nível de pixel. Descreveremos como usar redes totalmente convolucionais para prever a semântica em nível de pixel em imagens posteriormente neste capítulo.

13.8.5 Resumo

- Um modelo R-CNN seleciona várias regiões propostas e usa um CNN para realizar computação direta e extrair os recursos de cada região proposta. Em seguida, usa esses recursos para prever as categorias e caixas delimitadoras das regiões propostas.
- Fast R-CNN melhora o R-CNN realizando apenas cálculos de encaminhamento de CNN na imagem como um todo. Ele apresenta uma camada de pooling de RoI para extrair recursos do mesmo formato de RoIs de formatos diferentes.
- O R-CNN mais rápido substitui a pesquisa seletiva usada no Fast R-CNN por uma rede de proposta de região. Isso reduz o número de regiões propostas geradas, garantindo a detecção precisa do objeto.
- Mask R-CNN usa a mesma estrutura básica que R-CNN mais rápido, mas adiciona uma camada de convolução completa para ajudar a localizar objetos no nível de pixel e melhorar ainda mais a precisão da detecção de objetos.

13.8.6 Exercícios

1. Estude a implementação de cada modelo no [kit de ferramentas GluonCV¹⁶³](#) relacionado a esta seção.

[Discussões¹⁶⁴](#)

13.9 Segmentação Semântica e o Dataset

no discussãooob os problemas de detecção de objetos nas seções anteriores, usamos apenas caixas delimitadoras retangulares para rotular os objetos em imagens. Nesta seção, veremos a segmentação semântica, que tenta, em vez de segmentar imagens em diferentes categorias semânticas. Essas regiões semânticas rotulam e prevêem objetos no nível do pixel. [Fig. 13.9.1](#) mostra uma imagem semanticamente segmentada, com áreas marcadas como “cachorro”, “gato” e “fundo”. Como você pode ver, em comparação com a detecção de objetos, a segmentação semântica rotula áreas com bordas em nível de pixel, para uma precisão significativamente maior.

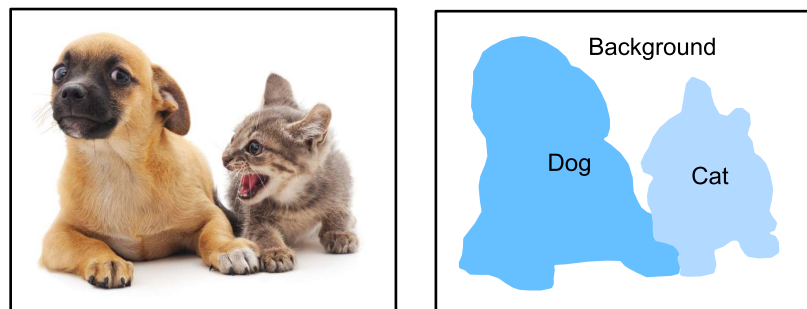


Fig. 13.9.1: Imagem segmentada semanticamente, com áreas rotuladas “cachorro”, “gato” e “plano de fundo”.

¹⁶³ <https://github.com/dmlc/gluon-cv/>

¹⁶⁴ <https://discuss.d2l.ai/t/1409>

13.9.1 Segmentação de Imagem e Segmentação de Detecção e Instância

No campo da visão computacional, existem dois métodos importantes relacionados à segmentação semântica: segmentação de detecção: imagens e segmentação de detecção e instâncias. Aqui, vamos distinguir esses conceitos da segmentação semântica da seguinte forma:

- A segmentação de imagem divide uma imagem em várias regiões into constituinte método regionado geralmente usa as correlações entre pixels em uma imagem. Durante o treinamento, os rótulos não são necessários para pixels de imagem. No entanto, durante a previsão, esse método não pode garantir que as regiões segmentadas em uma imagem em .10, a segmentação da imagem pode dividir o cão em duas regiões cobrindo a boca do cão e os olhos onde o preto é a corth and eyes where black is the proeminente e a outra color and the other cobrindo o resto do cão onde o amarelo é a cor the rest of the dog where yellow is the proeminente.
- A segmentação da instância também é chamada de detecção e segmentação simultâneas. Este método tenta identificar as regiões de nível de pixel de cada color.
- Instance segmentation is also called simultaneous detection and segmentation. This method attempts to identify the pixel-level regions of each object instância de objeto em uma em an prominent corin prominent instnc imagem.mIn cont semnticraste com a segmentação semântica, a segmentação de instância não apenas sgmentation distingue semânticas, ms o differetes instane objeto. Se umaes. I imagem contém dois cães, a segmentação de instância distingue quais pixels pertencem a cada cachorro.

13.9.2 O Conjunto de Dados de Segmentação Semântica Pascal VOC2012

No campo de segmentação, dois cães distinguem quais pixels pertencem a qual cão.

13.9.3 The Pascal VOC2012 Semantic Segmentation Dataset

Na segmentação, um conjunto de dados importante é o campo de segmentação, um importante conjunto de dados é o Pascal VOC2012¹⁶⁵. Para entender melhor este conjunto de dados, devemos primeiro obter o código necessário para o nosso experimento.

```
%matplotlib inline
import os
import torch
import torchvision
from d2l import torch as d2l
```

O site original pode ser instável, portanto, baixamos os dados de um site espelho. O arquivo tem cerca de 2 GB, por isso levará algum tempo para fazer o download, o conjunto de dados está localizado no download. decomp mi ../data/VOCdevkit/VOC2012.

```
#save
d2l.DATA_HUB['voc2012'] = (d2l.DATA_URL + 'VOCtrainval_11-May-2012.tar',
                           '4e443f8a2eca6b1dac8a6c57641b67dd40621a49')

voc_dir = d2l.download_extract('voc2012', 'VOCdevkit/VOC2012')
```

¹⁶⁵ <http://host.robots.ox.ac.uk/pascal/VOC/voc2012/>

```
Downloading ../data/VOCtrainval_11-May-2012.tar from http://d2l-data.s3-accelerate.amazonaws.com/VOCtrainval_11-May-2012.tar...
```

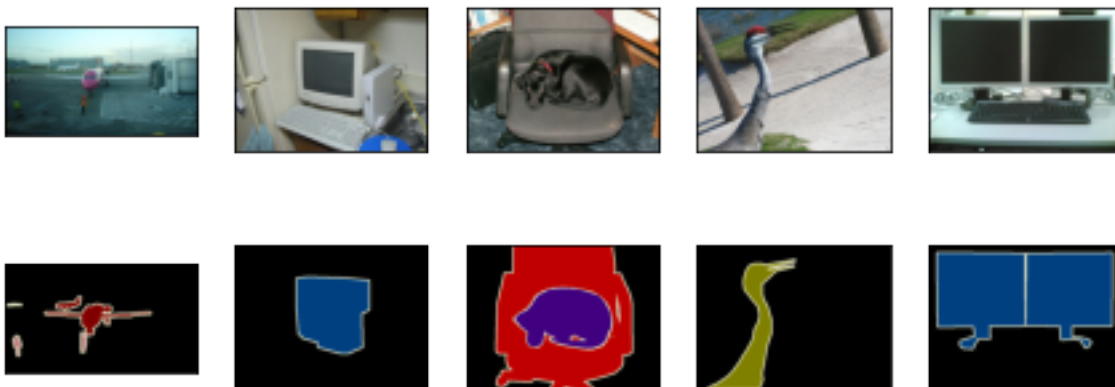
Vá para `../data/VOCdevkit/VOC2012` para ver as diferentes partes do conjunto de dados. O caminho `ImageSets/Segmentation` contém arquivos de texto que especificam os exemplos de treinamento e teste. Os cinco `JPEGImages` e `SegmentationClass` contêm as paths contain the example input imagens de entrada de exemplo e rótulos, respectivamente. Essas etiquetas também estão em formato de imagem, com as mesmas `and labels, respectively`. These labels are also in image format, with the same images `dimensões` das put imagens de entrada às quais correspondem. Nos rótulos, os pixels com a mesma cor pertencem à mesma.

```
#@save
def read_voc_images(voc_dir, is_train=True):
    """Read all VOC feature and label images."""
    txt_fname = os.path.join(voc_dir, 'ImageSets', 'Segmentation',
                              'train.txt' if is_train else 'val.txt')
    mode = torchvision.io.image.ImageReadMode.RGB
    with open(txt_fname, 'r') as f:
        images = f.read().split()
    features, labels = [], []
    for i, fname in enumerate(images):
        features.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'JPEGImages', f'{fname}.jpg')))
        labels.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'SegmentationClass', f'{fname}.png'), mode))
    return features, labels

train_features, train_labels = read_voc_images(voc_dir, True)
```

Desenhamos as primeiras cinco `We draw the first five input` imagens de entrada e seus rótulos. Nas imagens do rótulo, o branco `and their labels`. In the label images, white representa as bordas e o preto `represents the foreground` and black represents the background. Outras cores `represent the background`. Outras cores correspondem a diferentes categorias.

```
n = 5
imgs = train_features[0:n] + train_labels[0:n]
imgs = [img.permute(1,2,0) for img in imgs]
d2l.show_images(imgs, 2, n);
```



A seguir, listamos cada valor de cor RGB nos rótulos e as categoriaes que eles rl.

```
#@save
VOC_COLORMAP = [[0, 0, 0], [128, 0, 0], [0, 128, 0], [128, 128, 0],
                [0, 0, 128], [128, 0, 128], [0, 128, 128], [128, 128, 128],
                [64, 0, 0], [192, 0, 0], [64, 128, 0], [192, 128, 0],
                [64, 0, 128], [192, 0, 128], [64, 128, 128], [192, 128, 128],
                [0, 64, 0], [128, 64, 0], [0, 192, 0], [128, 192, 0],
                [0, 64, 128]]

#@save
VOC_CLASSES = ['background', 'aeroplane', 'bicycle', 'bird', 'boat',
               'bottle', 'bus', 'car', 'cat', 'chair', 'cow',
               'diningtable', 'dog', 'horse', 'motorbike', 'person',
               'potted plant', 'sheep', 'sofa', 'train', 'tv/monitor']
```

Depois de definir as duasAfter defining the two constantes acima, podemos encontrar facilmente o índice de categoria para cada pixel nos rótulosabove, we can easily find the category index for each pixel in the labels.

```
#@save
def build_colormap2label():
    """Build an RGB color to label mapping for segmentation."""
    colormap2label = torch.zeros(256 ** 3, dtype=torch.long)
    for i, colormap in enumerate(VOC_COLORMAP):
        colormap2label[(colormap[0]*256 + colormap[1])*256 + colormap[2]] = i
    return colormap2label

#@save
def voc_label_indices(colormap, colormap2label):
    """Map an RGB color to a label."""
    colormap = colormap.permute(1,2,0).numpy().astype('int32')
    idx = ((colormap[:, :, 0] * 256 + colormap[:, :, 1]) * 256
           + colormap[:, :, 2])
    return colormap2label[idx]
```

PFfor exeamplo, na primeira imagem dein the first exeamplo, o índice de categoria para a parte frontal do avião é 1 e o índice para o fundo é image, the cat 0.

```
y = voc_label_indices(train_labels[0], build_colormap2label())
y[105:115, 130:140], VOC_CLASSES[1]
```

```
(tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 1]],
        'aeroplane')
```

NosData Preprocessing

In the preceding capítulos anteriores, dimensionamos as imagens para que se ajustassem à forma de entrada do modelo. Na segmentação semântica, esse método exigiria que mapeamos novamente as categorias de pixels previstas de volta à imagem de entrada do tamanho original. Seria muito difícil fazer isso com a segmentação, this method would require us to re-map the predicted pixel categories back to the original-size input image. It would be very difficult to do this precisely, especialmente em regiões segmentadas com semanticas in segmented regions with diferentes. Para evitar esse problema, recortamos as imagens para definir as dimensões e não as dimensionamos. Especificamente, usamos o método de corte aleatório usado no aumento da imagem para cortar a mesma região do conjunto de dimensões e não as dimensionamos. Specifically, we use the random cropping method used in image augmentation to crop the same region from input images de entrada e seus rótulos and their labels.

```
#save
def voc_rand_crop(feature, label, height, width):
    """Randomly crop for both feature and label images."""
    rect = torchvision.transforms.RandomCrop.get_params(feature,
                                                         (height, width))
    feature = torchvision.transforms.functional.crop(feature, *rect)
    label = torchvision.transforms.functional.crop(label, *rect)
    return feature, label
```

```
imgs = []
for _ in range(n):
    imgs += voc_rand_crop(train_features[0], train_labels[0], 200, 300)

imgs = [img.permute(1,2,0) for img in imgs]
d2l.show_images(imgs[:2] + imgs[1::2], 2, n);
```



Datasets para Segmentação Semântica Personalizada

Usamos a classe Dataset herdada fornecida pelo Gluon para personalizar a classe de conjunto de dados de segmentação semântica VOCSegDataset. Implementando a função `__getitem__`, podemos acessar arbitrariamente a imagem de entrada com o índice `idx` e os índices de categoria para cada um de seus pixels do conjunto de dados. Como algumas vezes, we can arbitrarily access the input image with the index `idx` and the category indexes for each of its pixels from the dataset. As some imagens no conjunto de dados podem ser menores do que as dimensões de saída especificadas para corte aleatório, devemos remover essas no dataset may be smaller than the output dimensions specified for random cropping, we must remove these exemplos usando uma função `filter` personalizada. Além disso, definimos a função `normalize_image` para normalizar cada um dos três canais RGB das imagens de entrada e by using a custom filter function. In addition, we define the `normalize_image` function to normalize each of the three RGB channels of the input images.

```
#@save
class VOCSegDataset(torch.utils.data.Dataset):
    """A customized dataset to load VOC dataset."""

    def __init__(self, is_train, crop_size, voc_dir):
        self.transform = torchvision.transforms.Normalize(
            mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        self.crop_size = crop_size
        features, labels = read_voc_images(voc_dir, is_train=is_train)
        self.features = [self.normalize_image(feature)
                        for feature in self.filter(features)]
        self.labels = self.filter(labels)
        self.colormap2label = build_colormap2label()
        print('read ' + str(len(self.features)) + ' examples')

    def normalize_image(self, img):
        return self.transform(img.float())

    def filter(self, imgs):
        return [img for img in imgs if (
            img.shape[1] >= self.crop_size[0] and
            img.shape[2] >= self.crop_size[1])]

    def __getitem__(self, idx):
        feature, label = voc_rand_crop(self.features[idx], self.labels[idx],
                                      *self.crop_size)
        return (feature, voc_label_indices(label, self.colormap2label))

    def __len__(self):
        return len(self.features)
```

Lendo o Reading the Dataset

Usando a classe `VOCegDataset` personalizada, criamos o conjunto de treino e teste. Assumimos que a operação de corte aleatório produzindo set and testing set instances. We assume the random cropping operation output imagens no formato 320×480 . Abaixo, podemos ver o número de exemplos retidos nos conjuntos de treino e teste.

```
crop_size = (320, 480)
voc_train = VOCegDataset(True, crop_size, voc_dir)
voc_test = VOCegDataset(False, crop_size, voc_dir)
```

```
read 1114 examples
read 1078 examples
```

Definimos o tamanho do lote com `batch_size = 64` e definimos os iteradores para os conjuntos de treino e teste. Imprimimos a forma do primeiro batch. Em contraste com a classificação de imagens e o reconhecimento de objetos, os rótulos aqui são matrizes tridimensionais.

```
batch_size = 64
train_iter = torch.utils.data.DataLoader(voc_train, batch_size, shuffle=True,
                                         drop_last=True,
                                         num_workers=d2l.get_dataloader_workers())

for X, Y in train_iter:
    print(X.shape)
    print(Y.shape)
    break
```

```
torch.Size([64, 3, 320, 480])
torch.Size([64, 320, 480])
```

Juntando Tudo

Finalmente, definimos uma função `load_data_voc` que baixa e carrega este dataset.

Finally, we define a function `load_data_voc` that downloads and loads this dataset, e então retorna os iteradores de dados.

```
@save
def load_data_voc(batch_size, crop_size):
    """Download and load the VOC2012 semantic dataset."""
    voc_dir = d2l.download_extract('voc2012', os.path.join(
        'VOCdevkit', 'VOC2012'))
    num_workers = d2l.get_dataloader_workers()
    train_iter = torch.utils.data.DataLoader(
        VOCegDataset(True, crop_size, voc_dir), batch_size,
        shuffle=True, drop_last=True, num_workers=num_workers)
```

(continues on next page)

```
test_iter = torch.utils.data.DataLoader(
    VOCSegDataset(False, crop_size, voc_dir), batch_size,
    drop_last=True, num_workers=num_workers)
return train_iter, test_iter
```

13.9.4 Resumo

- A segmentação semântica analisa como asSummary
- Semantic segmentation looks at how imagens podem ser segmentadas em regiões com can be segmented into regions with diferentes categorias semânticas.
- No campo de segmentação semântica, um conjunto de dados importante é semantic categories.
- In the semantic segmentation field, one important dataset is Pascal VOC2012.
- Como asBecause the input imagens e rótulos de entrada na segmentação semântica têm uma and labels in semantic segmentation have a one-to-one correspondência um a um no nível do pixel, nós os cortamos aleatoriamente em um tamanho fixo, em vez de dimensioná-lose at the pixel level, we randomly crop them to a fixed size, rather than scaling them.

13.9.5 Exercícios

1. Lembre-se do conteúdo que vimos emRecall the content we covered in [Section 13.1](#). Qual dos métodos dWhich of the image augmento de imagem usados na classificação de imagens seria difícil de usar na segmentação semânticaation methods used in image classification would be hard to use in semantic segmentation?

Discussões¹⁶⁶

13.10 Convolação Transposta

As camadas que apresentamos até agora para redes neurais convolucionais, incluindo camadas convolucionais ([Section 6.2](#)) e camadas de pooling ([Section 6.5](#)), geralmente reduzem a largura e altura de entrada ou as mantêm inalteradas. Aplicativos como segmentação semântica ([sec_semantic_segmentation](#)) e redes adversárias geradoras ([Section 17.2](#)), no entanto, exigem prever valores para cada pixel e, portanto, precisam aumentar a largura e altura de entrada. A convolação transposta, também chamada de convolação fracionada ([Dumoulin & Visin, 2016](#)) ou deconvolação ([Long et al., 2015](#)), serve a este propósito.

```
import torch
from torch import nn
from d2l import torch as d2l
```

¹⁶⁶ <https://discuss.d2l.ai/t/1480>

13.10.1 Convolução Transposta 2D Básica

Vamos considerar um caso básico em que os canais de entrada e saída são 1, com 0 preenchimento e 1 passo. Fig. 13.10.1 ilustra como a convolução transposta com um *kernel* 2×2 é calculada na matriz de entrada 2×2 .

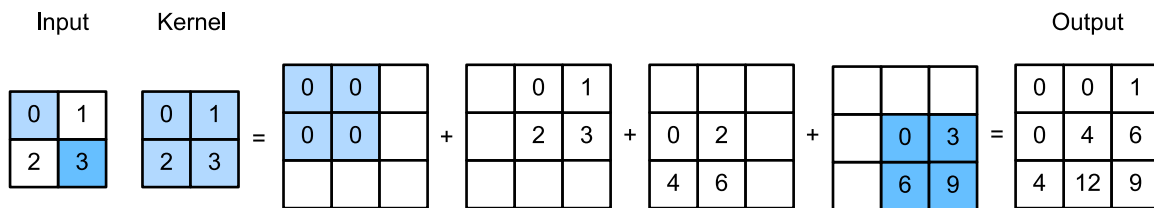


Fig. 13.10.1: Camada de convolução transposta com um *kernel* 2×2 .

Podemos implementar essa operação fornecendo o *kernel* da matriz *K* e a entrada da matriz *X*.

```
def trans_conv(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0] + h - 1, X.shape[1] + w - 1))
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            Y[i: i + h, j: j + w] += X[i, j] * K
    return Y
```

Lembre-se de que a convolução calcula os resultados por $Y[i, j] = (X[i: i + h, j: j + w] * K).sum()$ (consulte `corr2d` em [Section 6.2](#)), que resume os valores de entrada por meio do *kernel*. Enquanto a convolução transposta transmite valores de entrada por meio do *kernel*, o que resulta em uma forma de saída maior.

Verifique os resultados em [Fig. 13.10.1](#).

```
X = torch.tensor([[0., 1], [2, 3]])
K = torch.tensor([[0., 1], [2, 3]])
trans_conv(X, K)
```

```
tensor([[ 0.,  0.,  1.],
        [ 0.,  4.,  6.],
        [ 4., 12.,  9.]])
```

Ou podemos usar `nn.ConvTranspose2d` para obter os mesmos resultados. Como `nn.Conv2d`, tanto a entrada quanto o *kernel* devem ser tensores 4-D.

```
X, K = X.reshape(1, 1, 2, 2), K.reshape(1, 1, 2, 2)
tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, bias=False)
tconv.weight.data = K
tconv(X)
```

```
tensor([[[[ 0.,  0.,  1.],
           [ 0.,  4.,  6.],
           [ 4., 12.,  9.]]]], grad_fn=<SlowConvTranspose2DBackward>)
```

13.10.2 Preenchimento, Passos e Canais

Aplicamos elementos de preenchimento à entrada em convolução, enquanto eles são aplicados à saída em convolução transposta. Um preenchimento 1×1 significa que primeiro calculamos a saída como normal e, em seguida, removemos as primeiras/últimas linhas e colunas.

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, padding=1, bias=False)
tconv.weight.data = K
tconv(X)
```

```
tensor([[[[4.]]]], grad_fn=<SlowConvTranspose2DBackward>)
```

Da mesma forma, os avanços também são aplicados às saídas.

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, stride=2, bias=False)
tconv.weight.data = K
tconv(X)
```

```
tensor([[[[0., 0., 0., 1.],
          [0., 0., 2., 3.],
          [0., 2., 0., 3.],
          [4., 6., 6., 9.]]]], grad_fn=<SlowConvTranspose2DBackward>)
```

A extensão multicanal da convolução transposta é igual à convolução. Quando a entrada tem vários canais, denotados por c_i , a convolução transposta atribui uma matriz de *kernel* $k_h \times k_w$ a cada canal de entrada. Se a saída tem um tamanho de canal c_o , então temos um *kernel* $c_i \times k_h \times k_w$ para cada canal de saída.

Como resultado, se alimentarmos X em uma camada convolucional f para calcular $Y = f(X)$ e criarmos uma camada de convolução transposta g com os mesmos hiperparâmetros de f , exceto para o conjunto de canais de saída para ter o tamanho do canal de X , então $g(Y)$ deve ter o mesmo formato que X . Deixe-nos verificar esta afirmação.

```
X = torch.rand(size=(1, 10, 16, 16))
conv = nn.Conv2d(10, 20, kernel_size=5, padding=2, stride=3)
tconv = nn.ConvTranspose2d(20, 10, kernel_size=5, padding=2, stride=3)
tconv(conv(X)).shape == X.shape
```

```
True
```

13.10.3 Analogia à Transposição de Matriz

A convolução transposta leva o nome da transposição da matriz. Na verdade, as operações de convolução também podem ser realizadas por multiplicação de matrizes. No exemplo abaixo, definimos uma entrada X 3×3 com *kernel* K 2×2 , e então usamos `corr2d` para calcular a saída da convolução.

```
X = torch.arange(9.0).reshape(3, 3)
K = torch.tensor([[0, 1], [2, 3]])
```

(continues on next page)

```
Y = d2l.corr2d(X, K)
Y
```

```
tensor([[19., 25.],
        [37., 43.]])
```

A seguir, reescrevemos o *kernel* de convolução K como uma matriz W . Sua forma será $(4, 9)$, onde a linha i^{th} presente aplicando o *kernel* à entrada para gerar o i^{th} elemento de saída.

```
def kernel2matrix(K):
    k, W = torch.zeros(5), torch.zeros((4, 9))
    k[:2], k[3:5] = K[0, :], K[1, :]
    W[0, :5], W[1, 1:6], W[2, 3:8], W[3, 4:] = k, k, k, k
    return W

W = kernel2matrix(K)
W
```

```
tensor([[0., 1., 0., 2., 3., 0., 0., 0., 0.],
        [0., 0., 1., 0., 2., 3., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 2., 3., 0.],
        [0., 0., 0., 0., 0., 1., 0., 2., 3.]])
```

Então, o operador de convolução pode ser implementado por multiplicação de matriz com remodelagem adequada.

```
Y == torch.mv(W, X.reshape(-1)).reshape(2, 2)
```

```
tensor([[True, True],
        [True, True]])
```

Podemos implementar a convolução transposta como uma multiplicação de matriz reutilizando `kernel2matrix`. Para reutilizar o W gerado, construímos uma entrada 2×2 , de modo que a matriz de peso correspondente terá uma forma $(9, 4)$, que é W^T . Deixe-nos verificar os resultados.

```
X = torch.tensor([[0.0, 1], [2, 3]])
Y = trans_conv(X, K)
Y == torch.mv(W.T, X.reshape(-1)).reshape(3, 3)
```

```
tensor([[True, True, True],
        [True, True, True],
        [True, True, True]])
```

13.10.4 Resumo

- Em comparação com as convoluções que reduzem as entradas por meio de *kernels*, as convoluções transpostas transmitem as entradas.
- Se uma camada de convolução reduz a largura e altura de entrada em n_w e h_h tempo, respectivamente. Então, uma camada de convolução transposta com os mesmos tamanhos de *kernel*, preenchimento e passos aumentará a largura e altura de entrada em n_w e h_h , respectivamente.
- Podemos implementar operações de convolução pela multiplicação da matriz, as convoluções transpostas correspondentes podem ser feitas pela multiplicação da matriz transposta.

13.10.5 Exercícios

1. É eficiente usar a multiplicação de matrizes para implementar operações de convolução? Por quê?

Discussões¹⁶⁷

13.11 Redes Totalmente Convolucionais (*Fully Convolutional Networks, FCN*)

Discutimos anteriormente a segmentação semântica usando cada pixel em uma imagem para previsão de categoria. Uma rede totalmente convolucional (FCN) (Long et al., 2015) usa uma rede neural convolucional para transformar os pixels da imagem em categorias de pixels. Ao contrário das redes neurais convolucionais previamente introduzidas, uma FCN transforma a altura e largura do mapa de recurso da camada intermediária de volta ao tamanho da imagem de entrada por meio do camada de convolução transposta, de modo que as previsões tenham uma correspondência com a imagem de entrada em dimensão espacial (altura e largura). Dado uma posição na dimensão espacial, a saída da dimensão do canal será uma previsão de categoria do pixel correspondente ao local.

Primeiro importaremos o pacote ou módulo necessário para o experimento e depois explicaremos a camada de convolução transposta.

```
%matplotlib inline
import torch
import torchvision
from torch import nn
from torch.nn import functional as F
from d2l import torch as d2l
```

¹⁶⁷ <https://discuss.d2l.ai/t/1450>

13.11.1 Construindo um Modelo

Aqui, demonstramos o projeto mais básico de um modelo de rede totalmente convolucional. Conforme mostrado em Fig. 13.11.1, a rede totalmente convolucional primeiro usa a rede neural convolucional para extrair características da imagem, então transforma o número de canais no número de categorias através da camada de convolução 1×1 e, finalmente, transforma a altura e largura do mapa de recursos para o tamanho da imagem de entrada usando a camada de convolução transposta Section 13.10. A saída do modelo tem a mesma altura e largura da imagem de entrada e uma correspondência de um para um nas posições espaciais. O canal de saída final contém a previsão da categoria do pixel da posição espacial correspondente.

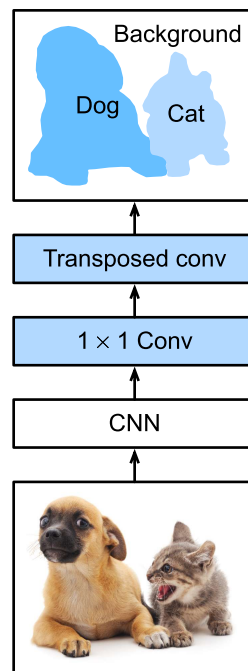


Fig. 13.11.1: Rede totalmente convolucional.

Abaixo, usamos um modelo ResNet-18 pré-treinado no conjunto de dados ImageNet para extrair recursos de imagem e registrar a instância de rede como `pretrained_net`. Como você pode ver, as duas últimas camadas da variável membro do modelo `features` são a camada de agrupamento global médio `GlobalAvgPool2D` e a camada de nivelamento de exemplo `Flatten`. O módulo `output` contém a camada totalmente conectada usada para saída. Essas camadas não são necessárias para uma rede totalmente convolucional.

```
pretrained_net = torchvision.models.resnet18(pretrained=True)
pretrained_net.layer4[1], pretrained_net.avgpool, pretrained_net.fc
```

```
(BasicBlock(
  (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
),
```

(continues on next page)

```
AdaptiveAvgPool2d(output_size=(1, 1)),
Linear(in_features=512, out_features=1000, bias=True))
```

Em seguida, criamos a instância de rede totalmente convolucional net. Ela duplica todas as camadas neurais, exceto as duas últimas camadas da variável membro de instância features de pretrained_net e os parâmetros do modelo obtidos após o pré-treinamento.

```
net = nn.Sequential(*list(pretrained_net.children())[:-2])
```

Dada uma entrada de altura e largura de 320 e 480 respectivamente, o cálculo direto de net reduzirá a altura e largura da entrada para 1/32 do original, ou seja, 10 e 15.

```
X = torch.rand(size=(1, 3, 320, 480))
net(X).shape
```

```
torch.Size([1, 512, 10, 15])
```

Em seguida, transformamos o número de canais de saída para o número de categorias de Pascal VOC2012 (21) por meio da camada de convolução 1×1 . Finalmente, precisamos ampliar a altura e largura do mapa de feições por um fator de 32 para alterá-los de volta para a altura e largura da imagem de entrada. Lembre-se do cálculo método para a forma de saída da camada de convolução descrita em [Section 6.3](#). Porque $(320 - 64 + 16 \times 2 + 32)/32 = 10$ e $(480 - 64 + 16 \times 2 + 32)/32 = 15$, construímos uma camada de convolução transposta com uma distância de 32 e definimos a altura e largura do *kernel* de convolução para 64 e o preenchimento para 16. Não é difícil ver que, se o passo for s , o preenchimento é $s/2$ (assumindo que $s/2$ é um inteiro), e a altura e largura do *kernel* de convolução são $2s$, o *kernel* de convolução transposto aumentará a altura e a largura da entrada por um fator de s .

```
num_classes = 21
net.add_module('final_conv', nn.Conv2d(512, num_classes, kernel_size=1))
net.add_module('transpose_conv', nn.ConvTranspose2d(num_classes, num_classes,
                                                    kernel_size=64, padding=16, stride=32))
```

13.11.2 Inicializando a Camada de Convolução Transposta

Já sabemos que a camada de convolução transposta pode ampliar um mapa de feições. No processamento de imagem, às vezes precisamos ampliar a imagem, ou seja, *upsampling*. Existem muitos métodos para aumentar a amostragem e um método comum é a interpolação bilinear. Simplesmente falando, para obter o pixel da imagem de saída nas coordenadas (x, y) , as coordenadas são primeiro mapeadas para as coordenadas da imagem de entrada (x', y') . Isso pode ser feito com base na proporção do tamanho de três entradas em relação ao tamanho da saída. Os valores mapeados x' e y' são geralmente números reais. Então, encontramos os quatro pixels mais próximos da coordenada (x', y') na imagem de entrada. Finalmente, os pixels da imagem de saída nas coordenadas (x, y) são calculados com base nesses quatro pixels na imagem de entrada e suas distâncias relativas a (x', y') . O *upsampling* por interpolação bilinear pode ser implementado pela camada de convolução transposta do *kernel* de convolução construído usando a seguinte função `bilinear_kernel`. Devido a limitações de espaço, fornecemos apenas a implementação da função `bilinear_kernel` e não discutiremos os princípios do algoritmo.

```
def bilinear_kernel(in_channels, out_channels, kernel_size):
    factor = (kernel_size + 1) // 2
    if kernel_size % 2 == 1:
        center = factor - 1
    else:
        center = factor - 0.5
    og = (torch.arange(kernel_size).reshape(-1, 1),
          torch.arange(kernel_size).reshape(1, -1))
    filt = (1 - torch.abs(og[0] - center) / factor) * \
           (1 - torch.abs(og[1] - center) / factor)
    weight = torch.zeros((in_channels, out_channels,
                          kernel_size, kernel_size))
    weight[range(in_channels), range(out_channels), :, :] = filt
    return weight
```

Agora, vamos experimentar com upsampling de interpolação bilinear implementado por camadas de convolução transpostas. Construa uma camada de convolução transposta que amplie a altura e a largura da entrada por um fator de 2 e inicialize seu kernel de convolução com a função `bilinear_kernel`.

```
conv_trans = nn.ConvTranspose2d(3, 3, kernel_size=4, padding=1, stride=2,
                                bias=False)
conv_trans.weight.data.copy_(bilinear_kernel(3, 3, 4));
```

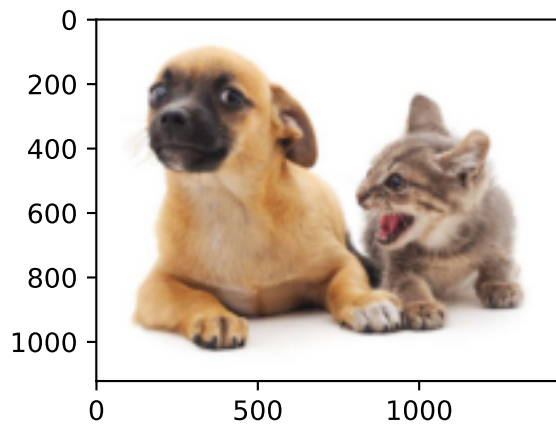
Leia a imagem `X` e registre o resultado do upsampling como `Y`. Para imprimir a imagem, precisamos ajustar a posição da dimensão do canal.

```
img = torchvision.transforms.ToTensor()(d2l.Image.open('../img/catdog.jpg'))
X = img.unsqueeze(0)
Y = conv_trans(X)
out_img = Y[0].permute(1, 2, 0).detach()
```

Como você pode ver, a camada de convolução transposta amplia a altura e largura da imagem em um fator de 2. Vale ressaltar que, além da diferença na escala de coordenadas, a imagem ampliada por interpolação bilinear e a imagem original impressa em [Section 13.3](#) tem a mesma aparência.

```
d2l.set_figsize()
print('input image shape:', img.permute(1, 2, 0).shape)
d2l.plt.imshow(img.permute(1, 2, 0));
print('output image shape:', out_img.shape)
d2l.plt.imshow(out_img);
```

```
input image shape: torch.Size([561, 728, 3])
output image shape: torch.Size([1122, 1456, 3])
```



Em uma rede totalmente convolucional, inicializamos a camada de convolução transposta para interpolação bilinear com `upsampled`. Para uma camada de convolução 1×1 , usamos o Xavier para inicialização aleatória.

```
W = bilinear_kernel(num_classes, num_classes, 64)
net.transpose_conv.weight.data.copy_(W);
```

13.11.3 Lendo o Dataset

Lemos o *dataset* usando o método descrito na seção anterior. Aqui, especificamos a forma da imagem de saída cortada aleatoriamente como 320×480 , portanto, a altura e a largura são divisíveis por 32.

```
batch_size, crop_size = 32, (320, 480)
train_iter, test_iter = d2l.load_data_voc(batch_size, crop_size)
```

```
read 1114 examples
read 1078 examples
```

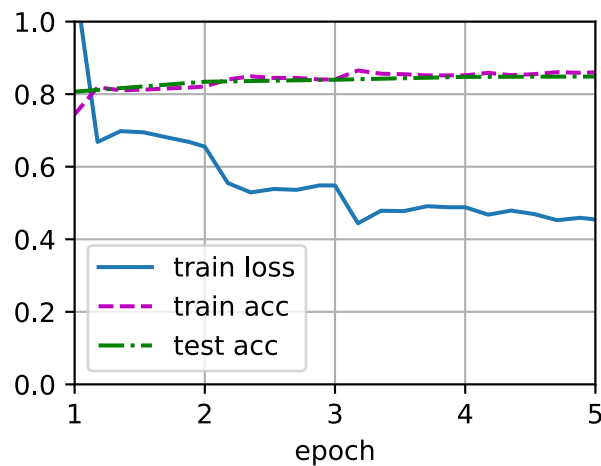
13.11.4 Treinamento

Agora podemos começar a treinar o modelo. A função de perda e o cálculo de precisão aqui não são substancialmente diferentes daqueles usados na classificação de imagens. Como usamos o canal da camada de convolução transposta para prever as categorias de pixels, a opção `axis = 1` (dimensão do canal) é especificada em `SoftmaxCrossEntropyLoss`. Além disso, o modelo calcula a precisão com base em se a categoria de previsão de cada pixel está correta.

```
def loss(inputs, targets):
    return F.cross_entropy(inputs, targets, reduction='none').mean(1).mean(1)

num_epochs, lr, wd, devices = 5, 0.001, 1e-3, d2l.try_all_gpus()
trainer = torch.optim.SGD(net.parameters(), lr=lr, weight_decay=wd)
d2l.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs, devices)
```

```
loss 0.455, train acc 0.860, test acc 0.849
222.9 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



13.11.5 Predição

Durante a previsão, precisamos padronizar a imagem de entrada em cada canal e transformá-los no formato de entrada quadridimensional exigido pela rede neural convolucional.

```
def predict(img):
    X = test_iter.dataset.normalize_image(img).unsqueeze(0)
    pred = net(X.to(devices[0])).argmax(dim=1)
    return pred.reshape(pred.shape[1], pred.shape[2])
```

Para visualizar as categorias previstas para cada pixel, mapeamos as categorias previstas de volta às suas cores rotuladas no conjunto de dados.

```
def label2image(pred):
    colormap = torch.tensor(d2l.VOC_COLORMAP, device=devices[0])
    X = pred.long()
    return colormap[X, :]
```

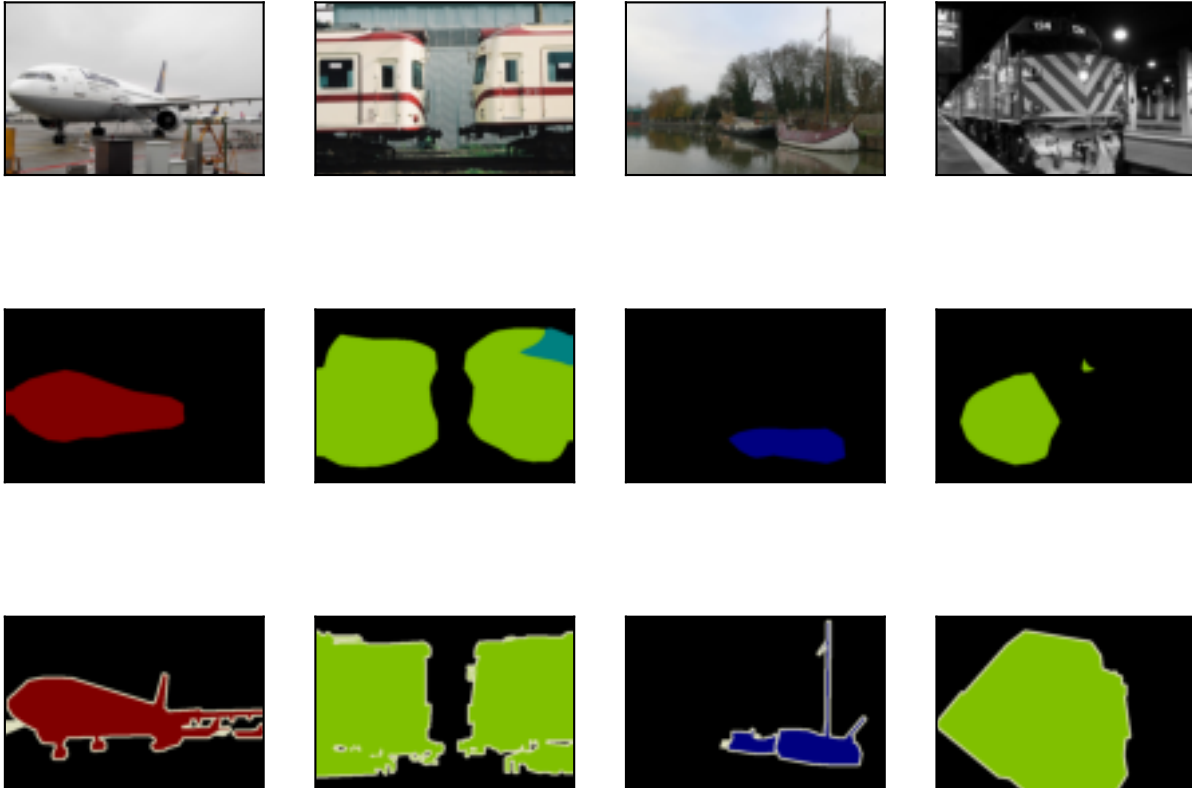
O tamanho e a forma das imagens no conjunto de dados de teste variam. Como o modelo usa uma camada de convolução transposta com uma distância de 32, quando a altura ou largura da imagem de entrada não é divisível por 32, a altura ou largura da saída da camada de convolução transposta se desvia do tamanho da imagem de entrada. Para resolver esse problema, podemos recortar várias áreas retangulares na imagem com alturas e larguras como múltiplos inteiros de 32 e, em seguida, realizar cálculos para a frente nos pixels nessas áreas. Quando combinadas, essas áreas devem cobrir completamente a imagem de entrada. Quando um pixel é coberto por várias áreas, a média da saída da camada de convolução transposta no cálculo direto das diferentes áreas pode ser usada como uma entrada para a operação softmax para prever a categoria.

Para simplificar, lemos apenas algumas imagens de teste grandes e recortamos uma área com um formato de 320×480 no canto superior esquerdo da imagem. Apenas esta área é usada para previsão. Para a imagem de entrada, imprimimos primeiro a área cortada, depois imprimimos o resultado previsto e, por fim, imprimimos a categoria rotulada.

```

voc_dir = d2l.download_extract('voc2012', 'VOCdevkit/VOC2012')
test_images, test_labels = d2l.read_voc_images(voc_dir, False)
n, imgs = 4, []
for i in range(n):
    crop_rect = (0, 0, 320, 480)
    X = torchvision.transforms.functional.crop(test_images[i], *crop_rect)
    pred = label2image(predict(X))
    imgs += [X.permute(1,2,0), pred.cpu(),
             torchvision.transforms.functional.crop(
                 test_labels[i], *crop_rect).permute(1,2,0)]
d2l.show_images(imgs[:3] + imgs[1::3] + imgs[2::3], 3, n, scale=2);

```



13.11.6 Resumo

- A rede totalmente convolucional primeiro usa a rede neural convolucional para extrair características da imagem, depois transforma o número de canais no número de categorias por meio da camada de convolução 1×1 e, finalmente, transforma a altura e largura do mapa de características para o tamanho da imagem de entrada usando a camada de convolução transposta para produzir a categoria de cada pixel.
- Em uma rede totalmente convolucional, inicializamos a camada de convolução transposta para interpolação bilinear com *upsampling*.

13.11.7 Exercícios

1. Se usarmos Xavier para inicializar aleatoriamente a camada de convolução transposta, o que acontecerá com o resultado?
2. Você pode melhorar ainda mais a precisão do modelo ajustando os hiperparâmetros?
3. Preveja as categorias de todos os pixels na imagem de teste.
4. As saídas de algumas camadas intermediárias da rede neural convolucional também são usadas no artigo sobre redes totalmente convolucionais (Long et al., 2015). Tente implementar essa ideia.

Discussões¹⁶⁸

13.12 Transferência de Estilo Neural

Se você usa aplicativos de compartilhamento social ou é um fotógrafo amador, está familiarizado com os filtros. Os filtros podem alterar os estilos de cor das fotos para tornar o fundo mais nítido ou o rosto das pessoas mais branco. No entanto, um filtro geralmente só pode alterar um aspecto de uma foto. Para criar a foto ideal, muitas vezes você precisa tentar muitas combinações de filtros diferentes. Esse processo é tão complexo quanto ajustar os hiperparâmetros de um modelo.

Nesta seção, discutiremos como podemos usar redes neurais de convolução (CNNs) para aplicar automaticamente o estilo de uma imagem a outra imagem, uma operação conhecida como transferência de estilo (Gatys et al., 2016). Aqui, precisamos de duas imagens de entrada, uma imagem de conteúdo e uma imagem de estilo. Usamos uma rede neural para alterar a imagem do conteúdo de modo que seu estilo espelhe o da imagem do estilo. Em Fig. 13.12.1, a imagem do conteúdo é uma foto de paisagem que o autor tirou na Mount Rainier National Park perto de Seattle. A imagem do estilo é uma pintura a óleo de carvalhos no outono. A imagem composta de saída retém as formas gerais dos objetos na imagem de conteúdo, mas aplica a pincelada de pintura a óleo da imagem de estilo e torna a cor geral mais vívida.

¹⁶⁸ <https://discuss.d2l.ai/t/1582>

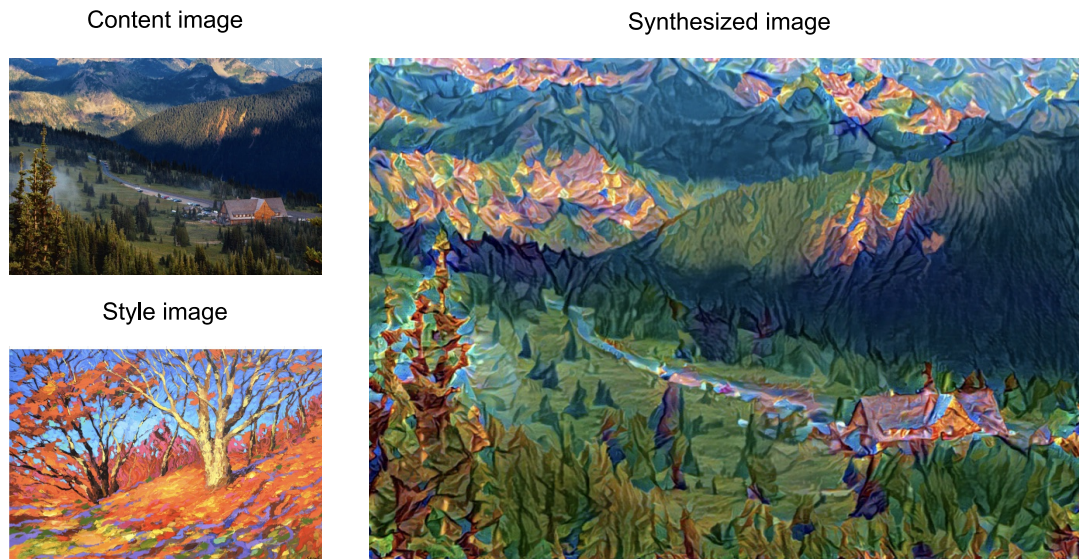


Fig. 13.12.1: Imagens de entrada de conteúdo e estilo e imagem composta produzida por transferência de estilo.

13.12.1 Técnica

O modelo de transferência de estilo baseado em CNN é mostrado em Fig. 13.12.2. Primeiro, inicializamos a imagem composta. Por exemplo, podemos inicializá-la como a imagem do conteúdo. Esta imagem composta é a única variável que precisa ser atualizada no processo de transferência de estilo, ou seja, o parâmetro do modelo a ser atualizado na transferência de estilo. Em seguida, selecionamos uma CNN pré-treinada para extrair recursos de imagem. Esses parâmetros do modelo não precisam ser atualizados durante o treinamento. O CNN profundo usa várias camadas neurais que extraem sucessivamente recursos de imagem. Podemos selecionar a saída de certas camadas para usar como recursos de conteúdo ou recursos de estilo. Se usarmos a estrutura em Fig. 13.12.2, a rede neural pré-treinada contém três camadas convolucionais. A segunda camada produz os recursos de conteúdo da imagem, enquanto as saídas da primeira e terceira camadas são usadas como recursos de estilo. Em seguida, usamos a propagação para a frente (na direção das linhas sólidas) para calcular a função de perda de transferência de estilo e a propagação para trás (na direção das linhas pontilhadas) para atualizar o parâmetro do modelo, atualizando constantemente a imagem composta. As funções de perda usadas na transferência de estilo geralmente têm três partes: 1. A perda de conteúdo é usada para fazer a imagem composta se aproximar da imagem de conteúdo no que diz respeito aos recursos de conteúdo. 2. A perda de estilo é usada para fazer com que a imagem composta se aproxime da imagem de estilo em termos de recursos de estilo. 3. A perda total de variação ajuda a reduzir o ruído na imagem composta. Finalmente, depois de terminar de treinar o modelo, produzimos os parâmetros do modelo de transferência de estilo para obter a imagem composta final.

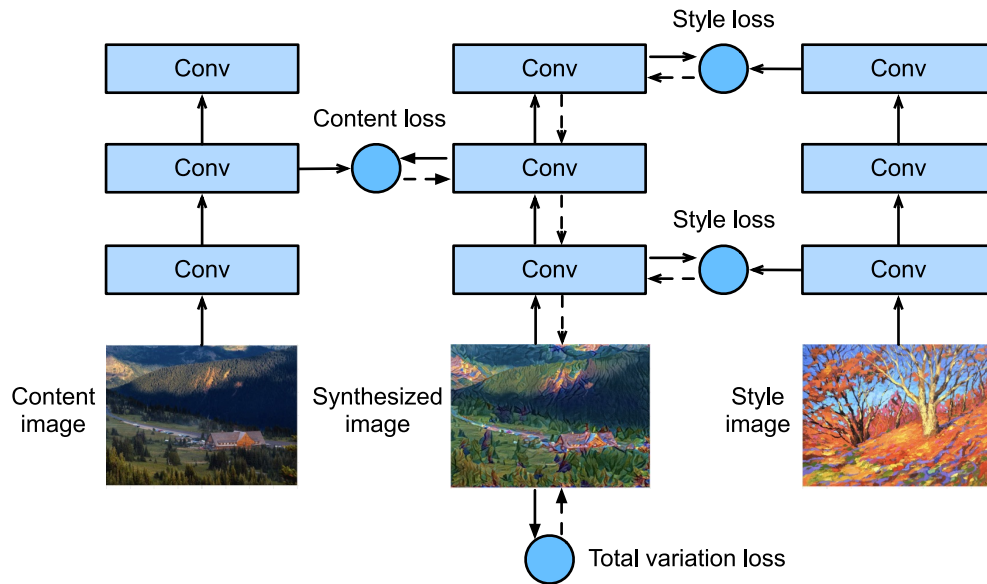


Fig. 13.12.2: Processo de transferência de estilo baseado em CNN. As linhas sólidas mostram a direção da propagação para a frente e as linhas pontilhadas mostram a propagação para trás.

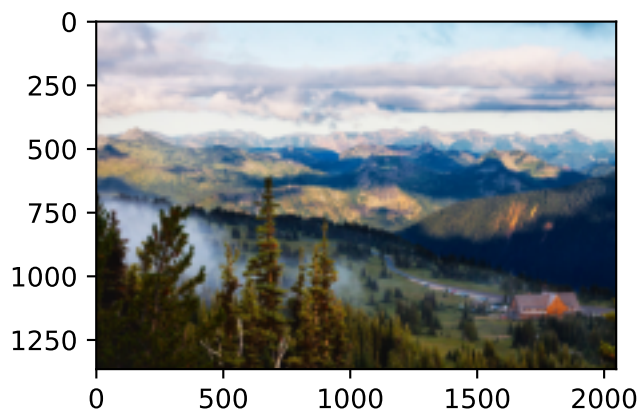
A seguir, faremos um experimento para nos ajudar a entender melhor os detalhes técnicos da transferência de estilo.

13.12.2 Lendo o Conteúdo e as Imagens de Estilo

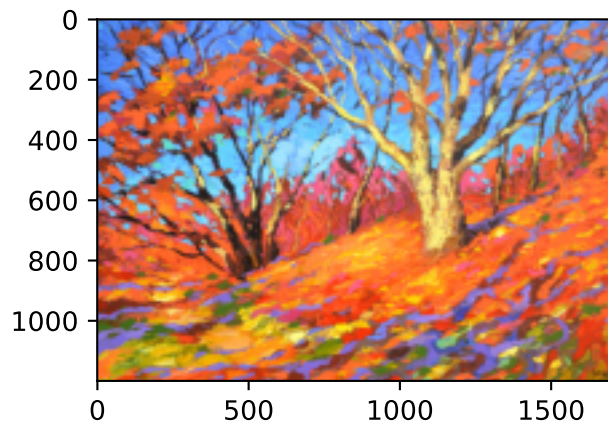
Primeiro, lemos o conteúdo e as imagens de estilo. Ao imprimir os eixos de coordenadas da imagem, podemos ver que eles têm dimensões diferentes.

```
%matplotlib inline
import torch
import torchvision
from torch import nn
from d2l import torch as d2l

d2l.set_figsize()
content_img = d2l.Image.open('../img/rainier.jpg')
d2l.plt.imshow(content_img);
```



```
style_img = d2l.Image.open('../img/autumn-oak.jpg')
d2l.plt.imshow(style_img);
```



13.12.3 Pré-processamento e Pós-processamento

A seguir, definimos as funções de pré-processamento e pós-processamento de imagens. A função pré-processamento normaliza cada um dos três canais RGB das imagens de entrada e transforma os resultados em um formato que pode ser inserido na CNN. A função postprocess restaura os valores de pixel na imagem de saída para seus valores originais antes da normalização. Como a função de impressão de imagem requer que cada pixel tenha um valor de ponto flutuante de 0 a 1, usamos a função clip para substituir valores menores que 0 ou maiores que 1 por 0 ou 1, respectivamente.

```
rgb_mean = torch.tensor([0.485, 0.456, 0.406])
rgb_std = torch.tensor([0.229, 0.224, 0.225])

def preprocess(img, image_shape):
    transforms = torchvision.transforms.Compose([
        torchvision.transforms.Resize(image_shape),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(mean=rgb_mean, std=rgb_std)])
    return transforms(img).unsqueeze(0)

def postprocess(img):
    img = img[0].to(rgb_std.device)
    img = torch.clamp(img.permute(1, 2, 0) * rgb_std + rgb_mean, 0, 1)
    return torchvision.transforms.ToPILImage()(img.permute(2, 0, 1))
```

13.12.4 Extrair Features

Usamos o modelo VGG-19 pré-treinado no conjunto de dados ImageNet para extrair características da imagem [1].

```
pretrained_net = torchvision.models.vgg19(pretrained=True)
```

Para extrair o conteúdo da imagem e os recursos de estilo, podemos selecionar as saídas de certas camadas na rede VGG. Em geral, quanto mais próxima uma saída estiver da camada de entrada, mais fácil será extrair informações detalhadas da imagem. Quanto mais longe uma saída estiver, mais fácil será extrair informações globais. Para evitar que a imagem composta retenha muitos detalhes da imagem de conteúdo, selecionamos uma camada de rede VGG próxima à camada de saída para produzir os recursos de conteúdo da imagem. Essa camada é chamada de camada de conteúdo. Também selecionamos as saídas de diferentes camadas da rede VGG para combinar os estilos local e global. Elas são chamadas de camadas de estilo. Como mencionamos em [Section 7.2](#), as redes VGG têm cinco blocos convolucionais. Neste experimento, selecionamos a última camada convolucional do quarto bloco convolucional como a camada de conteúdo e a primeira camada de cada bloco como camadas de estilo. Podemos obter os índices para essas camadas imprimindo a instância `pretrained_net`.

```
style_layers, content_layers = [0, 5, 10, 19, 28], [25]
```

Durante a extração de *features*, só precisamos usar todas as camadas VGG da camada de entrada até a camada de conteúdo ou estilo mais próxima da camada de saída. Abaixo, construímos uma nova rede, `net`, que retém apenas as camadas da rede VGG que precisamos usar. Em seguida, usamos `net` para extrair *features*.

```
net = nn.Sequential(*[pretrained_net.features[i] for i in
                      range(max(content_layers + style_layers) + 1)])
```

Dada a entrada X , se simplesmente chamarmos a computação direta de `net(X)`, só podemos obter a saída da última camada. Como também precisamos das saídas das camadas intermediárias, precisamos realizar computação camada por camada e reter o conteúdo e as saídas da camada de estilo.

```
def extract_features(X, content_layers, style_layers):
    contents = []
    styles = []
    for i in range(len(net)):
        X = net[i](X)
        if i in style_layers:
            styles.append(X)
        if i in content_layers:
            contents.append(X)
    return contents, styles
```

A seguir, definimos duas funções: a função `get_contents` obtém as *features* de conteúdo extraídos da imagem do conteúdo, enquanto a função `get_styles` obtém os recursos de estilo extraídos da imagem de estilo. Como não precisamos alterar os parâmetros do modelo VGG pré-treinado durante o treinamento, podemos extrair as *features* de conteúdo da imagem de conteúdo e recursos de estilo da imagem de estilo antes do início do treinamento. Como a imagem composta é o parâmetro do modelo que deve ser atualizado durante a transferência do estilo, só podemos

chamar a função `extract_features` durante o treinamento para extrair o conteúdo e os recursos de estilo da imagem composta.

```
def get_contents(image_shape, device):
    content_X = preprocess(content_img, image_shape).to(device)
    contents_Y, _ = extract_features(content_X, content_layers, style_layers)
    return content_X, contents_Y

def get_styles(image_shape, device):
    style_X = preprocess(style_img, image_shape).to(device)
    _, styles_Y = extract_features(style_X, content_layers, style_layers)
    return style_X, styles_Y
```

13.12.5 Definindo a Função de Perda

A seguir, veremos a função de perda usada para transferência de estilo. A função de perda inclui a perda de conteúdo, perda de estilo e perda total de variação.

Perda de Conteúdo

Semelhante à função de perda usada na regressão linear, a perda de conteúdo usa uma função de erro quadrado para medir a diferença nos recursos de conteúdo entre a imagem composta e a imagem de conteúdo. As duas entradas da função de erro quadrada são ambas saídas da camada de conteúdo obtidas da função `extract_features`.

```
def content_loss(Y_hat, Y):
    # we 'detach' the target content from the tree used
    # to dynamically compute the gradient: this is a stated value,
    # not a variable. Otherwise the loss will throw an error.
    return torch.square(Y_hat - Y.detach()).mean()
```

Perda de Estilo

A perda de estilo, semelhante à perda de conteúdo, usa uma função de erro quadrático para medir a diferença de estilo entre a imagem composta e a imagem de estilo. Para expressar a saída de estilos pelas camadas de estilo, primeiro usamos a função `extract_features` para calcular a saída da camada de estilo. Supondo que a saída tenha 1 exemplo, c canais, e uma altura e largura de h e w , podemos transformar a saída na matriz $\mathbf{X}$, que tem c linhas e $h \cdot w$ colunas. Você pode pensar na matriz $\mathbf{X}$ como a combinação dos vetores c e $\mathbf{x}_1, \dots, \mathbf{x}_c$, que têm um comprimento de hw . Aqui, o vetor $\mathbf{x}_i$ representa a característica de estilo do canal i . Na matriz Gram desses vetores $\mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{c \times c}$, elemento x_{ij} na linha i coluna j é o produto interno dos vetores $\mathbf{x}_i$ e $\mathbf{x}_j$. Ele representa a correlação dos recursos de estilo dos canais i e j . Usamos esse tipo de matriz de Gram para representar a saída do estilo pelas camadas de estilo. Você deve notar que, quando o valor $h \cdot w$ é grande, isso geralmente leva a valores grandes na matriz de Gram. Além disso, a altura e a largura da matriz de Gram são o número de canais c . Para garantir que a perda de estilo não seja afetada pelo tamanho desses valores, definimos a função `gram` abaixo para dividir a matriz de Gram pelo número de seus elementos, ou seja, $c \cdot h \cdot w$.

```
def gram(X):
    num_channels, n = X.shape[1], X.numel() // X.shape[1]
    X = X.reshape((num_channels, n))
    return torch.matmul(X, X.T) / (num_channels * n)
```

Naturalmente, as duas entradas de matriz de Gram da função de erro quadrado para perda de estilo são obtidas da imagem composta e das saídas da camada de estilo de imagem de estilo. Aqui, assumimos que a matriz de Gram da imagem de estilo, `gram_Y`, foi calculada antecipadamente.

```
def style_loss(Y_hat, gram_Y):
    return torch.square(gram(Y_hat) - gram_Y.detach()).mean()
```

Perda de Variância Total

Às vezes, as imagens compostas que aprendemos têm muito ruído de alta frequência, principalmente pixels claros ou escuros. Um método comum de redução de ruído é a redução total de ruído na variação. Assumimos que $x_{i,j}$ representa o valor do pixel na coordenada (i, j) , então a perda total de variância é:

$$\sum_{i,j} |x_{i,j} - x_{i+1,j}| + |x_{i,j} - x_{i,j+1}|. \quad (13.12.1)$$

Tentamos tornar os valores dos pixels vizinhos tão semelhantes quanto possível.

```
def tv_loss(Y_hat):
    return 0.5 * (torch.abs(Y_hat[:, :, 1:, :] - Y_hat[:, :, :-1, :]).mean() +
                  torch.abs(Y_hat[:, :, :, 1:] - Y_hat[:, :, :, :-1]).mean())
```

Função de Perda

A função de perda para transferência de estilo é a soma ponderada da perda de conteúdo, perda de estilo e perda total de variância. Ajustando esses hiperparâmetros de peso, podemos equilibrar o conteúdo retido, o estilo transferido e a redução de ruído na imagem composta de acordo com sua importância relativa.

```
content_weight, style_weight, tv_weight = 1, 1e3, 10

def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # Calculate the content, style, and total variance losses respectively
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # Add up all the losses
    l = sum(styles_l + contents_l + [tv_l])
    return contents_l, styles_l, tv_l, l
```

13.12.6 Criação e inicialização da imagem composta

Na transferência de estilo, a imagem composta é a única variável que precisa ser atualizada. Portanto, podemos definir um modelo simples, `GeneratedImage`, e tratar a imagem composta como um parâmetro do modelo. No modelo, a computação direta retorna apenas o parâmetro do modelo.

```
class GeneratedImage(nn.Module):
    def __init__(self, img_shape, **kwargs):
        super(GeneratedImage, self).__init__(**kwargs)
        self.weight = nn.Parameter(torch.rand(*img_shape))

    def forward(self):
        return self.weight
```

A seguir, definimos a função `get_inits`. Esta função cria uma instância de modelo de imagem composta e a inicializa na imagem `X`. A matriz de Gram para as várias camadas de estilo da imagem de estilo, `styles_Y_gram`, é calculada antes do treinamento.

```
def get_inits(X, device, lr, styles_Y):
    gen_img = GeneratedImage(X.shape).to(device)
    gen_img.weight.data.copy_(X.data)
    trainer = torch.optim.Adam(gen_img.parameters(), lr=lr)
    styles_Y_gram = [gram(Y) for Y in styles_Y]
    return gen_img(), styles_Y_gram, trainer
```

13.12.7 Treinamento

Durante o treinamento do modelo, extraímos constantemente o conteúdo e as *features* de estilo da imagem composta e calculamos a função de perda. Lembre-se de nossa discussão sobre como as funções de sincronização forçam o *front-end* a esperar pelos resultados de computação em [Section 12.2](#). Como apenas chamamos a função de sincronização `asnumpy` a cada 10 épocas, o processo pode ocupar uma grande quantidade de memória. Portanto, chamamos a função de sincronização `waitall` durante cada época.

```
def train(X, contents_Y, styles_Y, device, lr, num_epochs, lr_decay_epoch):
    X, styles_Y_gram, trainer = get_inits(X, device, lr, styles_Y)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_decay_epoch)
    animator = d2l.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[10, num_epochs],
                           legend=['content', 'style', 'TV'],
                           ncols=2, figsize=(7, 2.5))
    for epoch in range(num_epochs):
        trainer.zero_grad()
        contents_Y_hat, styles_Y_hat = extract_features(
            X, content_layers, style_layers)
        contents_l, styles_l, tv_l, l = compute_loss(
            X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram)
        l.backward()
        trainer.step()
        scheduler.step()
        if (epoch + 1) % 10 == 0:
```

(continues on next page)

```

    animator.axes[1].imshow(postprocess(X))
    animator.add(epoch + 1, [float(sum(contents_l)),
                             float(sum(styles_l)), float(tv_l)])

return X

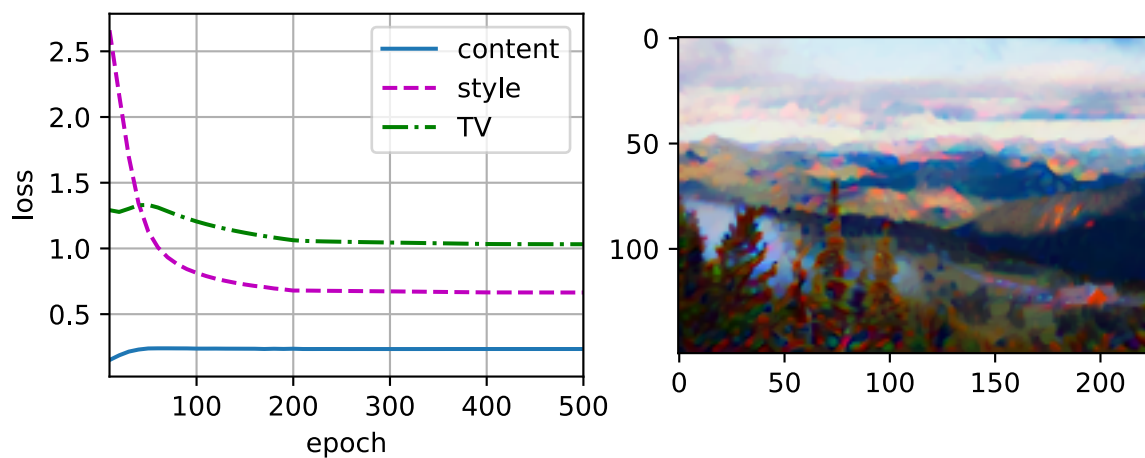
```

Em seguida, começamos a treinar o modelo. Primeiro, definimos a altura e a largura das imagens de conteúdo e estilo para 150 por 225 pixels. Usamos a imagem de conteúdo para inicializar a imagem composta.

```

device, image_shape = d2l.try_gpu(), (150, 225) # PIL Image (h, w)
net = net.to(device)
content_X, contents_Y = get_contents(image_shape, device)
_, styles_Y = get_styles(image_shape, device)
output = train(content_X, contents_Y, styles_Y, device, 0.01, 500, 200)

```



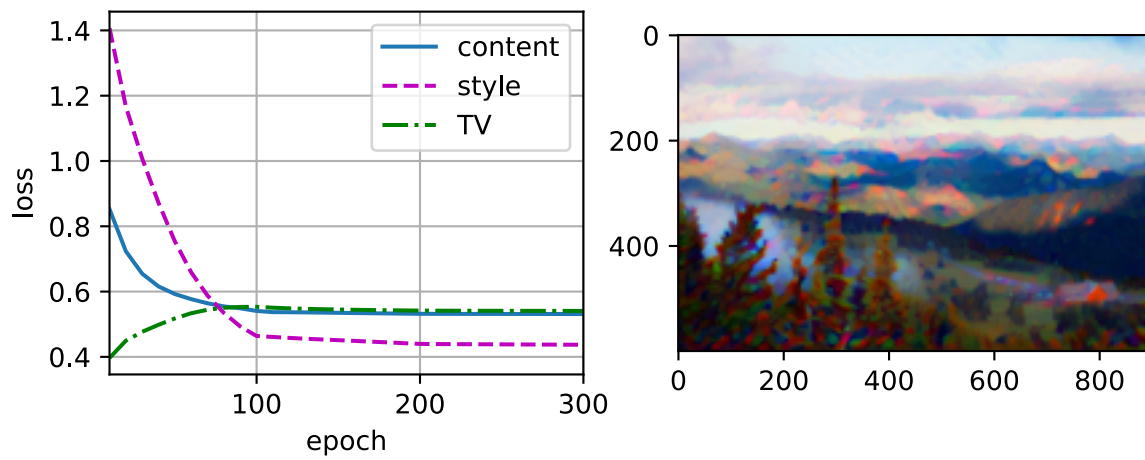
Como você pode ver, a imagem composta retém o cenário e os objetos da imagem de conteúdo, enquanto introduz a cor da imagem de estilo. Como a imagem é relativamente pequena, os detalhes são um pouco confusos.

Para obter uma imagem composta mais clara, treinamos o modelo usando um tamanho de imagem maior: 900×600 . Aumentamos a altura e a largura da imagem usada antes por um fator de quatro e inicializamos uma imagem composta maior.

```

image_shape = (600, 900) # PIL Image (h, w)
_, content_Y = get_contents(image_shape, device)
_, style_Y = get_styles(image_shape, device)
X = preprocess(postprocess(output), image_shape).to(device)
output = train(X, content_Y, style_Y, device, 0.01, 300, 100)
d2l.plt.imshow('..img/neural-style.jpg', postprocess(output))

```

Como você pode ver, cada época leva mais tempo devido ao tamanho maior da imagem. Conforme mostrado em Fig. 13.12.3, a imagem composta produzida retém mais detalhes devido ao seu tamanho maior. A imagem composta não só tem grandes blocos de cores como a imagem de estilo, mas esses blocos têm até a textura sutil de pinceladas.

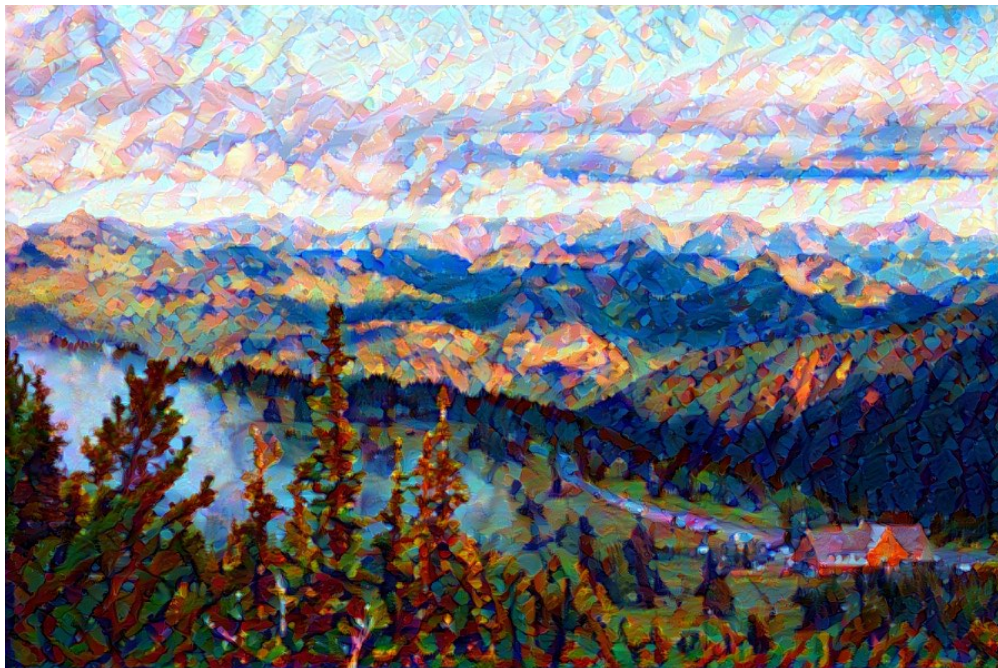


Fig. 13.12.3: Imagem 900×600 composta.

13.12.8 Resumo

- As funções de perda usadas na transferência de estilo geralmente têm três partes: 1. A perda de conteúdo é usada para fazer a imagem composta se aproximar da imagem de conteúdo no que diz respeito aos recursos de conteúdo. 2. A perda de estilo é usada para fazer com que a imagem composta se aproxime da imagem de estilo em termos de recursos de estilo. 3. A perda total de variação ajuda a reduzir o ruído na imagem composta.
- Podemos usar um CNN pré-treinado para extrair recursos de imagem e minimizar a função de perda para atualizar continuamente a imagem composta.
- Usamos uma matriz de Gram para representar a saída do estilo pelas camadas de estilo.

13.12.9 Exercícios

1. Como a saída muda quando você seleciona diferentes camadas de conteúdo e estilo?
2. Ajuste os hiperparâmetros de peso na função de perda. A saída retém mais conteúdo ou tem menos ruído?
3. Use imagens de conteúdo e estilo diferentes. Você pode criar imagens compostas mais interessantes?
4. Podemos aplicar transferência de estilo para texto? Dica: você pode consultar o documento de pesquisa de Hu et al. (Hu et al., 2020).

Discussões¹⁶⁹

13.13 Classificação de Imagens (CIFAR-10) no Kaggle

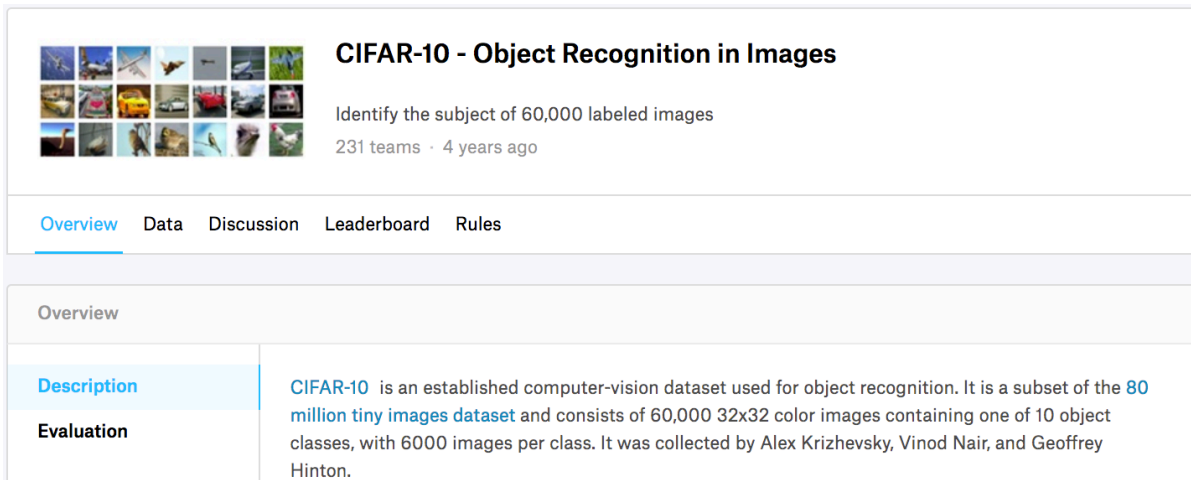
Até agora, temos usado o pacote data do Gluon para obter diretamente conjuntos de dados de imagem no formato tensor. Na prática, entretanto, os conjuntos de dados de imagem geralmente existem no formato de arquivos de imagem. Nesta seção, começaremos com os arquivos de imagem originais e organizaremos, leremos e converteremos os arquivos para o formato tensor passo a passo.

Realizamos um experimento no conjunto de dados CIFAR-10 em [Section 13.1](#). Este é um dado importante definido no campo de visão do computador. Agora, vamos aplicar o conhecimento que aprendemos em as seções anteriores para participar da competição Kaggle, que aborda problemas de classificação de imagens CIFAR-10. O endereço da competição na web é

<https://www.kaggle.com/c/cifar-10>

[Fig. 13.13.1](#) mostra as informações na página da competição. Para enviar os resultados, primeiro registre uma conta no site do Kaggle.

¹⁶⁹ <https://discuss.d2l.ai/t/1476>



CIFAR-10 - Object Recognition in Images

Identify the subject of 60,000 labeled images
231 teams · 4 years ago

[Overview](#) [Data](#) [Discussion](#) [Leaderboard](#) [Rules](#)

Overview

Description

Evaluation

CIFAR-10 is an established computer-vision dataset used for object recognition. It is a subset of the [80 million tiny images dataset](#) and consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.

Fig. 13.13.1: Informações da página da web do concurso de classificação de imagens CIFAR-10. O conjunto de dados da competição pode ser acessado clicando na guia “Dados”.

Primeiro, importe os pacotes ou módulos necessários para a competição.

```
import collections
import math
import os
import shutil
import pandas as pd
import torch
import torchvision
from torch import nn
from d2l import torch as d2l
```

13.13.1 Obtendo e Organizando o Dataset

Os dados da competição são divididos em um conjunto de treinamento e um conjunto de teste. O conjunto de treinamento contém 50.000 imagens. O conjunto de teste contém 300.000 imagens, das quais 10.000 imagens são usadas para pontuação, enquanto as outras 290.000 imagens sem pontuação são incluídas para evitar a rotulagem manual do conjunto de teste e o envio dos resultados da rotulagem. Os formatos de imagem em ambos os conjuntos de dados são PNG, com alturas e larguras de 32 pixels e três canais de cores (RGB). As imagens cobrem categorias de 10: aviões, carros, pássaros, gatos, veados, cães, sapos, cavalos, barcos e caminhões. O canto superior esquerdo de Fig. 13.13.1 mostra algumas imagens de aviões, carros e pássaros no conjunto de dados.

Baixando o Dataset

Após fazer o login no Kaggle, podemos clicar na guia “Dados” na página da competição de classificação de imagens CIFAR-10 mostrada em Fig. 13.13.1 e baixar o conjunto de dados clicando no botão “Download All”. Após descompactar o arquivo baixado em `../data` e descompactar `train.7z` e `test.7z` dentro dele, você encontrará o conjunto de dados inteiro nos seguintes caminhos:

- `../data/cifar-10/train/[1-50000].png`
- `../data/cifar-10/test/[1-300000].png`
- `../data/cifar-10/trainLabels.csv`
- `../data/cifar-10/sampleSubmission.csv`

Aqui, as pastas `train` e `test` contêm as imagens de treinamento e teste, respectivamente, `trainLabels.csv` tem rótulos para as imagens de treinamento e `sample_submission.csv` é um exemplo de envio.

Para facilitar o início, fornecemos uma amostra em pequena escala do conjunto de dados: ele contém as primeiras 1000 de imagens de treinamento e 5 de imagens de teste aleatórias. Para usar o conjunto de dados completo da competição Kaggle, você precisa definir a seguinte variável `demo` como `False`.

```
#@save
d2l.DATA_HUB['cifar10_tiny'] = (d2l.DATA_URL + 'kaggle_cifar10_tiny.zip',
                               '2068874e4b9a9f0fb07ebe0ad2b29754449ccacd')

# If you use the full dataset downloaded for the Kaggle competition, set
# `demo` to False
demo = True

if demo:
    data_dir = d2l.download_extract('cifar10_tiny')
else:
    data_dir = '../data/cifar-10/'
```

```
Downloading ../data/kaggle_cifar10_tiny.zip from http://d2l-data.s3-accelerate.amazonaws.com/
↪kaggle_cifar10_tiny.zip...
```

Organizando o Dataset

Precisamos organizar conjuntos de dados para facilitar o treinamento e teste do modelo. Vamos primeiro ler os rótulos do arquivo `csv`. A função a seguir retorna um dicionário que mapeia o nome do arquivo sem extensão para seu rótulo.

```
#@save
def read_csv_labels(fname):
    """Read fname to return a name to label dictionary."""
    with open(fname, 'r') as f:
        # Skip the file header line (column name)
        lines = f.readlines()[1:]
        tokens = [l.rstrip().split(',') for l in lines]
        return dict(((name, label) for name, label in tokens))
```

(continues on next page)

```
labels = read_csv_labels(os.path.join(data_dir, 'trainLabels.csv'))
print('# training examples:', len(labels))
print('# classes:', len(set(labels.values())))
```

```
# training examples: 1000
# classes: 10
```

Em seguida, definimos a função `reorg_train_valid` para segmentar o conjunto de validação do conjunto de treinamento original. O argumento `valid_ratio` nesta função é a razão entre o número de exemplos no conjunto de validação e o número de exemplos no conjunto de treinamento original. Em particular, seja n o número de imagens da classe com menos exemplos e r a proporção, então usaremos $\max(\lfloor nr \rfloor, 1)$ imagens para cada classe como o conjunto de validação. Vamos usar `valid_ratio = 0.1` como exemplo. Como o conjunto de treinamento original tem 50.000 imagens, haverá 45.000 imagens usadas para treinamento e armazenadas no caminho “train_valid_test/train” ao ajustar hiperparâmetros, enquanto as outras 5.000 imagens serão armazenadas como conjunto de validação no caminho “train_valid_test / valid”. Depois de organizar os dados, as imagens da mesma turma serão colocadas na mesma pasta para que possamos lê-las posteriormente.

```
##@save
def copyfile(filename, target_dir):
    """Copy a file into a target directory."""
    os.makedirs(target_dir, exist_ok=True)
    shutil.copy(filename, target_dir)

##@save
def reorg_train_valid(data_dir, labels, valid_ratio):
    # The number of examples of the class with the least examples in the
    # training dataset
    n = collections.Counter(labels.values()).most_common()[-1][1]
    # The number of examples per class for the validation set
    n_valid_per_label = max(1, math.floor(n * valid_ratio))
    label_count = {}
    for train_file in os.listdir(os.path.join(data_dir, 'train')):
        label = labels[train_file.split('.')[0]]
        fname = os.path.join(data_dir, 'train', train_file)
        # Copy to train_valid_test/train_valid with a subfolder per class
        copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                     'train_valid', label))
        if label not in label_count or label_count[label] < n_valid_per_label:
            # Copy to train_valid_test/valid
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'valid', label))
            label_count[label] = label_count.get(label, 0) + 1
        else:
            # Copy to train_valid_test/train
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'train', label))
    return n_valid_per_label
```

A função `reorg_test` abaixo é usada para organizar o conjunto de testes para facilitar a leitura durante a previsão.

```
#@save
def reorg_test(data_dir):
    for test_file in os.listdir(os.path.join(data_dir, 'test')):
        copyfile(os.path.join(data_dir, 'test', test_file),
                  os.path.join(data_dir, 'train_valid_test', 'test',
                               'unknown'))
```

Finalmente, usamos uma função para chamar as funções `read_csv_labels`, `reorg_train_valid` e `reorg_test` previamente definidas.

```
def reorg_cifar10_data(data_dir, valid_ratio):
    labels = read_csv_labels(os.path.join(data_dir, 'trainLabels.csv'))
    reorg_train_valid(data_dir, labels, valid_ratio)
    reorg_test(data_dir)
```

Definimos apenas o tamanho do lote em 4 para o conjunto de dados de demonstração. Durante o treinamento e os testes reais, o conjunto de dados completo da competição Kaggle deve ser usado e `batch_size` deve ser definido como um número inteiro maior, como 128. Usamos 10% dos exemplos de treinamento como o conjunto de validação para ajustar os hiperparâmetros.

```
batch_size = 4 if demo else 128
valid_ratio = 0.1
reorg_cifar10_data(data_dir, valid_ratio)
```

13.13.2 Aumento de Imagem

Para lidar com o *overfitting*, usamos o aumento da imagem. Por exemplo, adicionando `transforms.RandomFlipLeftRight()`, as imagens podem ser invertidas aleatoriamente. Também podemos realizar a normalização para os três canais RGB de imagens coloridas usando `transform.Normalize()`. Abaixo, listamos algumas dessas operações que você pode escolher para usar ou modificar, dependendo dos requisitos.

```
transform_train = torchvision.transforms.Compose([
    # Magnify the image to a square of 40 pixels in both height and width
    torchvision.transforms.Resize(40),
    # Randomly crop a square image of 40 pixels in both height and width to
    # produce a small square of 0.64 to 1 times the area of the original
    # image, and then shrink it to a square of 32 pixels in both height and
    # width
    torchvision.transforms.RandomResizedCrop(32, scale=(0.64, 1.0),
                                              ratio=(1.0, 1.0)),
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor(),
    # Normalize each channel of the image
    torchvision.transforms.Normalize([0.4914, 0.4822, 0.4465],
                                     [0.2023, 0.1994, 0.2010]))])
```

Para garantir a certeza da saída durante o teste, realizamos apenas normalização na imagem.

```
transform_test = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
```

(continues on next page)


```
torchvision.transforms.Normalize([0.4914, 0.4822, 0.4465],
                                 [0.2023, 0.1994, 0.2010]))
```

13.13.3 Lendo o Dataset

Em seguida, podemos criar a instância `ImageFolderDataset` para ler o conjunto de dados organizado contendo os arquivos de imagem originais, onde cada exemplo inclui a imagem e o rótulo.

```
train_ds, train_valid_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_train) for folder in ['train', 'train_valid']]

valid_ds, test_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_test) for folder in ['valid', 'test']]
```

Especificamos a operação de aumento de imagem definida em `DataLoader`. Durante o treinamento, usamos apenas o conjunto de validação para avaliar o modelo, portanto, precisamos garantir a certeza do resultado. Durante a previsão, treinaremos o modelo no conjunto de treinamento combinado e no conjunto de validação para fazer uso completo de todos os dados rotulados.

```
train_iter, train_valid_iter = [torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True, drop_last=True)
    for dataset in (train_ds, train_valid_ds)]

valid_iter = torch.utils.data.DataLoader(valid_ds, batch_size, shuffle=False,
                                         drop_last=True)

test_iter = torch.utils.data.DataLoader(test_ds, batch_size, shuffle=False,
                                         drop_last=False)
```

13.13.4 Definindo o Modelo

Aqui, construímos os blocos residuais com base na classe `HybridBlock`, que é ligeiramente diferente da implementação descrita em [Section 7.6](#). Isso é feito para melhorar a eficiência de execução.

A seguir, definimos o modelo ResNet-18.

O desafio de classificação de imagens CIFAR-10 usa 10 categorias. Vamos realizar a inicialização aleatória de Xavier no modelo antes do início do treinamento.

```
def get_net():
    num_classes = 10
    # PyTorch doesn't have the notion of hybrid model
    net = d2l.resnet18(num_classes, 3)
    return net

loss = nn.CrossEntropyLoss(reduction="none")
```


13.13.5 Definindo as Funções de Treinamento

Selecionaremos o modelo e ajustaremos os hiperparâmetros de acordo com o desempenho do modelo no conjunto de validação. Em seguida, definimos a função de treinamento do modelo treinar. Registramos o tempo de treinamento de cada época, o que nos ajuda a comparar os custos de tempo de diferentes modelos.

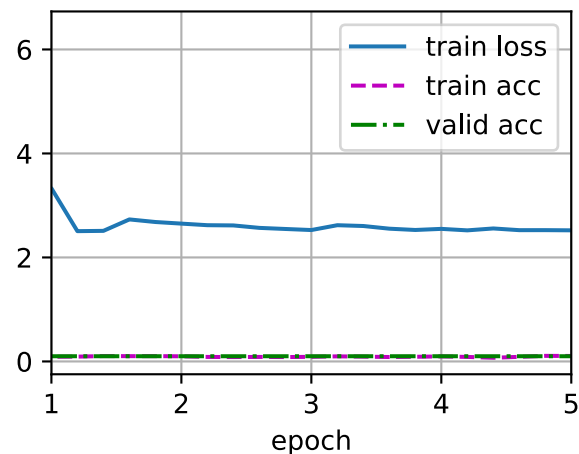
```
def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
         lr_decay):
    trainer = torch.optim.SGD(net.parameters(), lr=lr, momentum=0.9,
                              weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), d2l.Timer()
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                           legend=['train loss', 'train acc', 'valid acc'])
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        net.train()
        metric = d2l.Accumulator(3)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = d2l.train_batch_ch13(net, features, labels,
                                         loss, trainer, devices)
            metric.add(l, acc, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[2], metric[1] / metric[2],
                              None))
        if valid_iter is not None:
            valid_acc = d2l.evaluate_accuracy_gpu(net, valid_iter)
            animator.add(epoch + 1, (None, None, valid_acc))
        scheduler.step()
    if valid_iter is not None:
        print(f'loss {metric[0] / metric[2]:.3f}, '
              f'train acc {metric[1] / metric[2]:.3f}, '
              f'valid acc {valid_acc:.3f}')
    else:
        print(f'loss {metric[0] / metric[2]:.3f}, '
              f'train acc {metric[1] / metric[2]:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec '
          f'on {str(devices)}')
```

13.13.6 Treinamento e Validação do Modelo

Agora podemos treinar e validar o modelo. Os hiperparâmetros a seguir podem ser ajustados. Por exemplo, podemos aumentar o número de épocas. Como `lr_period` e `lr_decay` são definidos como 50 e 0,1 respectivamente, a taxa de aprendizado do algoritmo de otimização será multiplicada por 0,1 a cada 50 épocas. Para simplificar, treinamos apenas uma época aqui.

```
devices, num_epochs, lr, wd = d2l.try_all_gpus(), 5, 0.1, 5e-4
lr_period, lr_decay, net = 50, 0.1, get_net()
train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
      lr_decay)
```

```
loss 2.522, train acc 0.105, valid acc 0.100  
116.4 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```

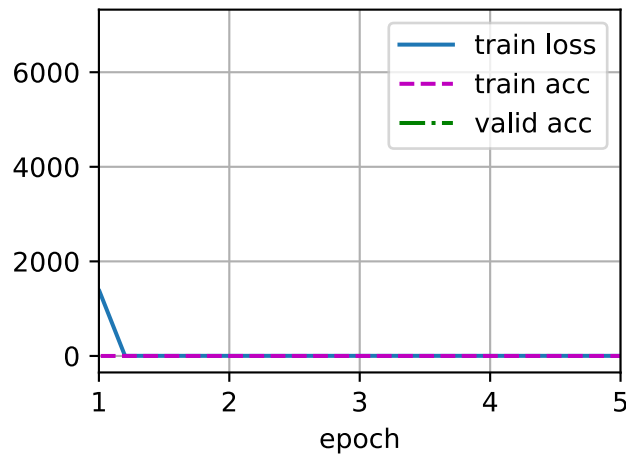


13.13.7 Classificando o Conjunto de Testes e Enviando Resultados no Kaggle

Depois de obter um design de modelo satisfatório e hiperparâmetros, usamos todos os conjuntos de dados de treinamento (incluindo conjuntos de validação) para treinar novamente o modelo e classificar o conjunto de teste.

```
net, preds = get_net(), []  
train(net, train_valid_iter, None, num_epochs, lr, wd, devices, lr_period,  
      lr_decay)  
  
for X, _ in test_iter:  
    y_hat = net(X.to(devices[0]))  
    preds.extend(y_hat.argmax(dim=1).type(torch.int32).cpu().numpy())  
sorted_ids = list(range(1, len(test_ds) + 1))  
sorted_ids.sort(key=lambda x: str(x))  
df = pd.DataFrame({'id': sorted_ids, 'label': preds})  
df['label'] = df['label'].apply(lambda x: train_valid_ds.classes[x])  
df.to_csv('submission.csv', index=False)
```

```
loss 2.501, train acc 0.097  
133.7 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



Após executar o código acima, obteremos um arquivo “submit.csv”. O formato deste arquivo é consistente com os requisitos da competição Kaggle. O método para enviar resultados é semelhante ao método em [Section 4.10](#).

13.13.8 Resumo

- Podemos criar uma instância `ImageFolderDataset` para ler o conjunto de dados contendo os arquivos de imagem originais.
- Podemos usar redes neurais convolucionais, aumento de imagens e programação híbrida para participar de uma competição de classificação de imagens.

13.13.9 Exercícios

1. Use o conjunto de dados CIFAR-10 completo para a competição Kaggle. Altere o `batch_size` e o número de épocas `num_epochs` para 128 e 100, respectivamente. Veja qual precisão e classificação você pode alcançar nesta competição.
2. Que precisão você pode alcançar quando não está usando o aumento de imagem?
3. Digitalize o código QR para acessar as discussões relevantes e trocar ideias sobre os métodos usados e os resultados obtidos com a comunidade. Você pode sugerir técnicas melhores?

Discussões¹⁷⁰

13.14 Identificação de Raça de Cachorro (*ImageNet Dogs*) no Kaggle

Nesta seção, abordaremos o desafio da identificação de raças de cães na Competição Kaggle. O endereço da competição na web é

<https://www.kaggle.com/c/dog-breed-identification>

¹⁷⁰ <https://discuss.d2l.ai/t/1479>

Nesta competição, tentamos identificar 120 raças diferentes de cães. O conjunto de dados usado nesta competição é, na verdade, um subconjunto do famoso conjunto de dados ImageNet. Diferente das imagens no conjunto de dados CIFAR-10 usado na seção anterior, as imagens no conjunto de dados ImageNet são mais altas e mais largas e suas dimensões são inconsistentes.

Fig. 13.14.1 mostra as informações na página da competição. Para enviar os resultados, primeiro registre uma conta no site do Kaggle.

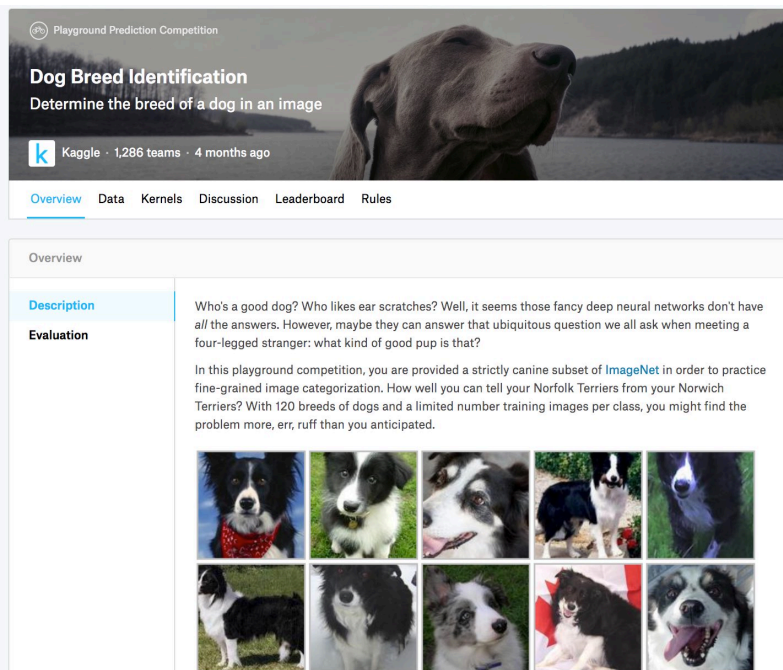


Fig. 13.14.1: Site de competição de identificação de raças de cães. O conjunto de dados da competição pode ser acessado clicando na guia “Data”.

Primeiro, importe os pacotes ou módulos necessários para a competição.

```
import os
import torch
import torchvision
from torch import nn
from d2l import torch as d2l
```

13.14.1 Obtenção e organização do Dataset

Os dados da competição são divididos em um conjunto de treinamento e um conjunto de teste. O conjunto de treinamento contém 10.222 imagens e o conjunto de teste contém 10.357 imagens. As imagens em ambos os conjuntos estão no formato JPEG. Essas imagens contêm três canais RGB (cores) e diferentes alturas e larguras. Existem 120 raças de cães no conjunto de treinamento, incluindo Labradores, Poodles, Dachshunds, Samoyeds, Huskies, Chihuahuas e Yorkshire Terriers.

Baixando o Dataset

Depois de fazer o login no Kaggle, podemos clicar na guia “Data” na página da competição de identificação de raças de cães mostrada em [Fig. 13.14.1](#) e baixar o conjunto de dados clicando no botão “Baixar tudo”. Após descompactar o arquivo baixado em `../data`, você encontrará todo o conjunto de dados nos seguintes caminhos:

- `../data/dog-breed-identification/labels.csv`
- `../data/dog-breed-identification/sample_submission.csv`
- `../data/dog-breed-identification/train`
- `../data/dog-breed-identification/test`

Você deve ter notado que a estrutura acima é bastante semelhante à da competição CIFAR-10 em [Section 13.13](#), onde as pastas `train/` e `test/` contêm imagens de treinamento e teste de cães, respectivamente, e rótulos. `csv` tem os rótulos das imagens de treinamento.

Da mesma forma, para facilitar o início, fornecemos uma amostra em pequena escala do conjunto de dados mencionado acima, “`train_valid_test_tiny.zip`”. Se você for usar o conjunto de dados completo para a competição Kaggle, você também precisará alterar a variável `demo` abaixo para `False`.

```
#@save
d2l.DATA_HUB['dog_tiny'] = (d2l.DATA_URL + 'kaggle_dog_tiny.zip',
                           '0cb91d09b814ecdc07b50f31f8dcad3e81d6a86d')

# If you use the full dataset downloaded for the Kaggle competition, change
# the variable below to False
demo = True
if demo:
    data_dir = d2l.download_extract('dog_tiny')
else:
    data_dir = os.path.join '..', 'data', 'dog-breed-identification')
```

Organizando o Dataset

Podemos organizar o conjunto de dados de forma semelhante ao que fizemos em [Section 13.13](#), nomeadamente separando um conjunto de validação do conjunto de treinamento e movendo imagens em subpastas agrupadas por rótulos.

A função `reorg_dog_data` abaixo é usada para ler os rótulos dos dados de treinamento, segmentar o conjunto de validação e organizar o conjunto de treinamento.

```
def reorg_dog_data(data_dir, valid_ratio):
    labels = d2l.read_csv_labels(os.path.join(data_dir, 'labels.csv'))
    d2l.reorg_train_valid(data_dir, labels, valid_ratio)
    d2l.reorg_test(data_dir)

batch_size = 4 if demo else 128
valid_ratio = 0.1
reorg_dog_data(data_dir, valid_ratio)
```

13.14.2 Aumento de Imagem

O tamanho das imagens nesta seção é maior do que as imagens na seção anterior. Aqui estão mais algumas operações de aumento de imagem que podem ser úteis.

```
transform_train = torchvision.transforms.Compose([
    # Randomly crop the image to obtain an image with an area of 0.08 to 1 of
    # the original area and height to width ratio between 3/4 and 4/3. Then,
    # scale the image to create a new image with a height and width of 224
    # pixels each
    torchvision.transforms.RandomResizedCrop(224, scale=(0.08, 1.0),
                                              ratio=(3.0/4.0, 4.0/3.0)),
    torchvision.transforms.RandomHorizontalFlip(),
    # Randomly change the brightness, contrast, and saturation
    torchvision.transforms.ColorJitter(brightness=0.4,
                                       contrast=0.4,
                                       saturation=0.4),

    # Add random noise
    torchvision.transforms.ToTensor(),
    # Standardize each channel of the image
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))
```

Durante o teste, usamos apenas operações de pré-processamento de imagens definidas.

```
transform_test = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    # Crop a square of 224 by 224 from the center of the image
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))
```

13.14.3 Lendo o Dataset

Como na seção anterior, podemos criar uma instância `ImageFolderDataset` para ler o conjunto de dados contendo os arquivos de imagem originais.

```
train_ds, train_valid_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_train) for folder in ['train', 'train_valid']]

valid_ds, test_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_test) for folder in ['valid', 'test']]
```

Aqui, criamos instâncias de `DataLoader`, assim como em [Section 13.13](#).

```
train_iter, train_valid_iter = [torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True, drop_last=True)
    for dataset in (train_ds, train_valid_ds)]

valid_iter = torch.utils.data.DataLoader(valid_ds, batch_size, shuffle=False,
```

(continues on next page)

```

drop_last=True)

test_iter = torch.utils.data.DataLoader(test_ds, batch_size, shuffle=False,
drop_last=False)

```

13.14.4 Definindo o Modelo

O conjunto de dados para esta competição é um subconjunto dos dados ImageNet. Portanto, podemos usar a abordagem discutida em [Section 13.2](#) para selecionar um modelo pré-treinado em todo o conjunto de dados ImageNet e usá-lo para extrair recursos de imagem a serem inseridos na rede de saída de pequena escala personalizada. A Gluon oferece uma ampla gama de modelos pré-treinados. Aqui, usaremos o modelo ResNet-34 pré-treinado. Como o conjunto de dados da competição é um subconjunto do conjunto de dados de pré-treinamento, simplesmente reutilizamos a entrada da camada de saída do modelo pré-treinado, ou seja, os recursos extraídos. Então, podemos substituir a camada de saída original por uma pequena camada personalizada de rede de saída que pode ser treinada, como duas camadas totalmente conectadas em uma série. Diferente da experiência em [Section 13.2](#), aqui, não retreinamos o modelo pré-treinado usado para extração de características. Isso reduz o tempo de treinamento e a memória necessária para armazenar gradientes de parâmetro do modelo.

Você deve notar que, durante o aumento da imagem, usamos os valores médios e desvios padrão dos três canais RGB para todo o conjunto de dados ImageNet para normalização. Isso é consistente com a normalização do modelo pré-treinado.

```

def get_net(devices):
    finetune_net = nn.Sequential()
    finetune_net.features = torchvision.models.resnet34(pretrained=True)
    # Define a new output network
    # There are 120 output categories
    finetune_net.output_new = nn.Sequential(nn.Linear(1000, 256),
                                           nn.ReLU(),
                                           nn.Linear(256, 120))

    # Move model to device
    finetune_net = finetune_net.to(devices[0])
    # Freeze feature layer params
    for param in finetune_net.features.parameters():
        param.requires_grad = False
    return finetune_net

```

Ao calcular a perda, primeiro usamos a variável-membro `features` para obter a entrada da camada de saída do modelo pré-treinado, ou seja, a característica extraída. Em seguida, usamos essa característica como a entrada para nossa pequena rede de saída personalizada e calculamos a saída.

```

loss = nn.CrossEntropyLoss(reduction='none')

def evaluate_loss(data_iter, net, devices):
    l_sum, n = 0.0, 0
    for features, labels in data_iter:
        features, labels = features.to(devices[0]), labels.to(devices[0])

```

(continues on next page)


```

    outputs = net(features)
    l = loss(outputs, labels)
    l_sum = l.sum()
    n += labels.numel()
    return l_sum / n

```

13.14.5 Definindo as Funções de Treinamento

Selecionaremos o modelo e ajustaremos os hiperparâmetros de acordo com o desempenho do modelo no conjunto de validação. A função de treinamento do modelo treinar apenas treina a pequena rede de saída personalizada.

```

def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
          lr_decay):
    # Only train the small custom output network
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.SGD((param for param in net.parameters()
                                if param.requires_grad), lr=lr,
                               momentum=0.9, weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), d2l.Timer()
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                            legend=['train loss', 'valid loss'])
    for epoch in range(num_epochs):
        metric = d2l.Accumulator(2)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            features, labels = features.to(devices[0]), labels.to(devices[0])
            trainer.zero_grad()
            output = net(features)
            l = loss(output, labels).sum()
            l.backward()
            trainer.step()
            metric.add(l, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[1], None))
        if valid_iter is not None:
            valid_loss = evaluate_loss(valid_iter, net, devices)
            animator.add(epoch + 1, (None, valid_loss))
        scheduler.step()
    if valid_iter is not None:
        print(f'train loss {metric[0] / metric[1]:.3f}, '
              f'valid loss {valid_loss:.3f}')
    else:
        print(f'train loss {metric[0] / metric[1]:.3f}')
    print(f'{metric[1] * num_epochs / timer.sum():.1f} examples/sec '
          f'on {str(devices)}')

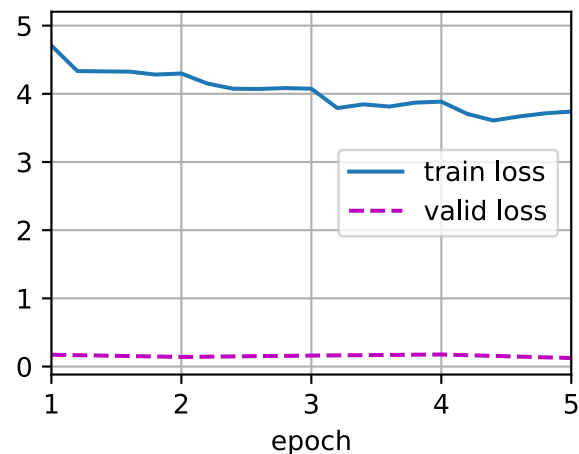
```

13.14.6 Treinamento e Validação do Modelo

Agora podemos treinar e validar o modelo. Os hiperparâmetros a seguir podem ser ajustados. Por exemplo, podemos aumentar o número de épocas. Como `lr_period` e `lr_decay` são definidos como 10 e 0,1 respectivamente, a taxa de aprendizado do algoritmo de otimização será multiplicada por 0,1 a cada 10 épocas.

```
devices, num_epochs, lr, wd = d2l.try_all_gpus(), 5, 0.001, 1e-4
lr_period, lr_decay, net = 10, 0.1, get_net(devices)
train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
      lr_decay)
```

```
train loss 3.740, valid loss 0.125
107.6 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



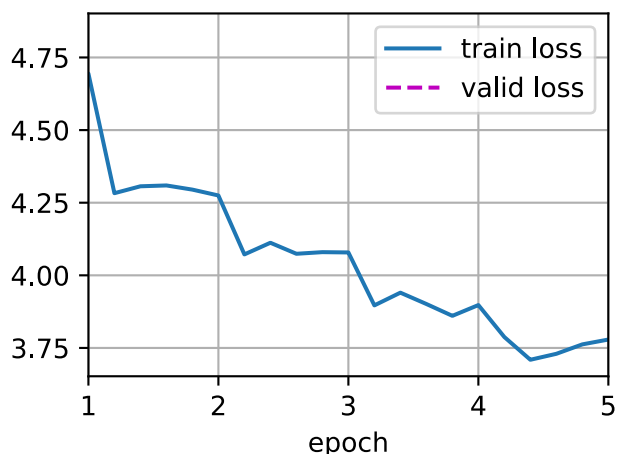
13.14.7 Classificando o Conjunto de Testes e Enviando Resultados no Kaggle

Depois de obter um design de modelo satisfatório e hiperparâmetros, usamos todos os conjuntos de dados de treinamento (incluindo conjuntos de validação) para treinar novamente o modelo e, em seguida, classificar o conjunto de teste. Observe que as previsões são feitas pela rede de saída que acabamos de treinar.

```
net = get_net(devices)
train(net, train_valid_iter, None, num_epochs, lr, wd, devices, lr_period,
      lr_decay)

preds = []
for data, label in test_iter:
    output = torch.nn.functional.softmax(net(data.to(devices[0])), dim=0)
    preds.extend(output.cpu().detach().numpy())
ids = sorted(os.listdir(
    os.path.join(data_dir, 'train_valid_test', 'test', 'unknown')))
with open('submission.csv', 'w') as f:
    f.write('id,' + ','.join(train_valid_ds.classes) + '\n')
    for i, output in zip(ids, preds):
        f.write(i.split('.')[0] + ',' + ','.join(
            [str(num) for num in output]) + '\n')
```

```
train loss 3.779
126.4 examples/sec on [device(type='cuda', index=0), device(type='cuda', index=1)]
```



Após executar o código acima, geraremos um arquivo “submit.csv”. O formato deste arquivo é consistente com os requisitos da competição Kaggle. O método para enviar resultados é semelhante ao método em [Section 4.10](#).

13.14.8 Resumo

- Podemos usar um modelo pré-treinado no conjunto de dados ImageNet para extrair recursos e treinar apenas uma pequena rede de saída personalizada. Isso nos permitirá classificar um subconjunto do conjunto de dados ImageNet com menor sobrecarga de computação e armazenamento.

13.14.9 Exercícios

1. Ao usar todo o conjunto de dados Kaggle, que tipo de resultados você obtém ao aumentar `batch_size` (tamanho do lote) `num_epochs` (número de épocas)?
2. Você obtém melhores resultados se usar um modelo pré-treinado mais profundo?
3. Digitalize o código QR para acessar as discussões relevantes e trocar ideias sobre os métodos usados e os resultados obtidos com a comunidade. Você pode sugerir técnicas melhores?

Discussões¹⁷¹

¹⁷¹ <https://discuss.d2l.ai/t/1481>